MightyADC™ - XDDoS + L4/L7 LB + AI Gateway

- L4 Load Balancer with DDoS Mitigation
- L7 Load Balancer with DDoS Mitigation
- **AI Gateway with DDoS Mitigation**

MightyLink Lab.
(www.mightyconn. com)
Doc. Revision: 0.96

Table of Contents

1. DDoS Mitigator - XDDoS
2. L4/L7 Load Balancer – L2DSR/L3DSR/L3NAT, HAProxy
3. AI Gateway – LLM/MCP/A2A
4. MightyADC™ - XDDoS + L4/L7 LB + AI Gateway

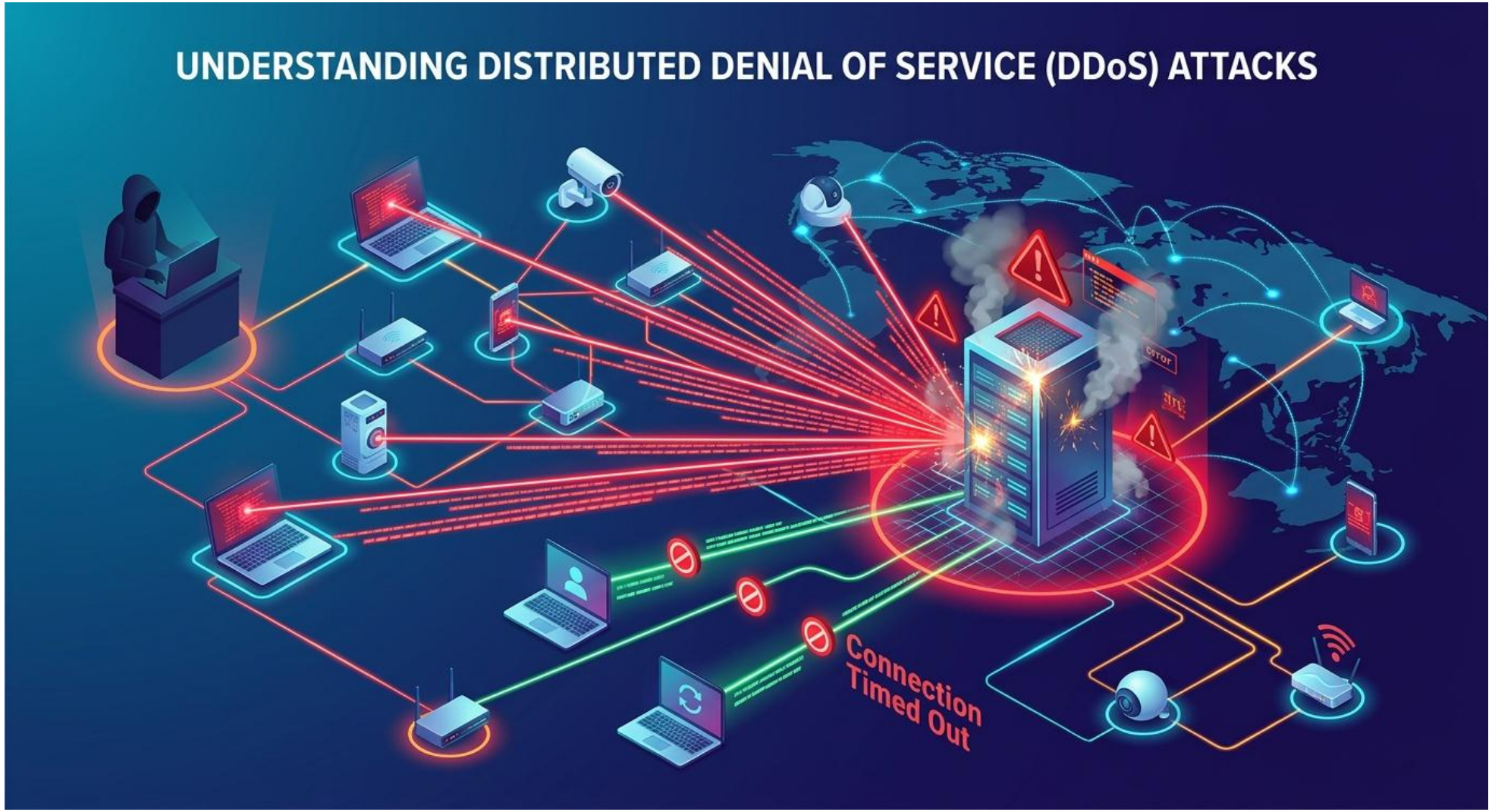
Appendix



1. DDoS Mitigator - XDDoS

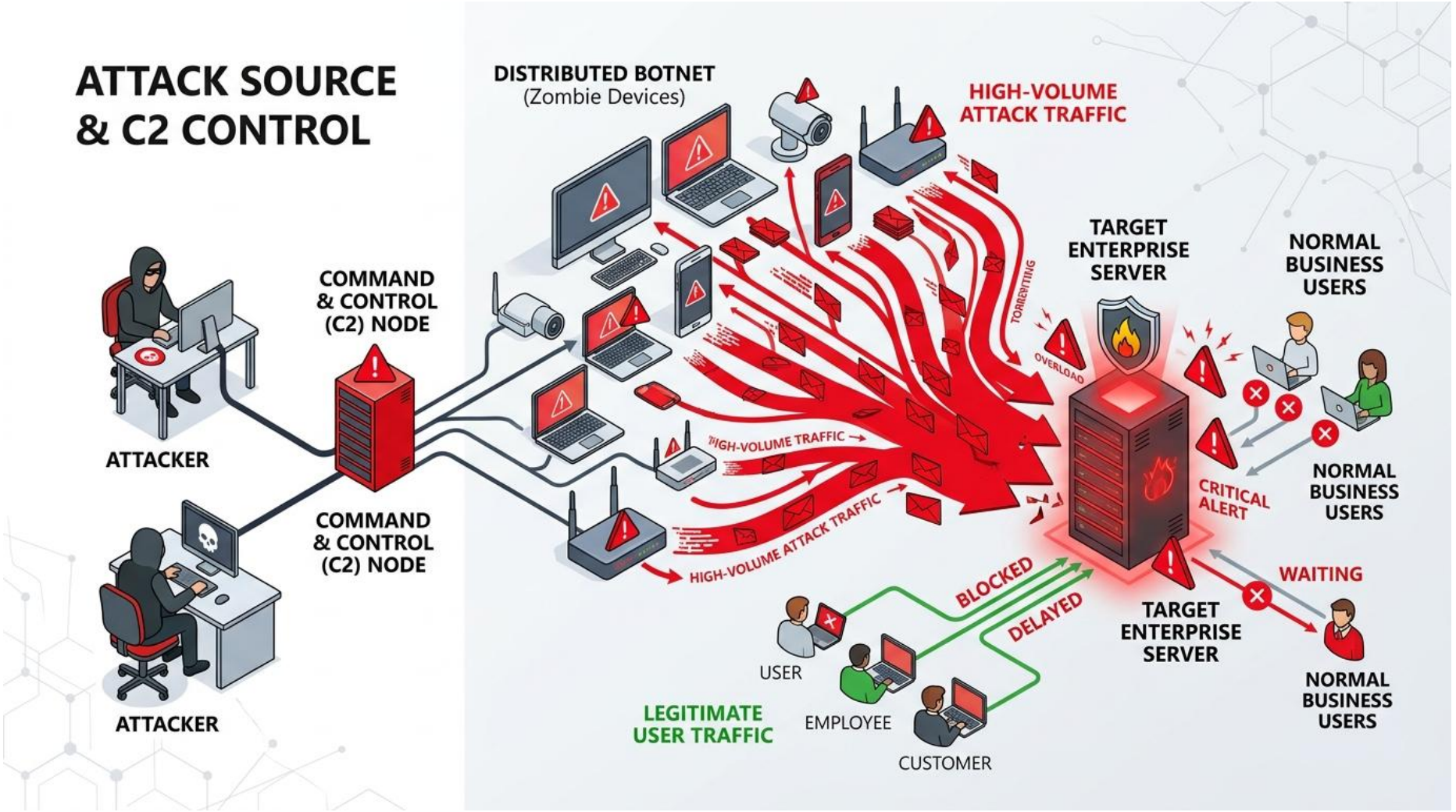


1. DDoS Mitigator(1) - DDoS Attack(1)



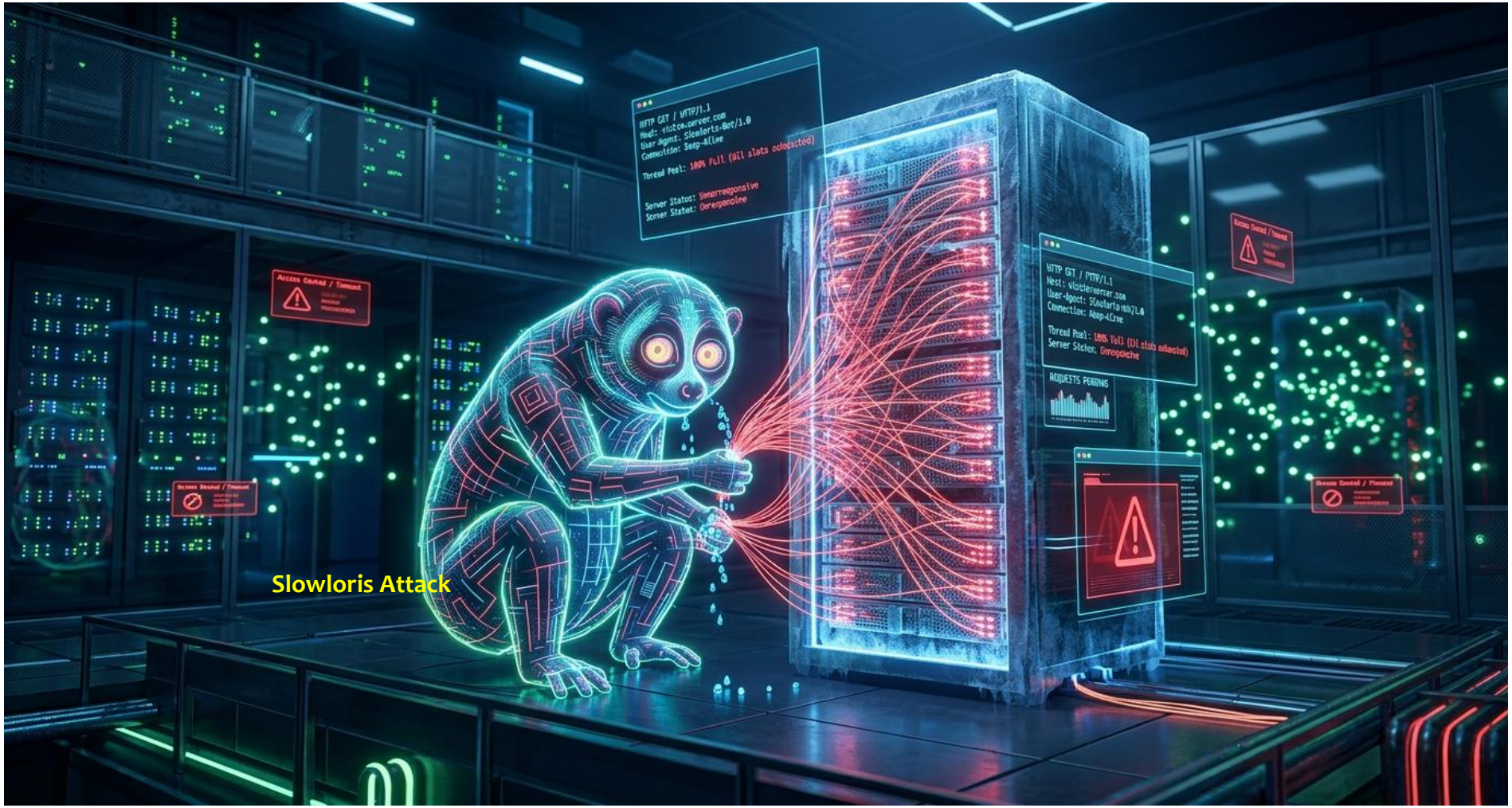
Generated by Gemini

1. DDoS Mitigator(1) - DDoS Attack(2)



Generated by Gemini

1. DDoS Mitigator(1) - DDoS Attack(3)



Generated by Gemini

1. DDoS Mitigator(1) - DDoS Attack(4)

Types of DDoS Attacks

Volumetric attacks(L3/L4)

- Flooding the network with a large volume of data to consume available bandwidth..

  Ex) UDP Flood, ICMP Flood(Ping Flood), DNS amplification attack

- **Protocol attacks(L3/L4)**
  - Exhausting server or firewall resources by exploiting vulnerabilities in communication protocols..

    Ex) SYN Flood, Ping of Death, Smurf Attack, Fragmentation Packet Attack

- **Application layer attacks(L7)**
  - Mimics the behavior of legitimate users to target specific applications and evade detection..

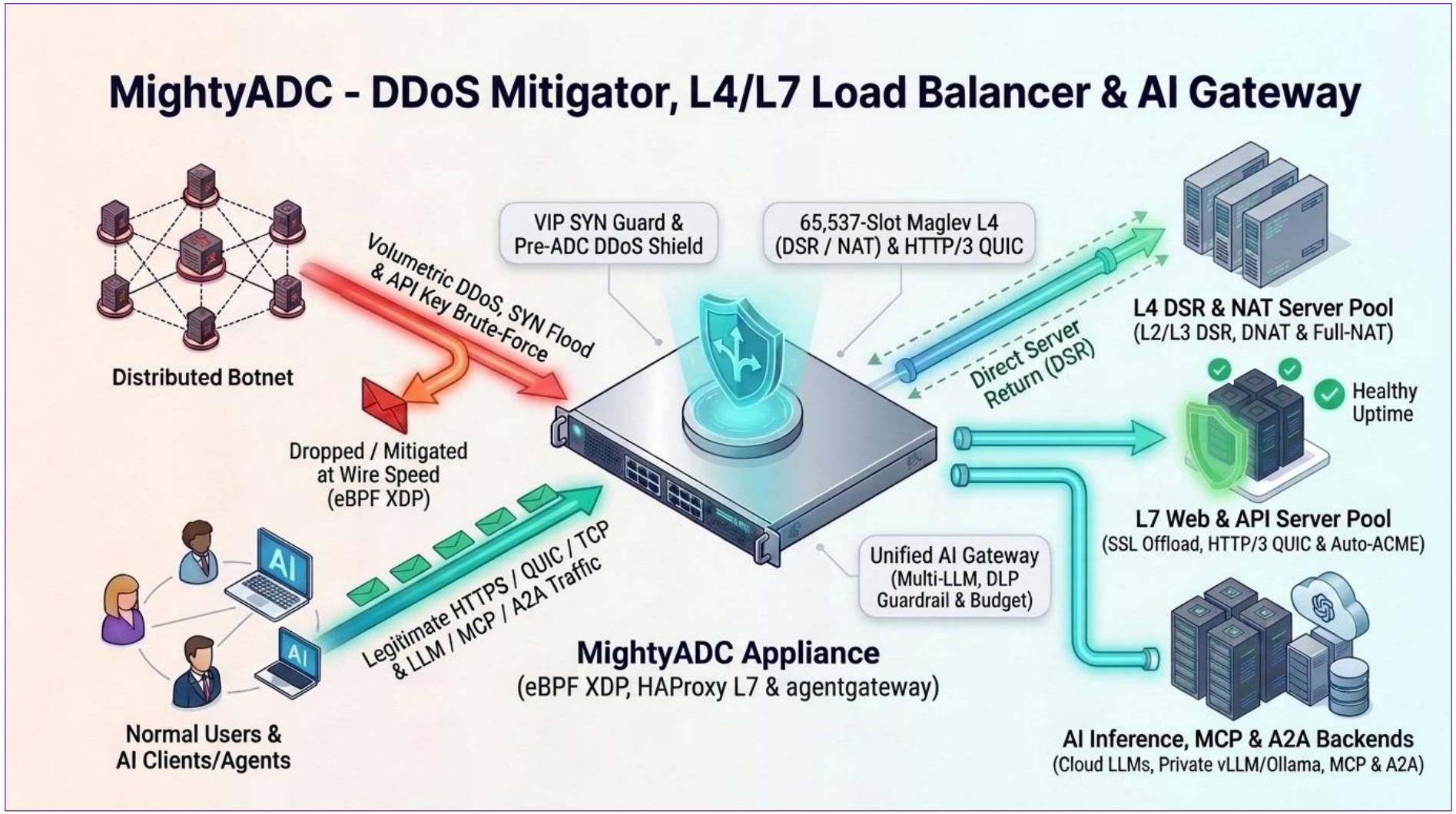
    Ex) HTTP GET/POST Flood, Slowloris Attack, XML-RPC Flood, WordPress Administrator Login Flood

1. DDoS Mitigator(2) – Detector & Mitigator(1)



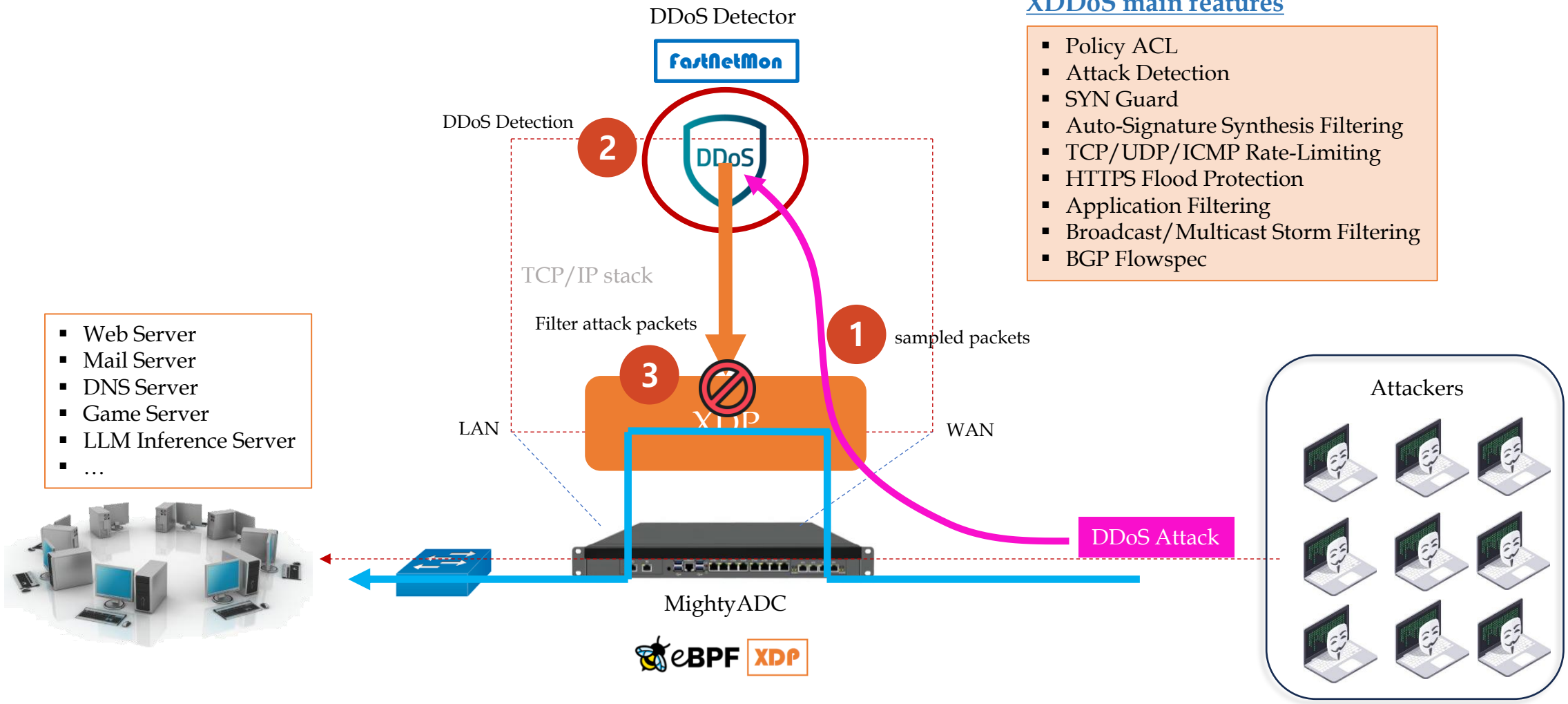
Generated by Gemini

1. DDoS Mitigator(2) - Detector & Mitigator(2)



Generated by Gemini

1. DDoS Mitigator(3) – eBPF XDP Mitigator(1)

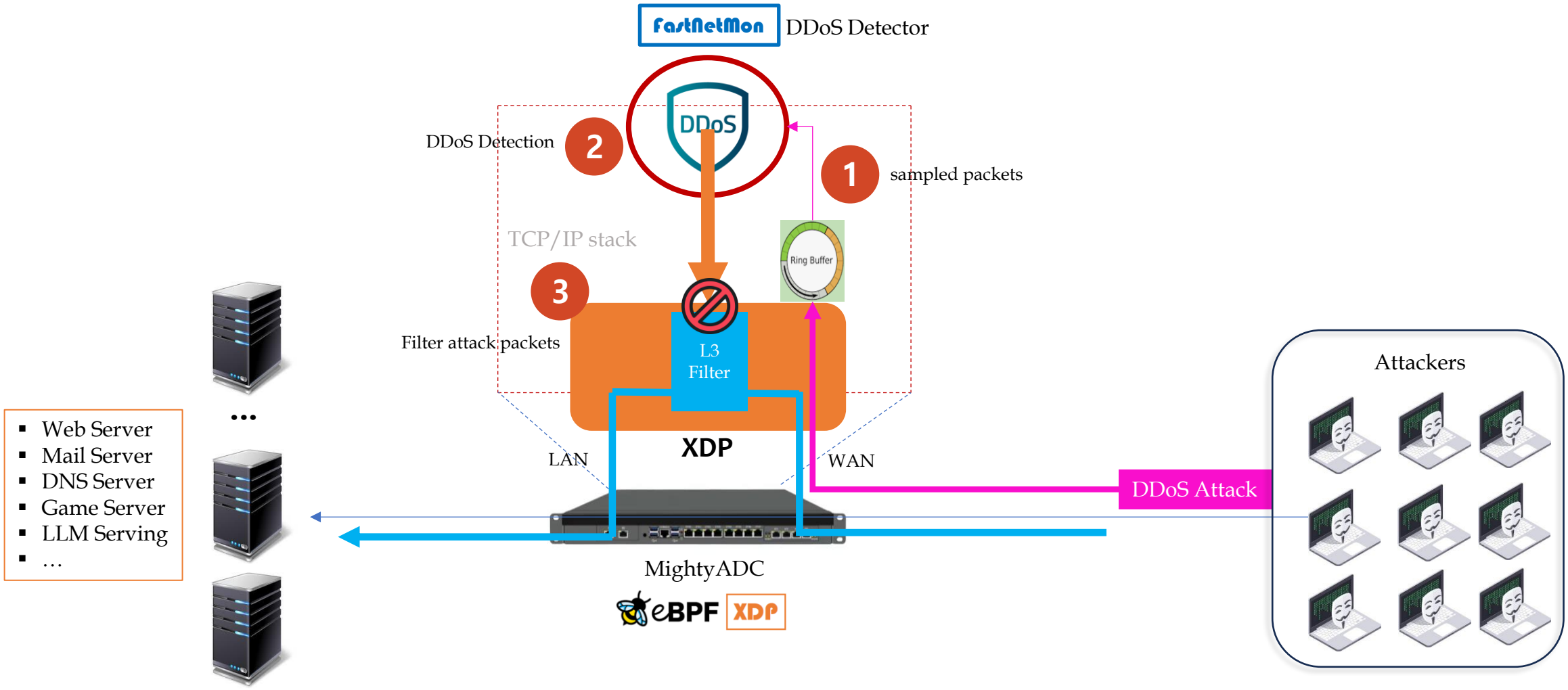


XDDoS main features

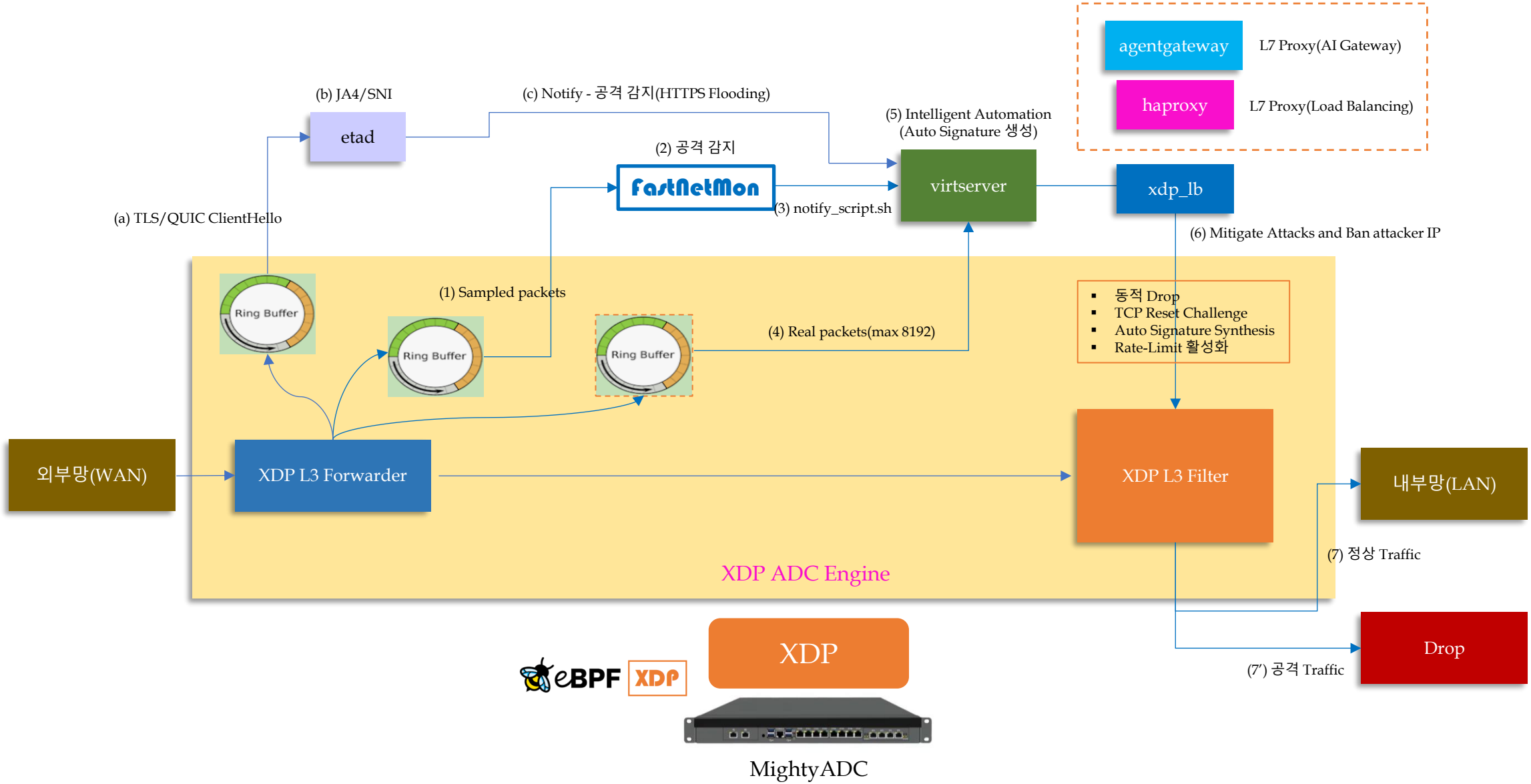
- Policy ACL
- Attack Detection
- SYN Guard
- Auto-Signature Synthesis Filtering
- TCP/UDP/ICMP Rate-Limiting
- HTTPS Flood Protection
- Application Filtering
- Broadcast/Multicast Storm Filtering
- BGP Flowspec

- Web Server
- Mail Server
- DNS Server
- Game Server
- LLM Inference Server
- …

1. DDoS Mitigator(3) – eBPF XDP Mitigator(2)



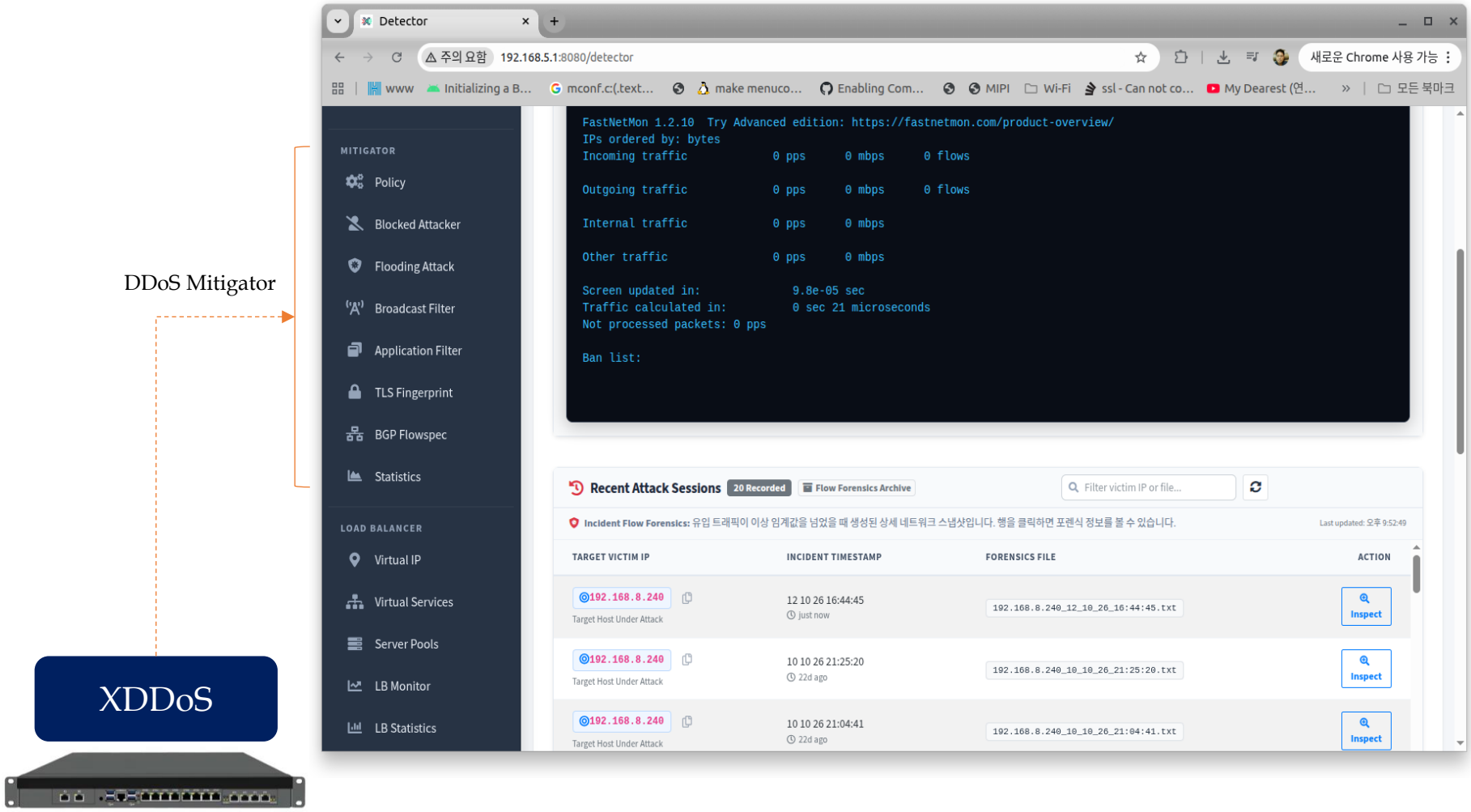
1. DDoS Mitigator(3) – eBPF XDP Mitigator(3)



1. DDoS Mitigator(4) – FastNetMon Detector(1)

Detector Page

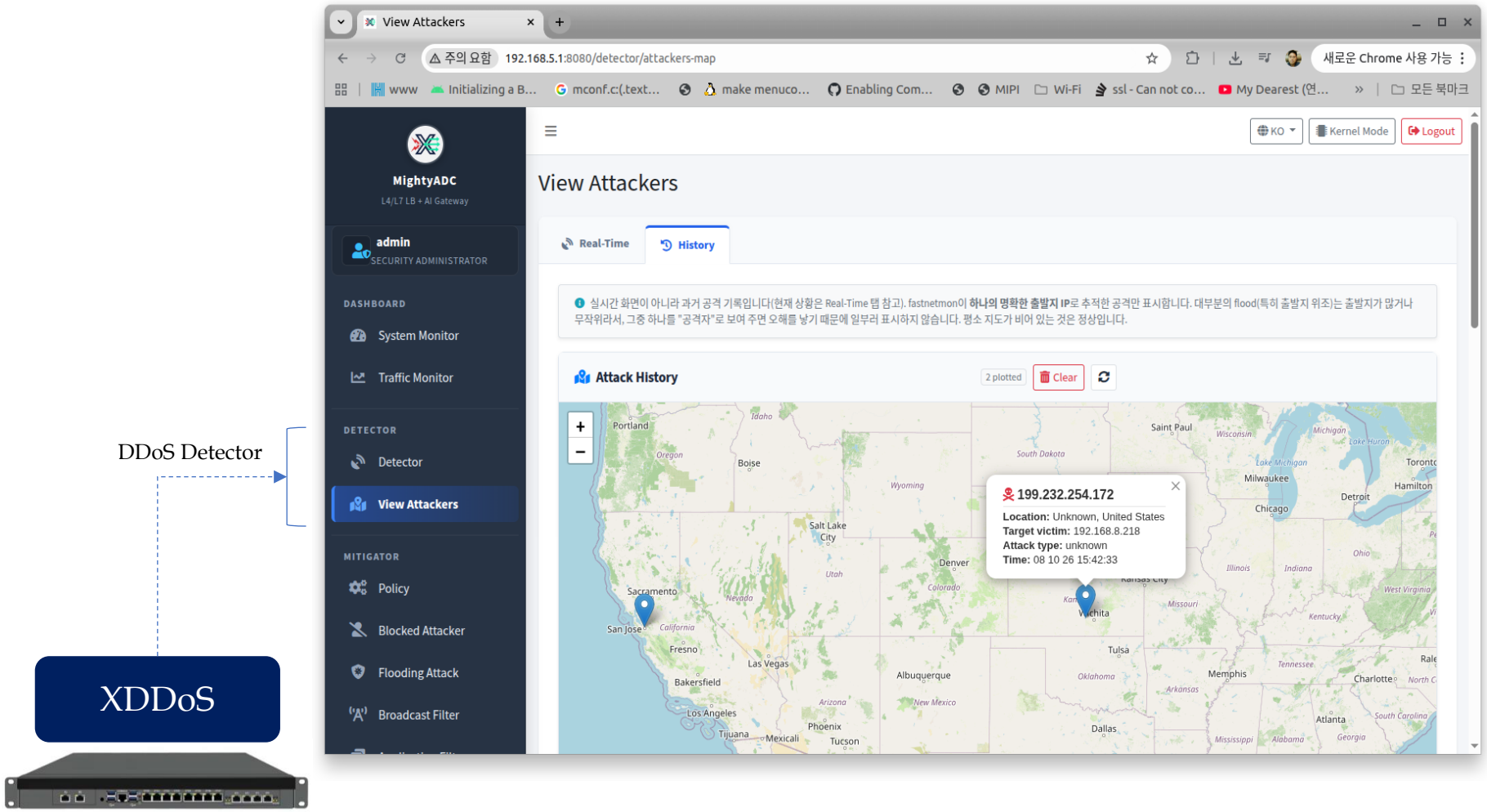
DDoS Mitigator



XDDoS

1. DDoS Mitigator(4) – FastNetMon Detector(2)

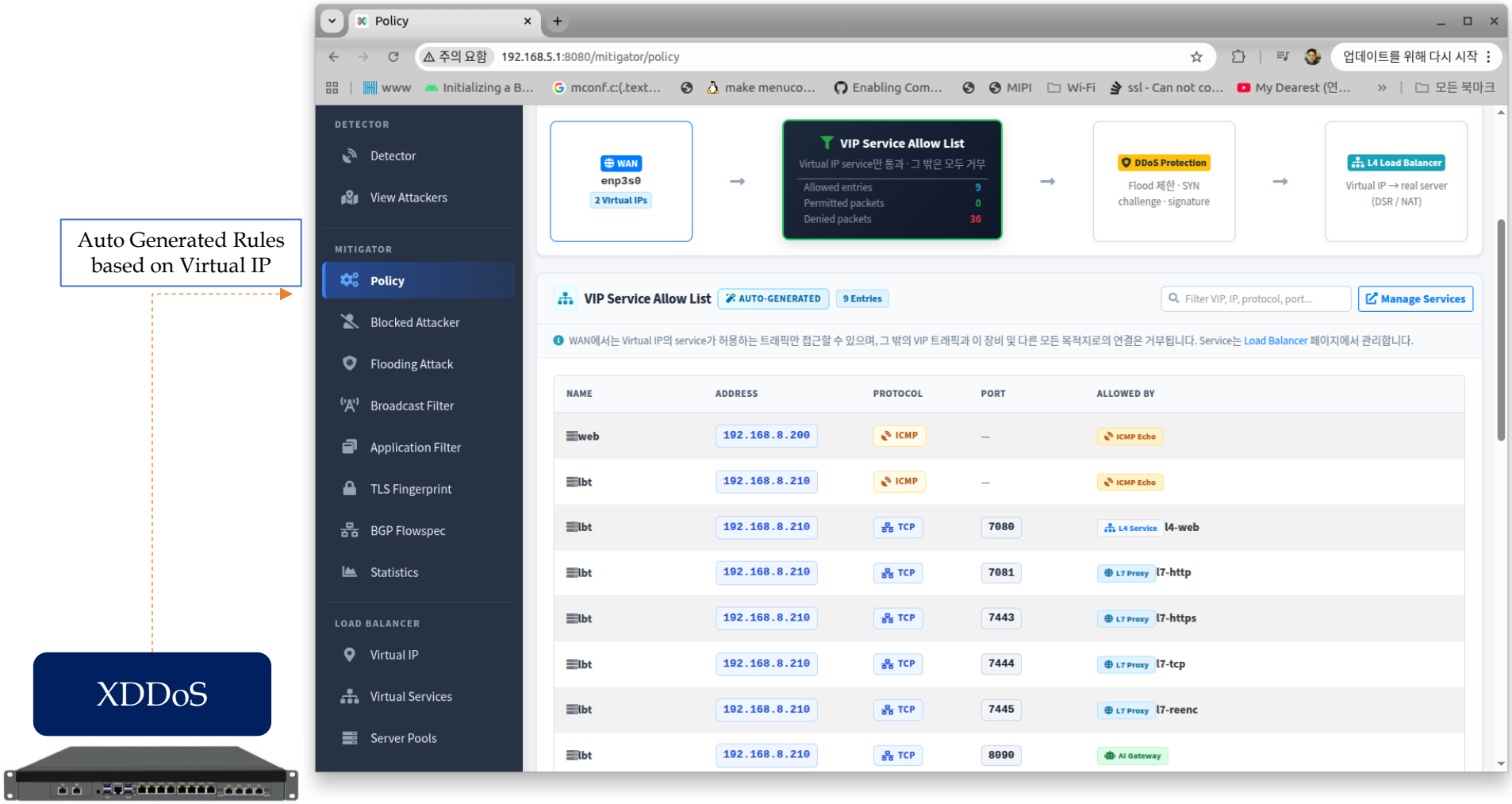
Detector -> View Attackers Page



1. DDoS Mitigator(5) – Policy ACL(1)

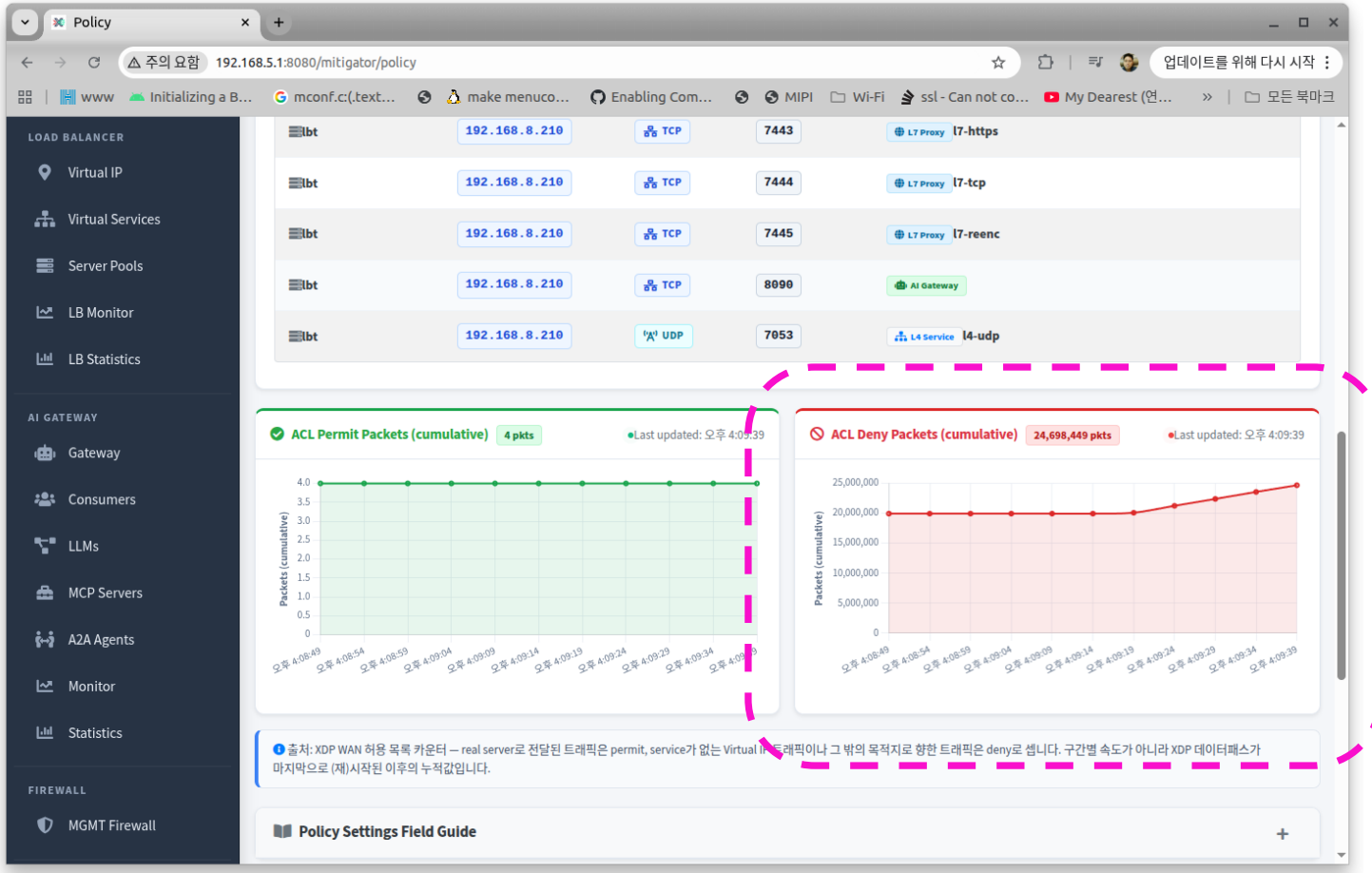
Mitigator Policy Page(1)

Auto Generated Rules based on Virtual IP



1. DDoS Mitigator(5) – Policy ACL(2)

Mitigator Policy Page(2)

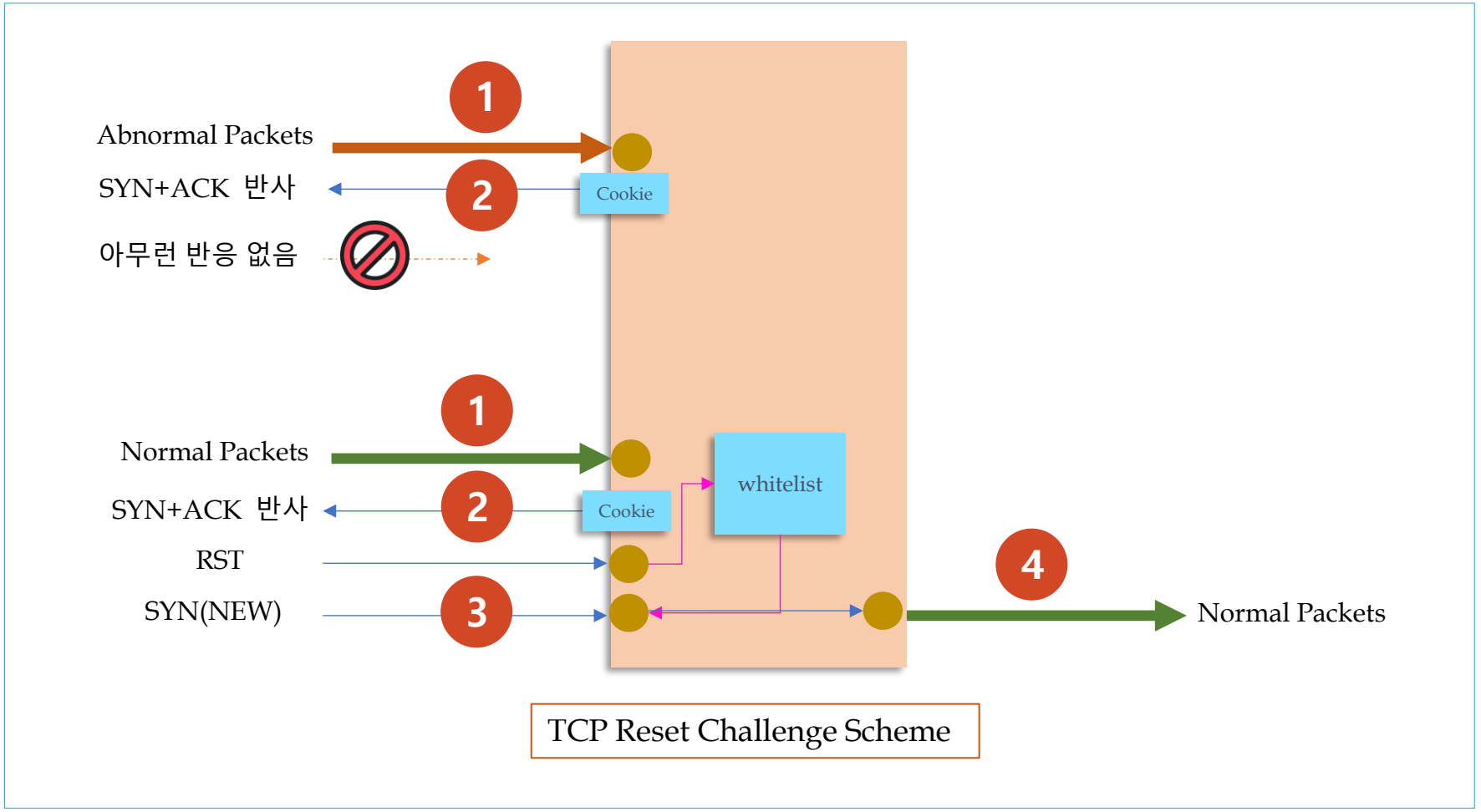




Unauthorized packets are forcibly blocked.

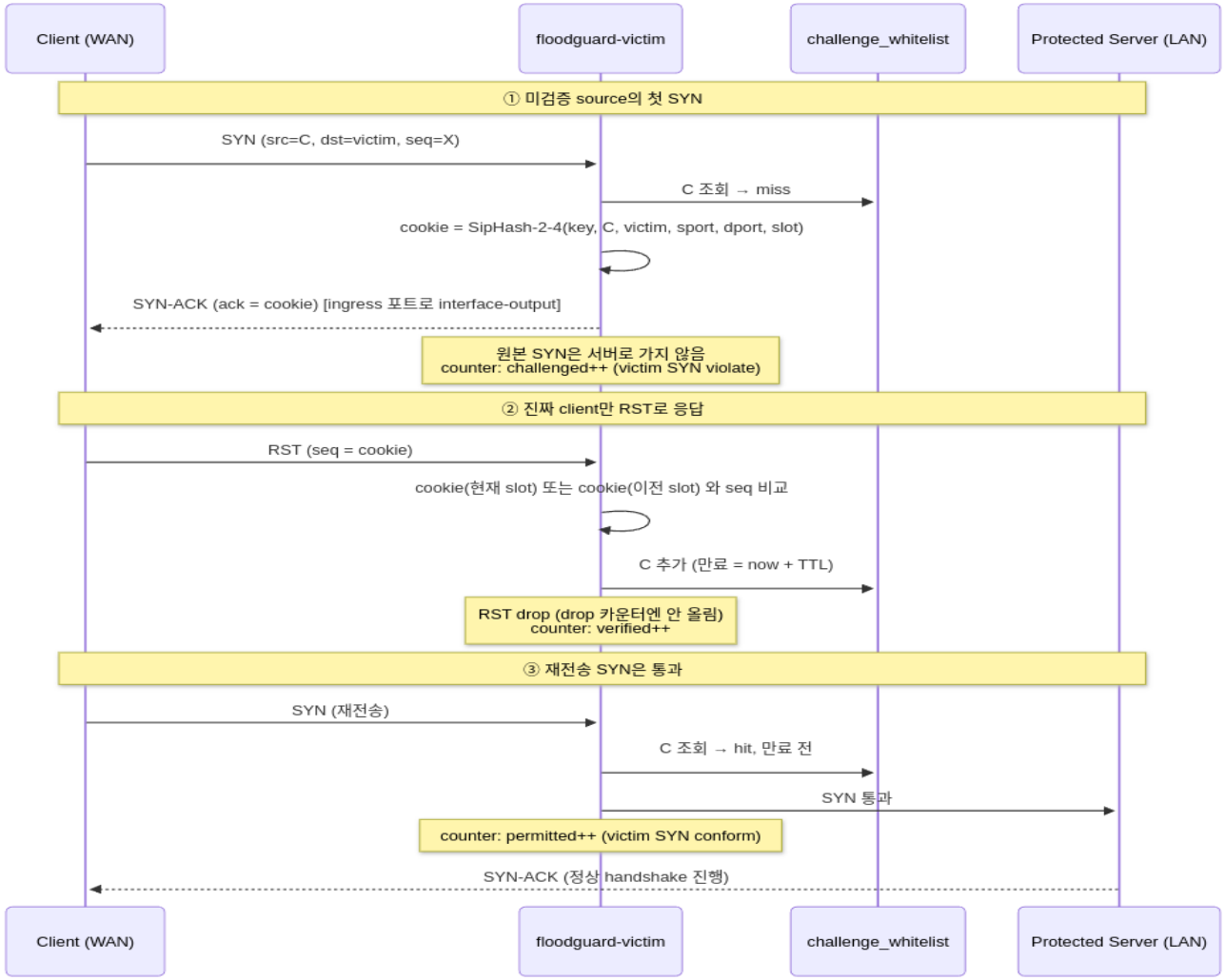
1. DDoS Mitigator(6) – VIP SYN Guard(1)

Attack 초기 단계 공격 패킷 차단, 정상 TCP 접속만 선별 후 통과



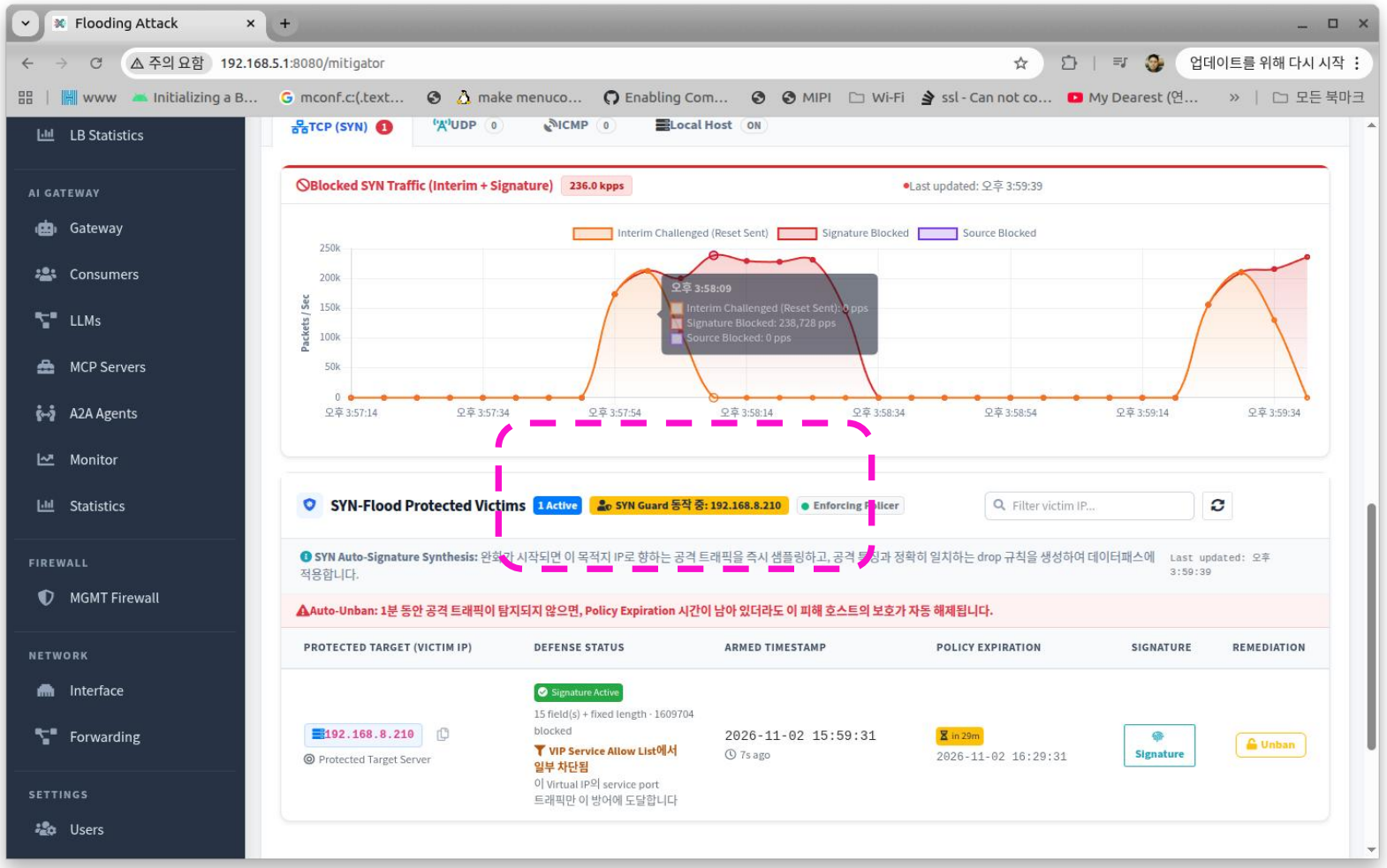
1. DDoS Mitigator(6) – VIP SYN Guard(2)



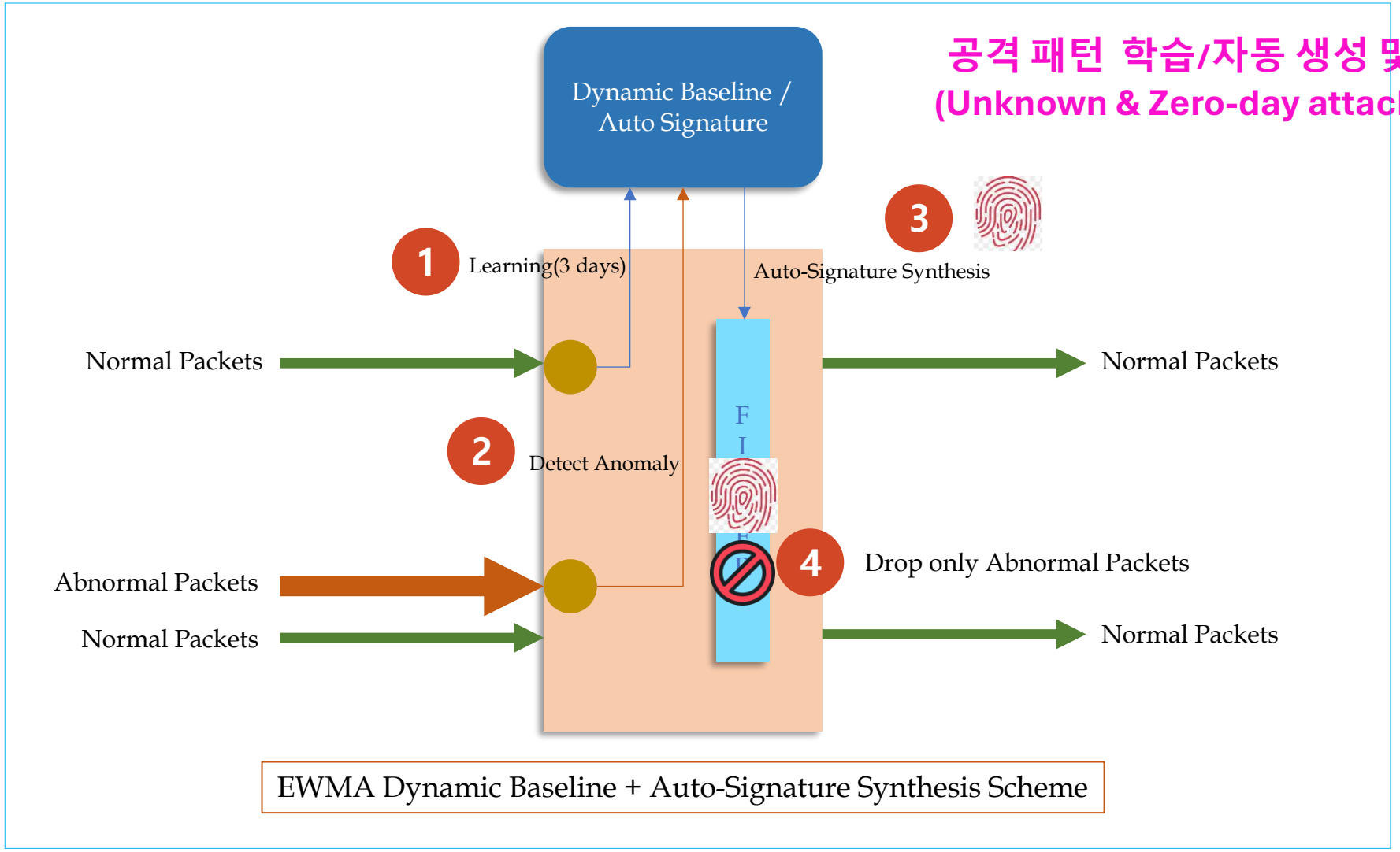


1. DDoS Mitigator(6) – VIP SYN Guard(3)

Mitigator Flooding Attack -> TCP(SYN) Page

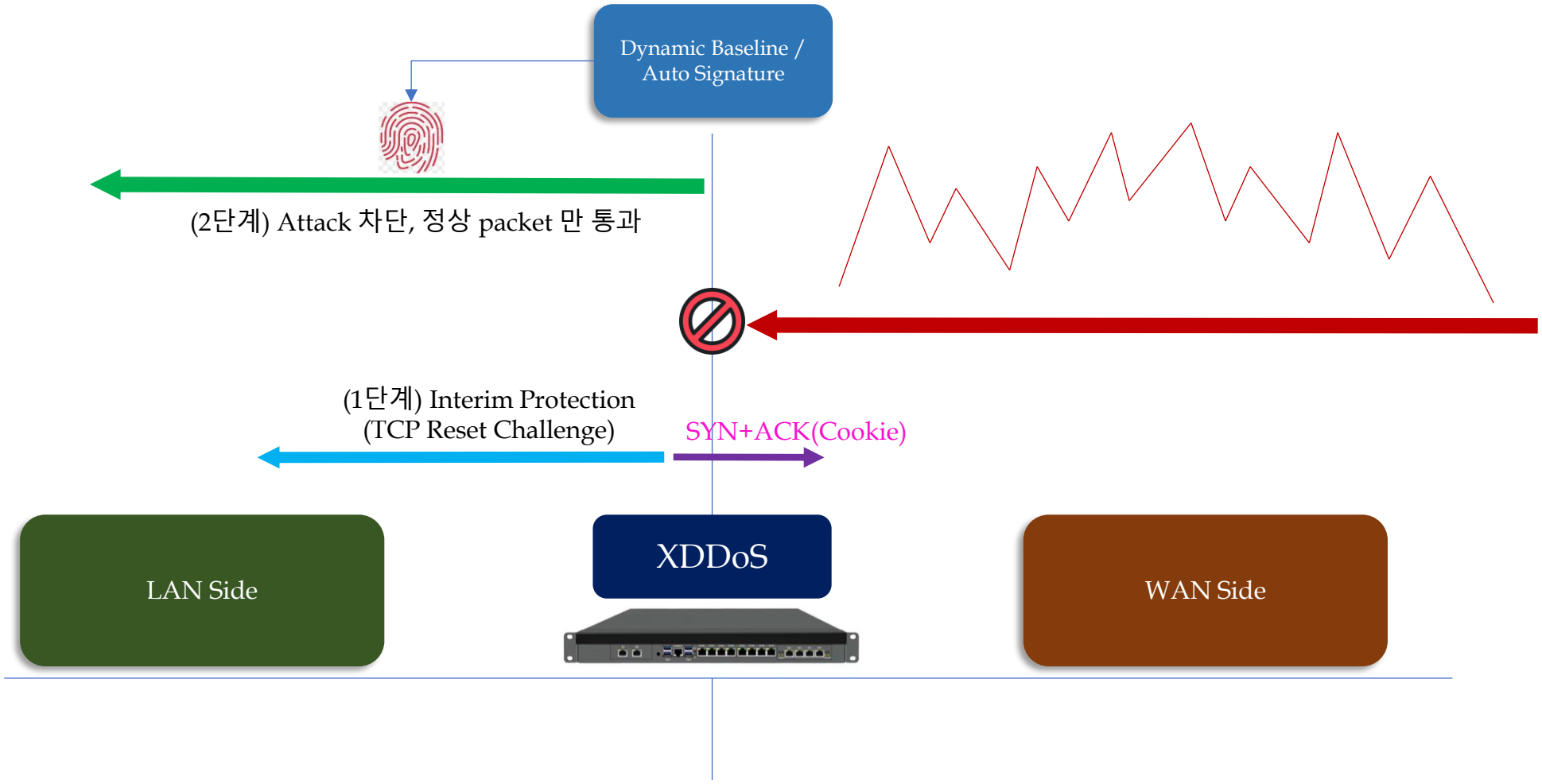


1. DDoS Mitigator(7) – Auto-Signature Synthesis(1)



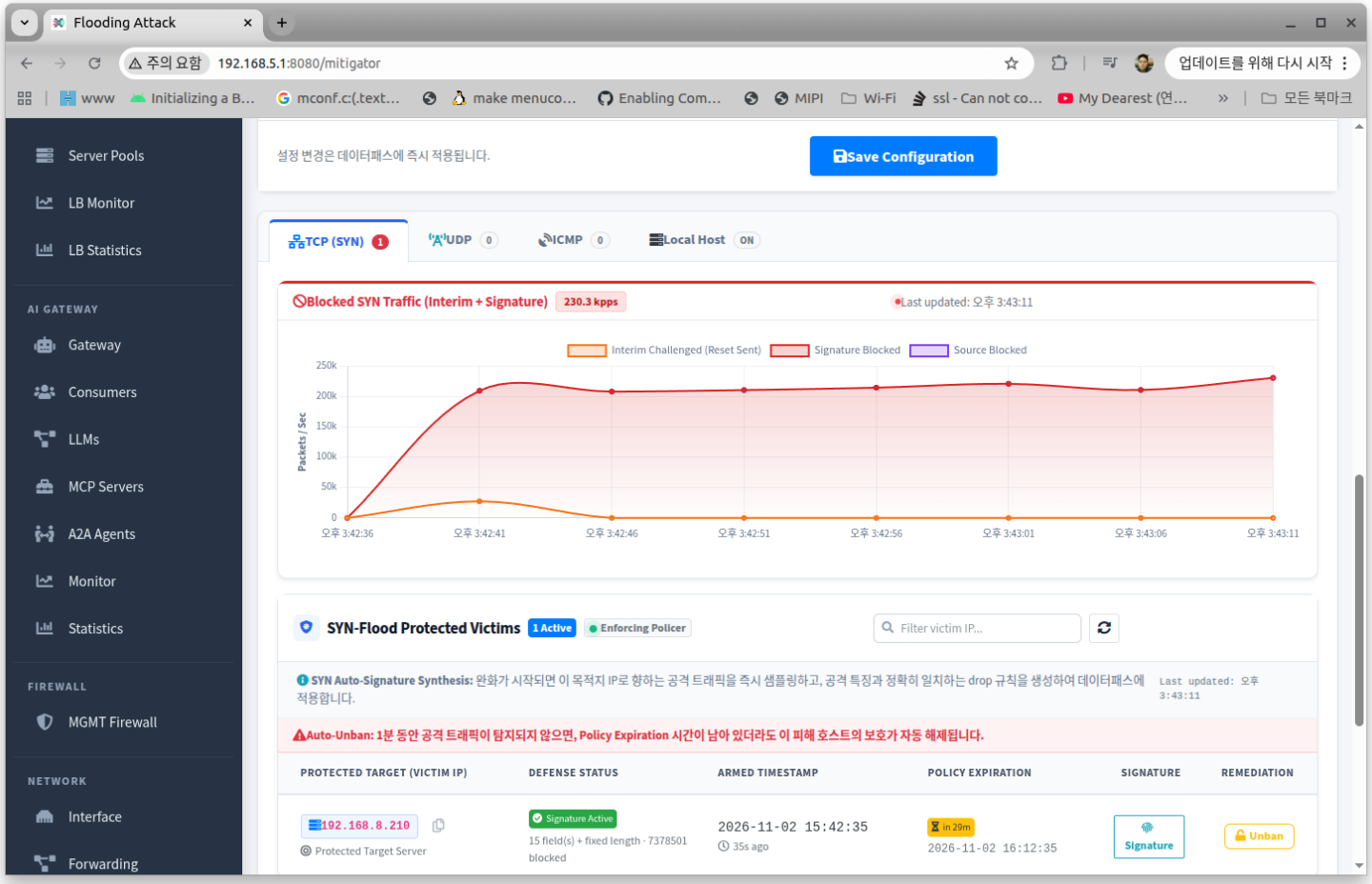
1. DDoS Mitigator(7) – Auto-Signature Synthesis(2)

Interim(TCP Reset Challenge) + Auto-Signature Synthesis Method



1. DDoS Mitigator(7) – Auto-Signature Synthesis(3)

Mitigator Flooding Attack -> TCP(SYN) Page



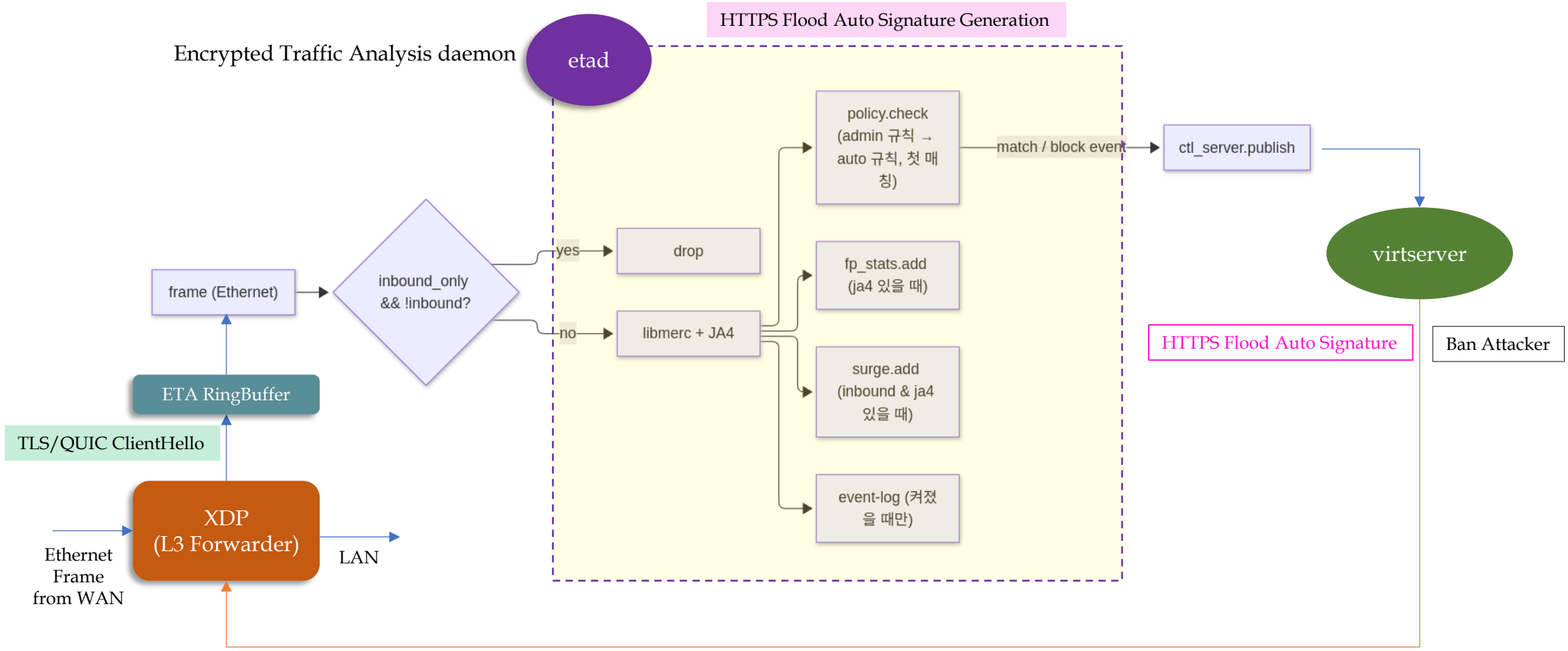
1. DDoS Mitigator(8) – Broadcast/Multicast Storm Filter

Mitigator Broadcast Filter -> Multicast Storm Telemetry Page



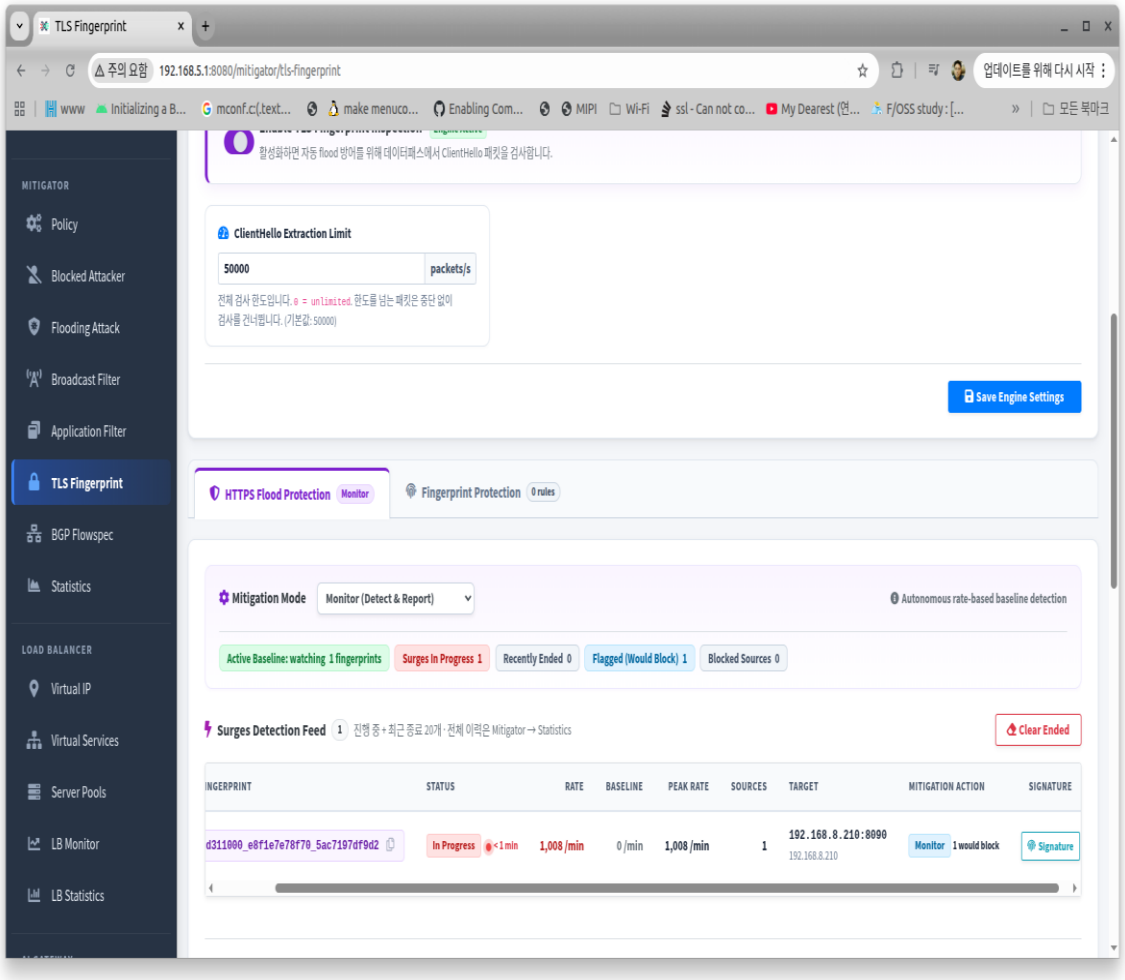
Multicast storm packets are forcibly blocked.

1. DDoS Mitigator(9) – HTTPS Flood Protection(1)

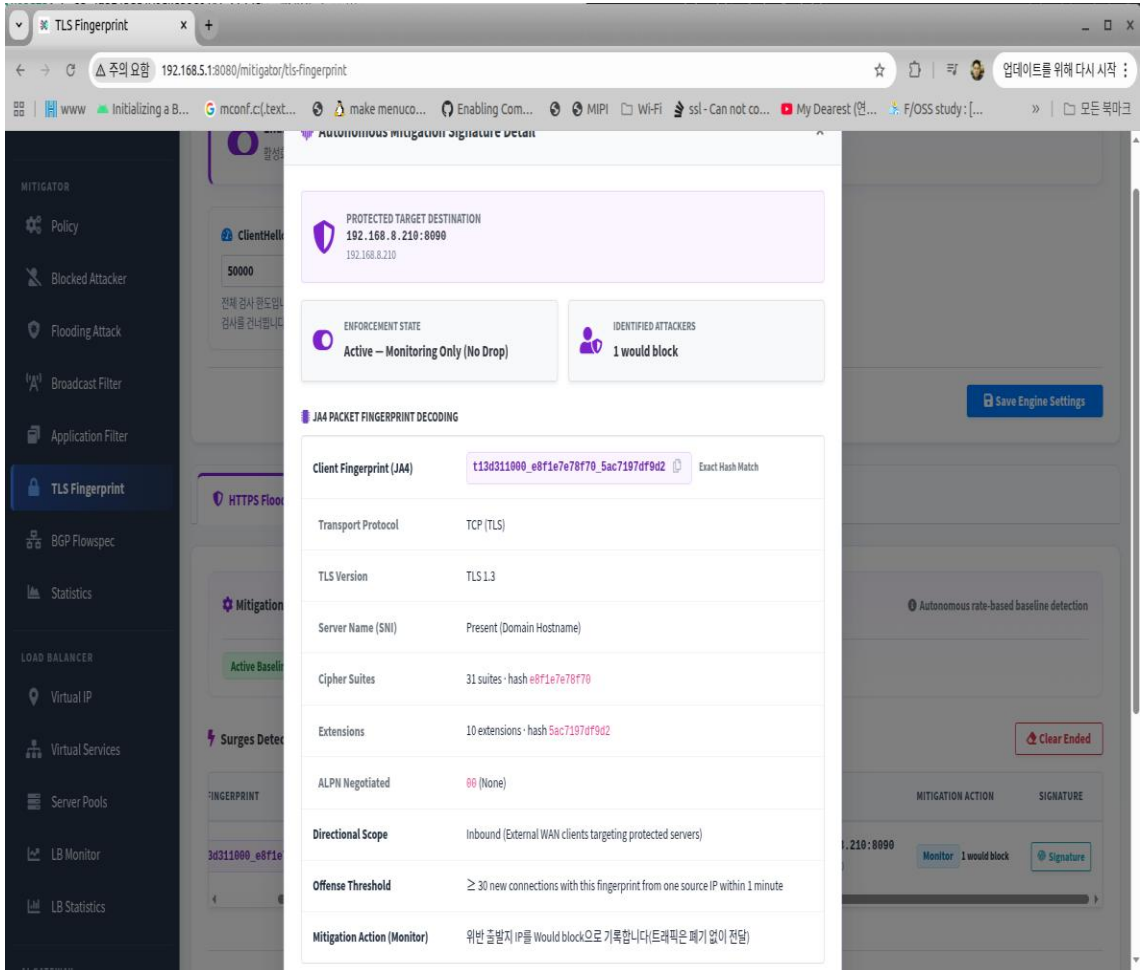


1. DDoS Mitigator(9) – HTTPS Flood Protection(2)

Mitigator TLS Fingerprint Page

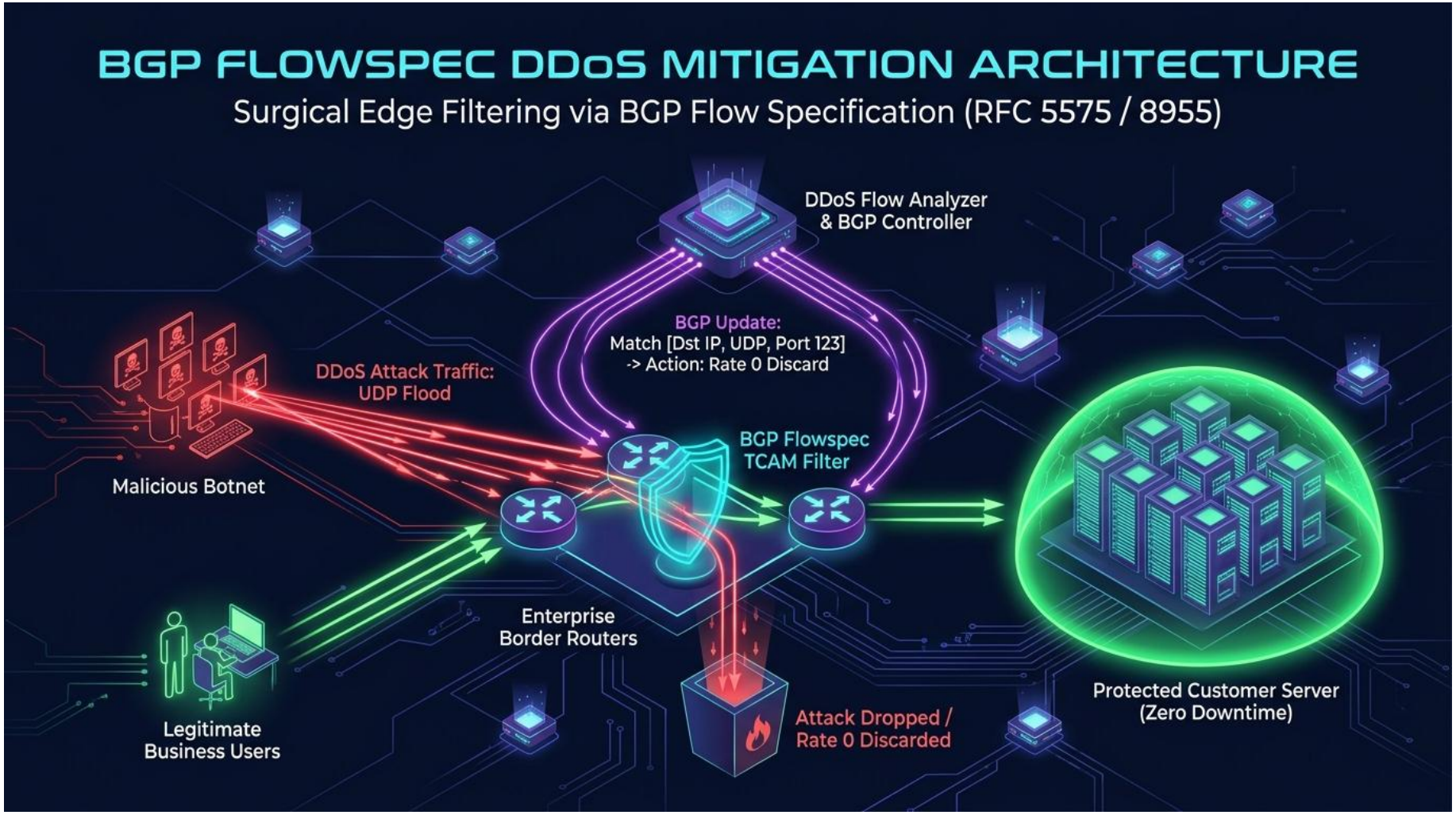


Mitigator TLS Fingerprint Page – Attack Signature



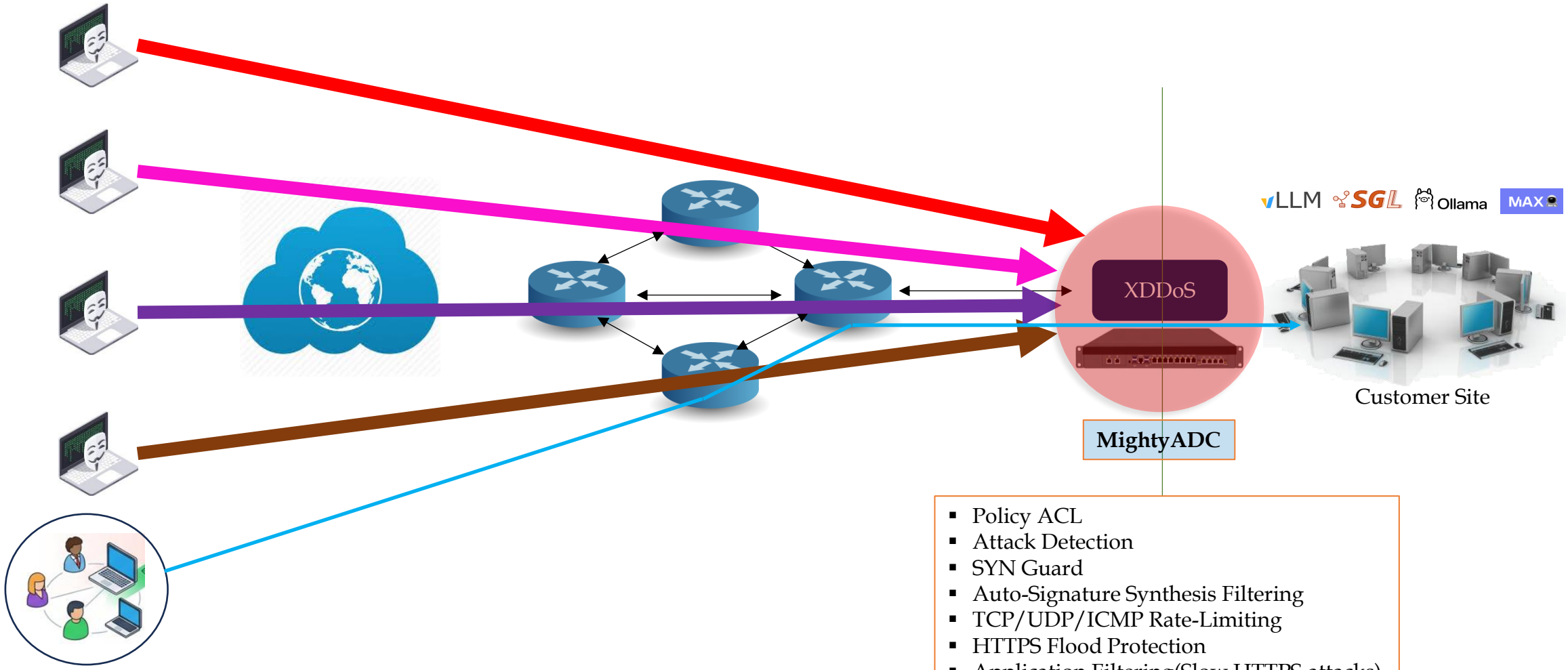
Blocking is possible based on attack signatures.

1. DDoS Mitigator(10) – BGP Flowspec(1)



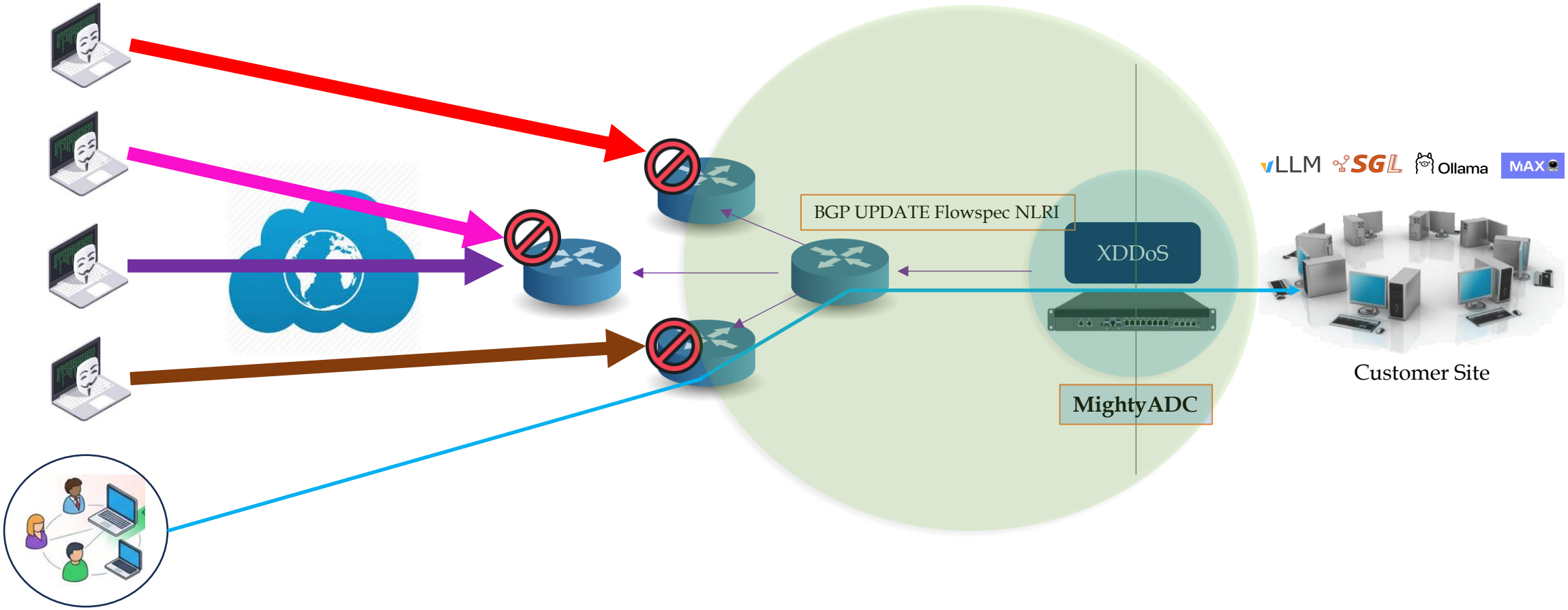
Generated by Gemini

1. DDoS Mitigator(10) – BGP Flowspec(2)



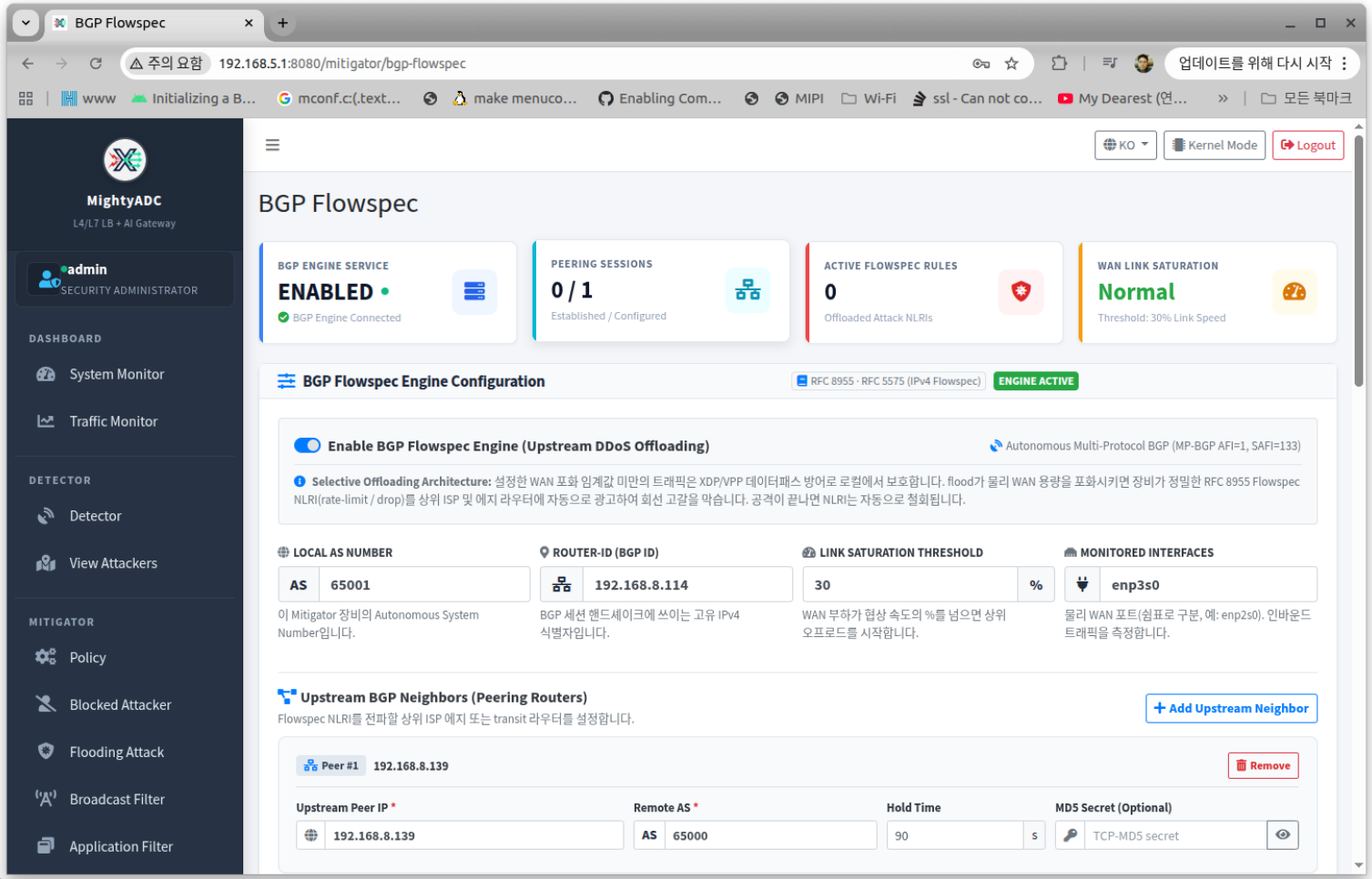
- Policy ACL
- Attack Detection
- SYN Guard
- Auto-Signature Synthesis Filtering
- TCP/UDP/ICMP Rate-Limiting
- HTTPS Flood Protection
- Application Filtering(Slow HTTPS attacks)
- **BGP Flowspec**

1. DDoS Mitigator(10) – BGP Flowspec(3)



1. DDoS Mitigator(10) – BGP Flowspec(4)

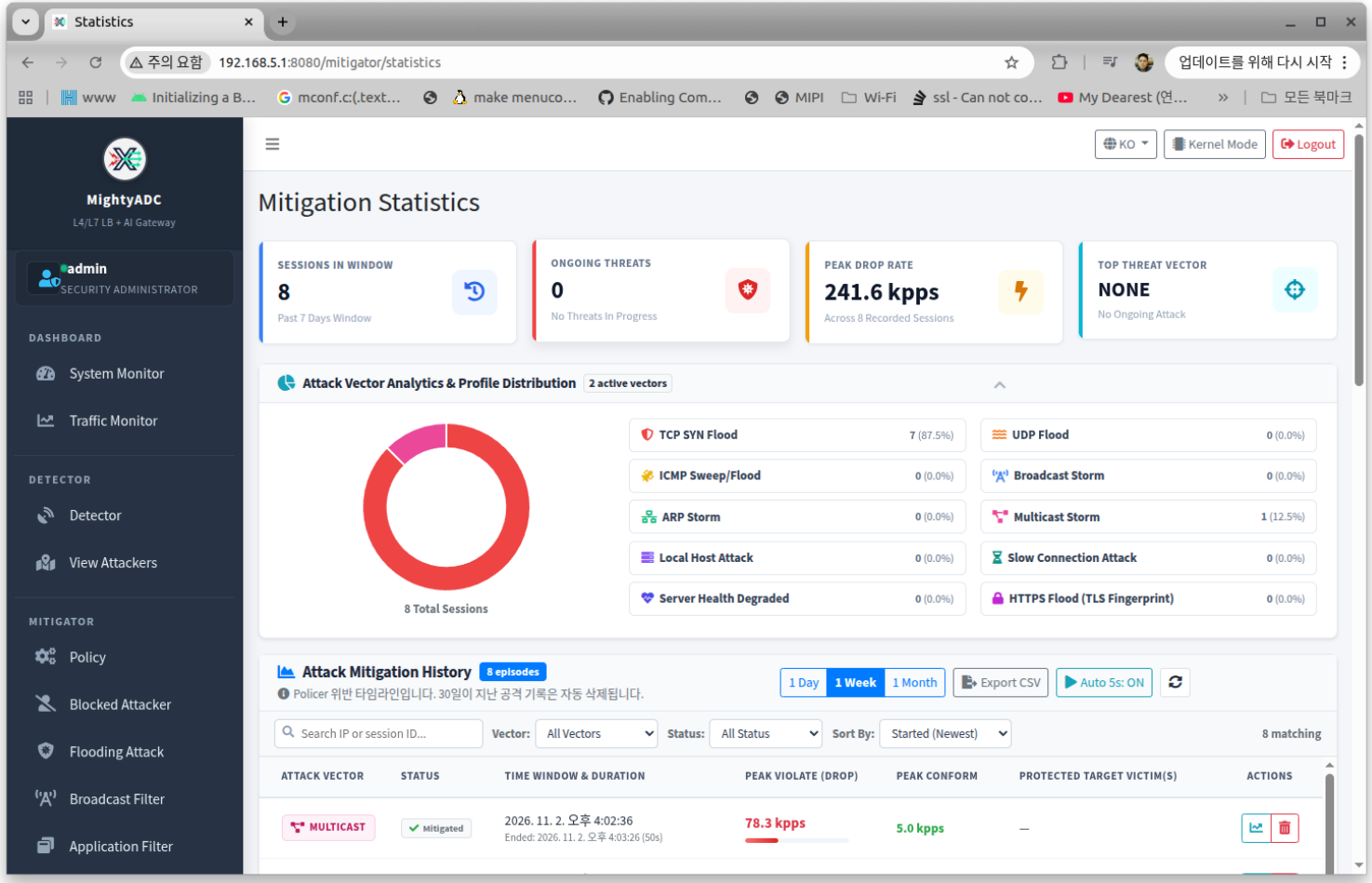
Mitigator BGP Flowspec Page



XDDoS

1. DDoS Mitigator(11) – Statistics

Mitigator Statistics Page

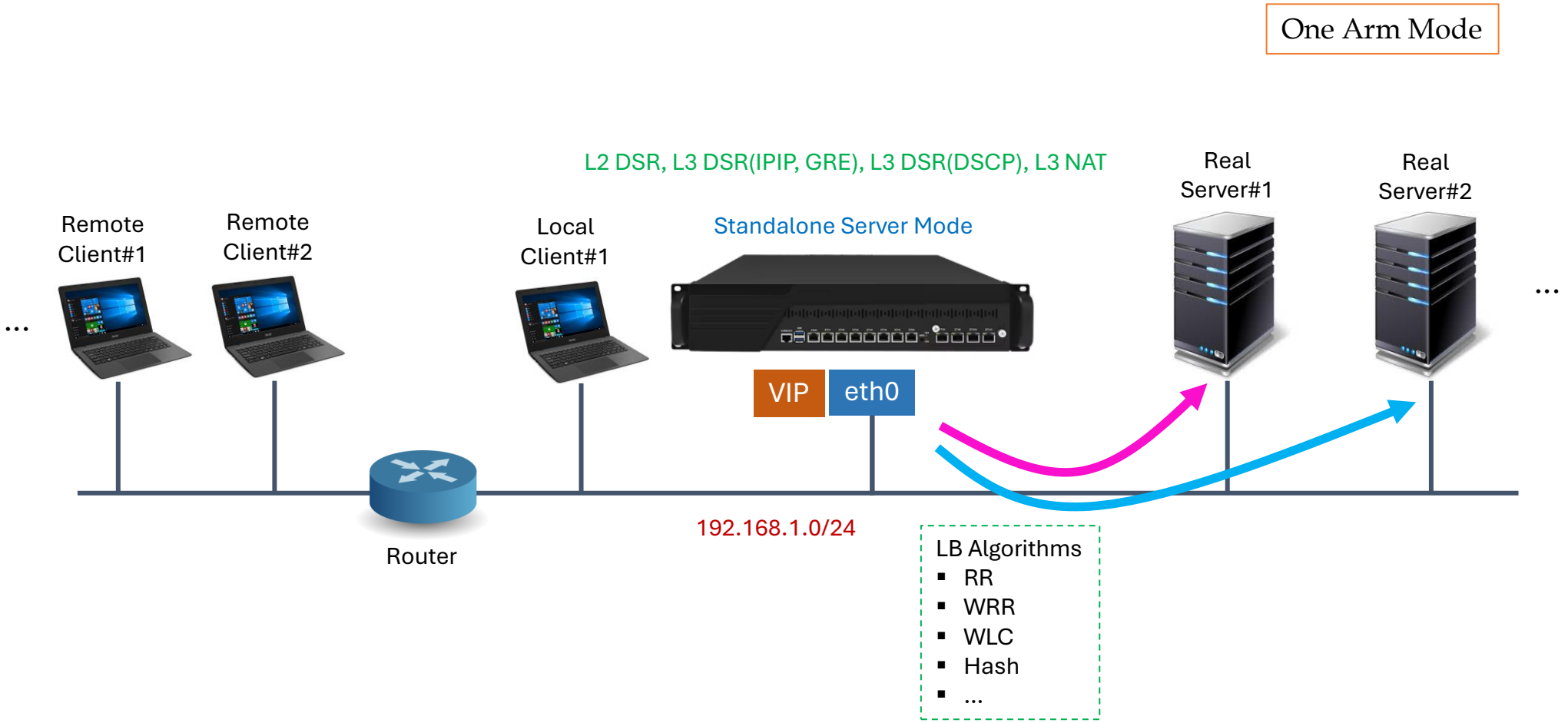


2. L4/L7 Load Balancer – L2DSR/L3DSR/L3NAT, HAProxy

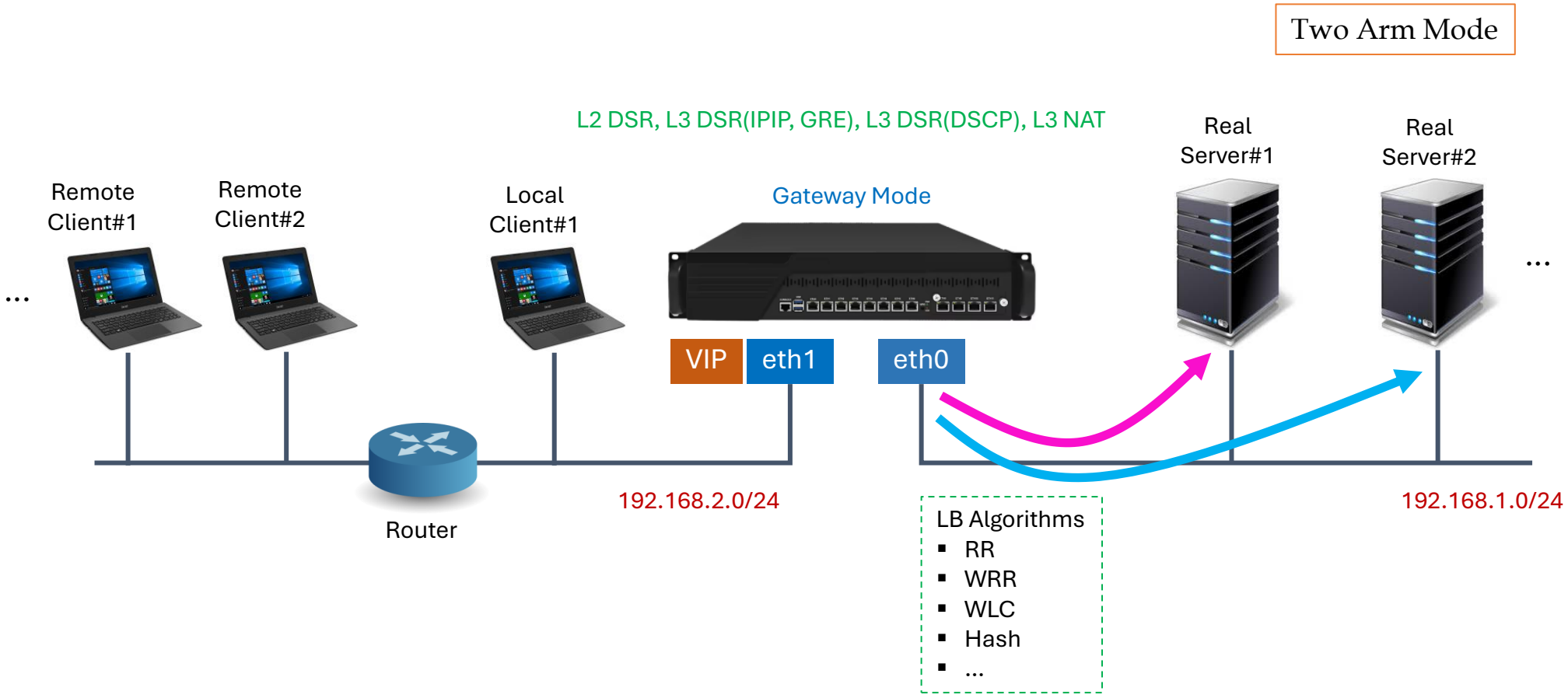


- L7 Proxy(HAProxy)
- L3 NAT(DNAT, FNAT)
- L3 DSR(IPIP, GRE, DSCP)
- L2 DSR

2. L4/L7 Load Balancer(1) – L4 LB Concept(1)

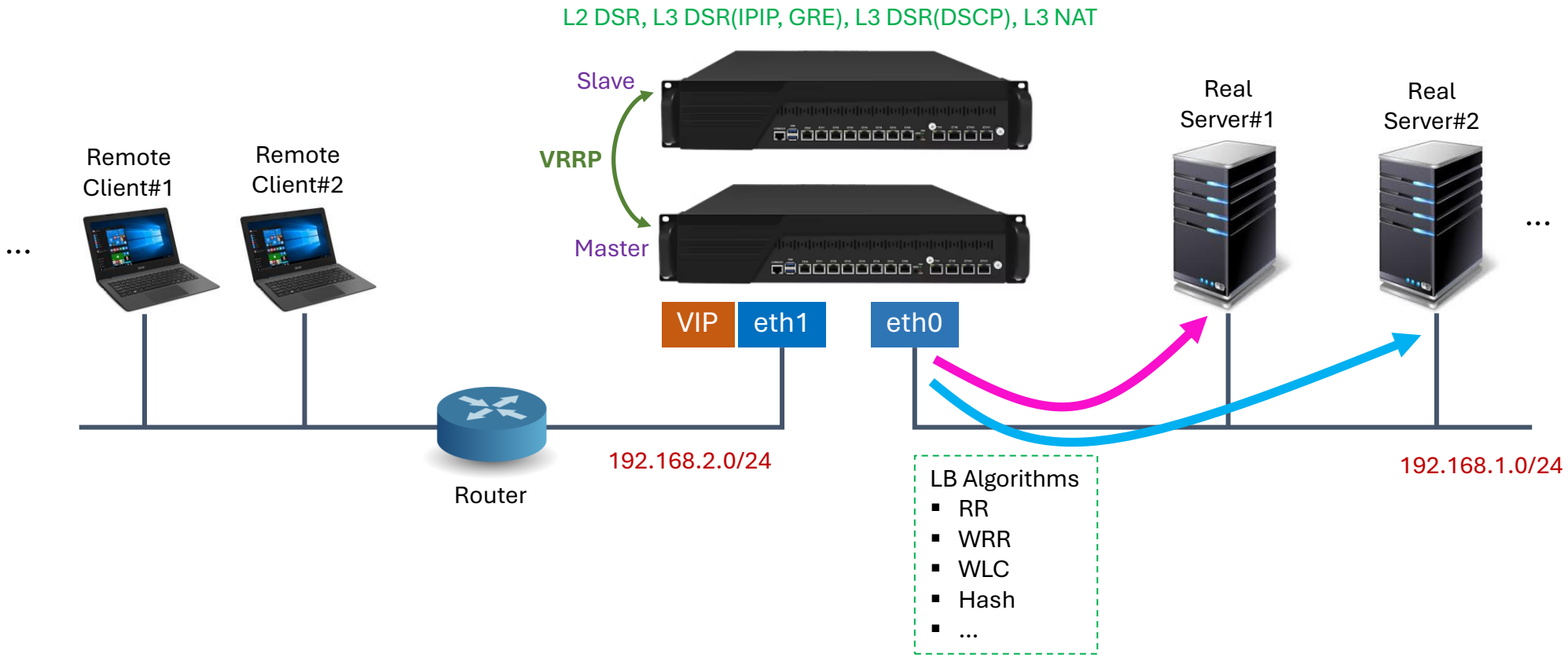


2. L4/L7 Load Balancer(1) – L4 LB Concept(2)



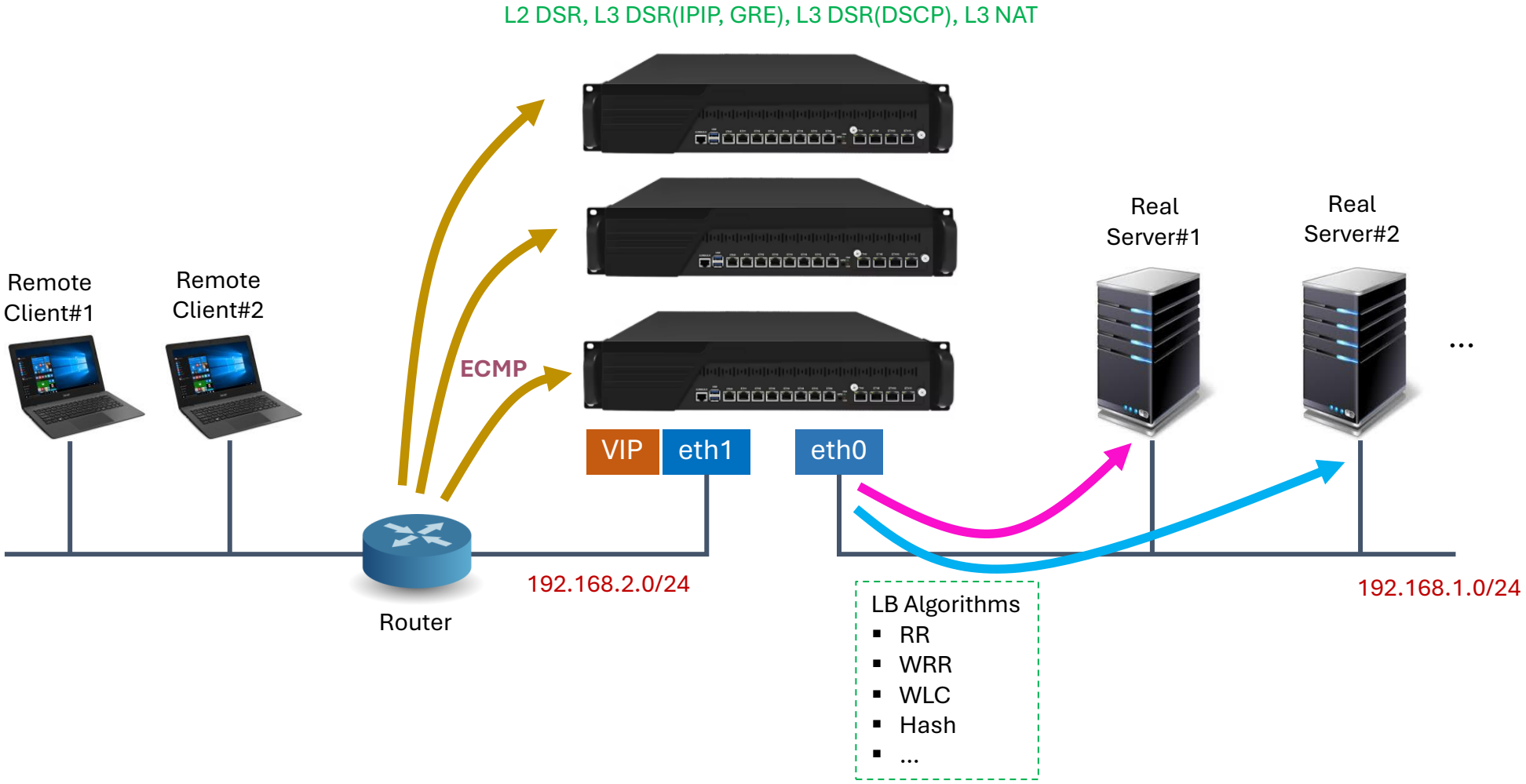
2. L4/L7 Load Balancer(1) – L4 LB Concept(3)

VRRP Redundancy/Two Arm Mode

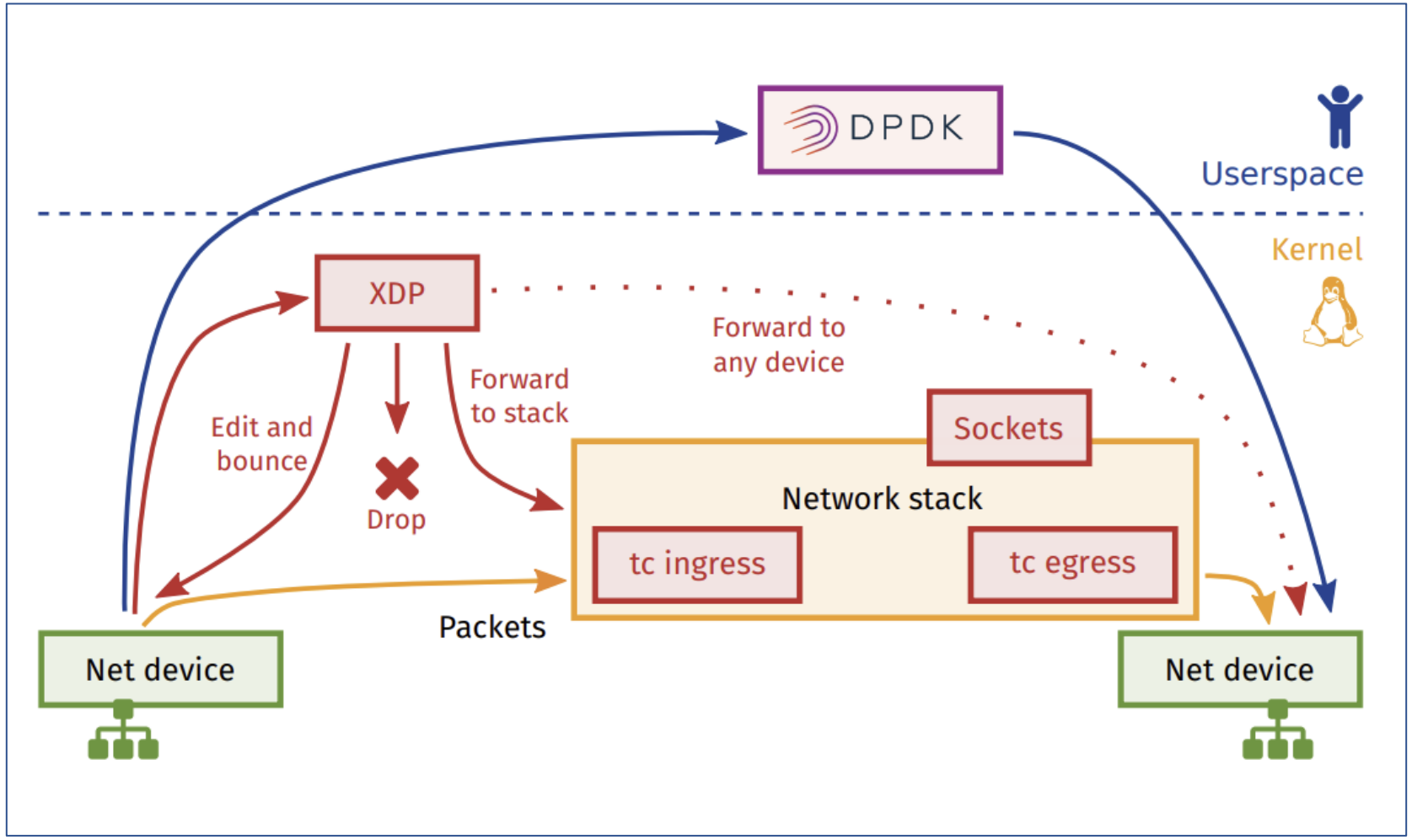


2. L4/L7 Load Balancer(1) – L4 LB Concept(4)

ECMP/Two Arm Mode

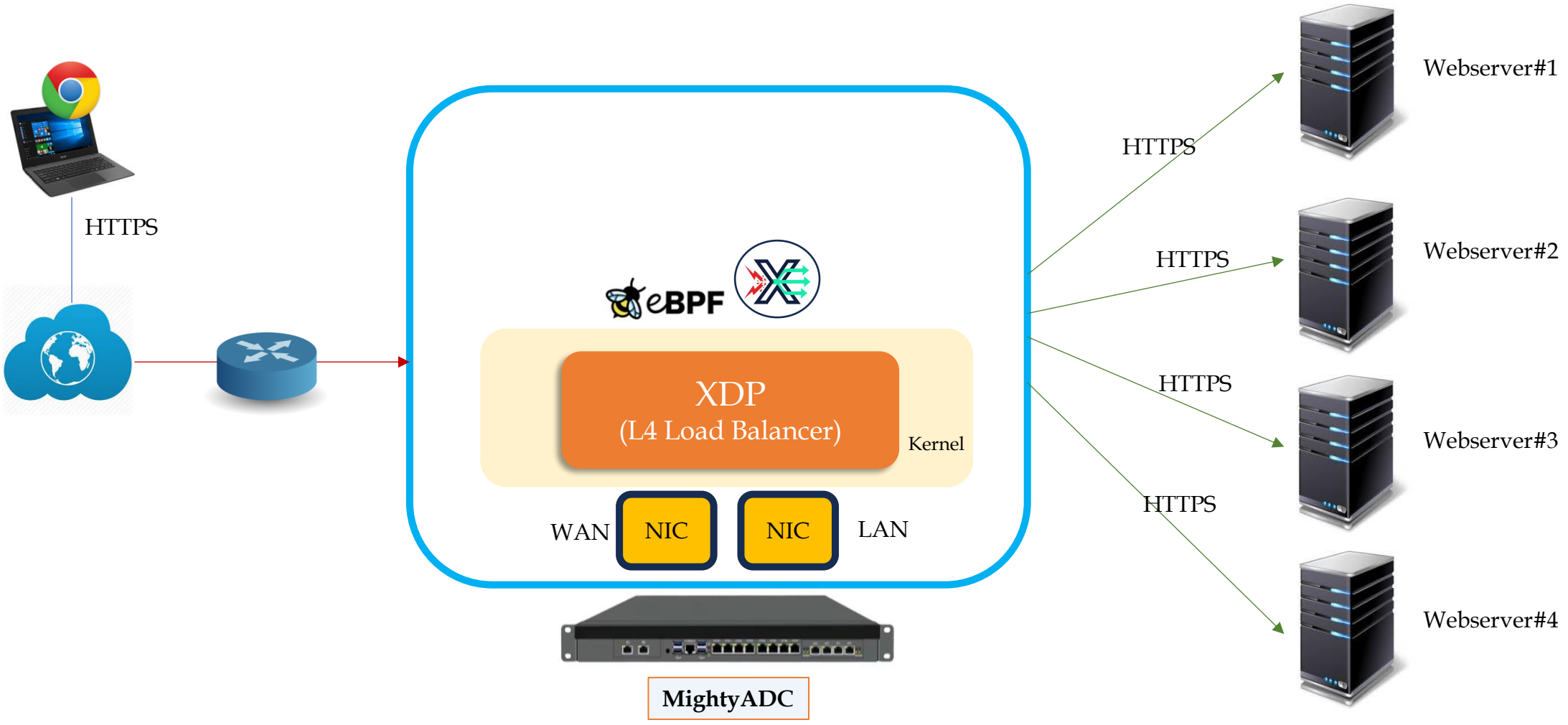


2. L4/L7 Load Balancer(2) – XDP DataPath(1)

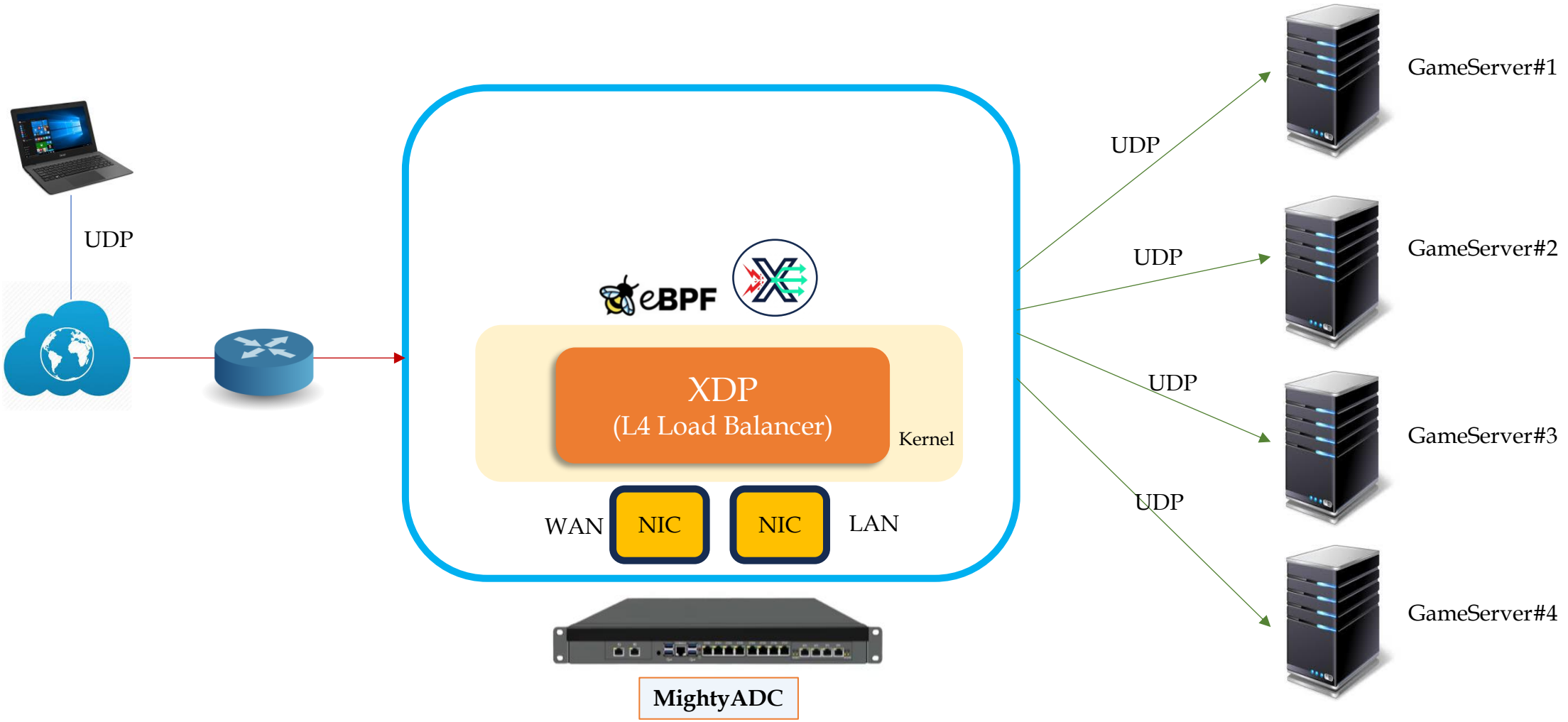


Ref) https://media.frnog.org/FRnOG_28/FRnOG_28-3.pdf

2. L4/L7 Load Balancer(2) – XDP DataPath(2)



2. L4/L7 Load Balancer(2) – XDP DataPath(3)



2. L4/L7 Load Balancer(3) – L4 LB Algorithm

Backend Selection Algorithm

1. **일반 모듈로 해싱의 한계**: `hash(5-tuple) % N` 방식은 백엔드 서버 대수(N)가 변경될 때 전체 해시 매핑이 뒤바뀌어 대규모 세션 단절(Connection Disruption)을 유발합니다.
2. **Maglev 순열(Permutation) 생성**:
   - 테이블 크기 M을 충분히 큰 소수(Prime Number, 예: $65,537$)로 고정.
   - 각 백엔드 i마다 두 개의 독립 해시 함수(h_1, h_2)로 오프셋과 스킵 크기를 계산:

$$\text{offset} = h_1(i) \pmod{M}$$

$$\text{skip} = h_2(i) \pmod{M-1} + 1$$

$$P_i[j] = (\text{offset} + j \times \text{skip}) \pmod{M} \quad (0 \le j < M)$$

   - 백엔드들이 라운드-로빈 순서로 선호 슬롯($P_i[j]$) 중 빈자리를 채워 M 크기의 룩업 테이블(LUT)을 완성.
   - 백엔드 풀 변경 시 오직 $\frac{1}{N}$ 비율의 트래픽만 재배치되며, 나머지 정상 백엔드의 연결은 100% 유지.

XDP

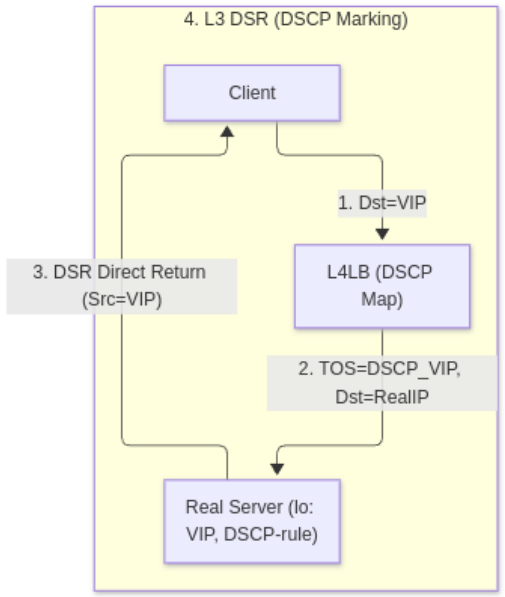
MightyADC LB Algorithms: Google Maglev Consistent Hashing + Weight



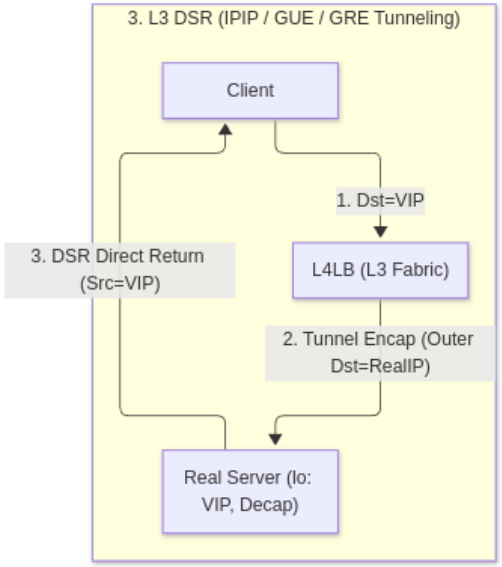
MightyADC

2. L4/L7 Load Balancer(4) – L4 LB Modes(1)

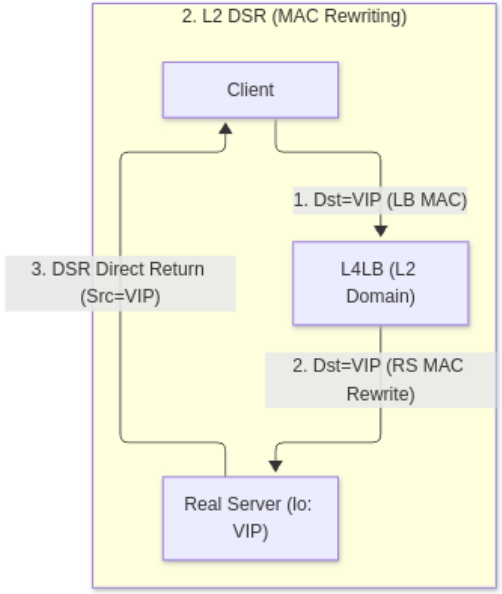
L3 DSR(DSCP Marking)



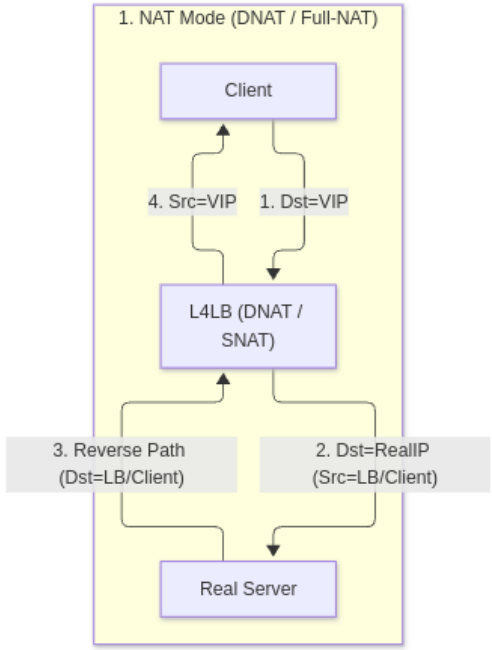
L3 DSR(IPIP, GRE Tunneling)



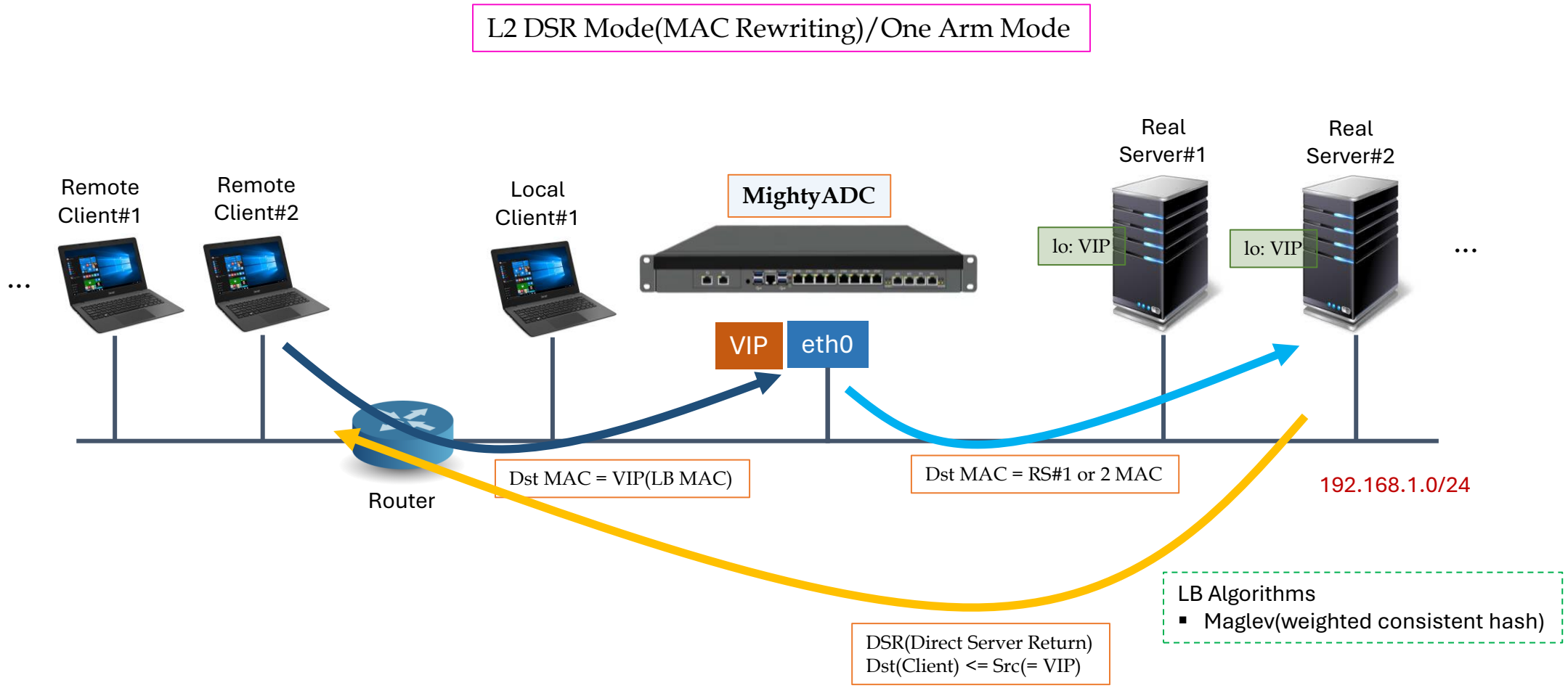
L2 DSR(MAC Rewriting)



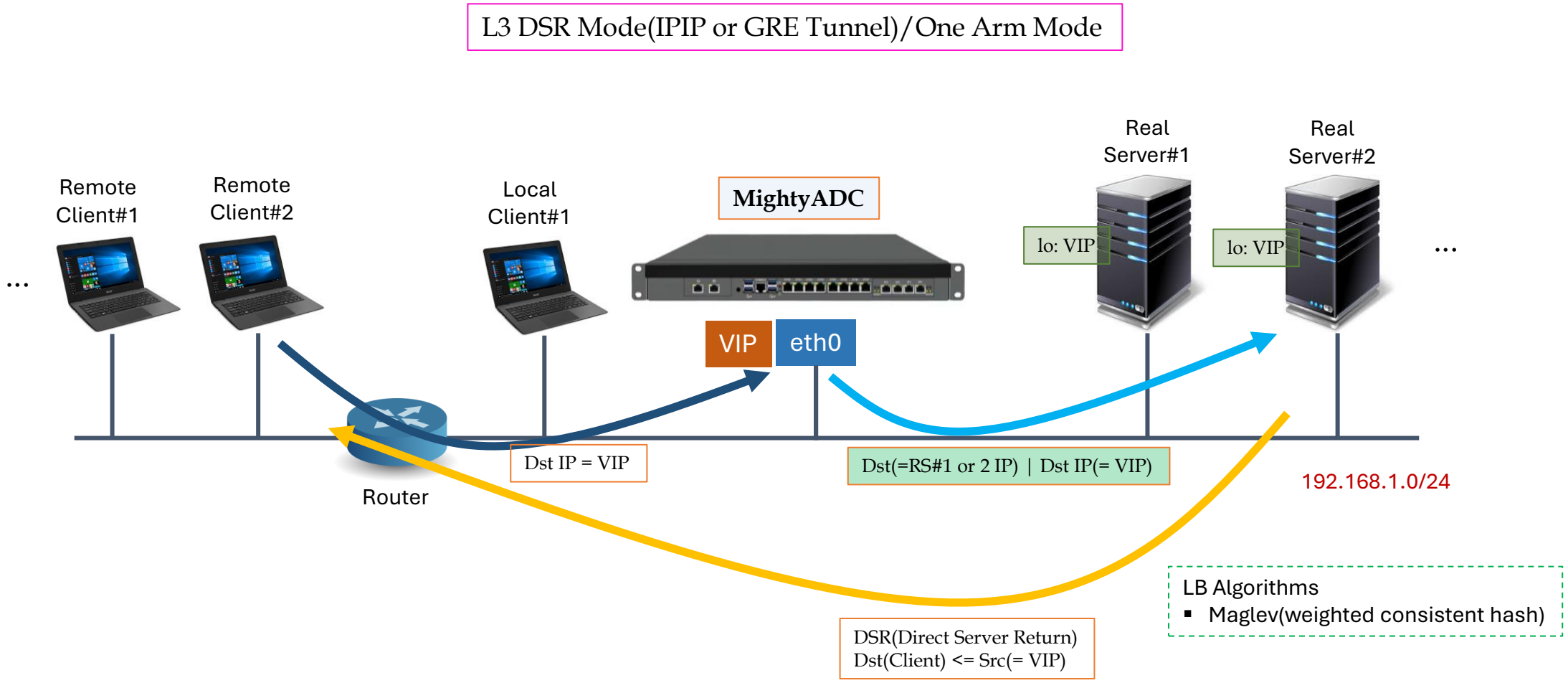
DNAT/FNAT



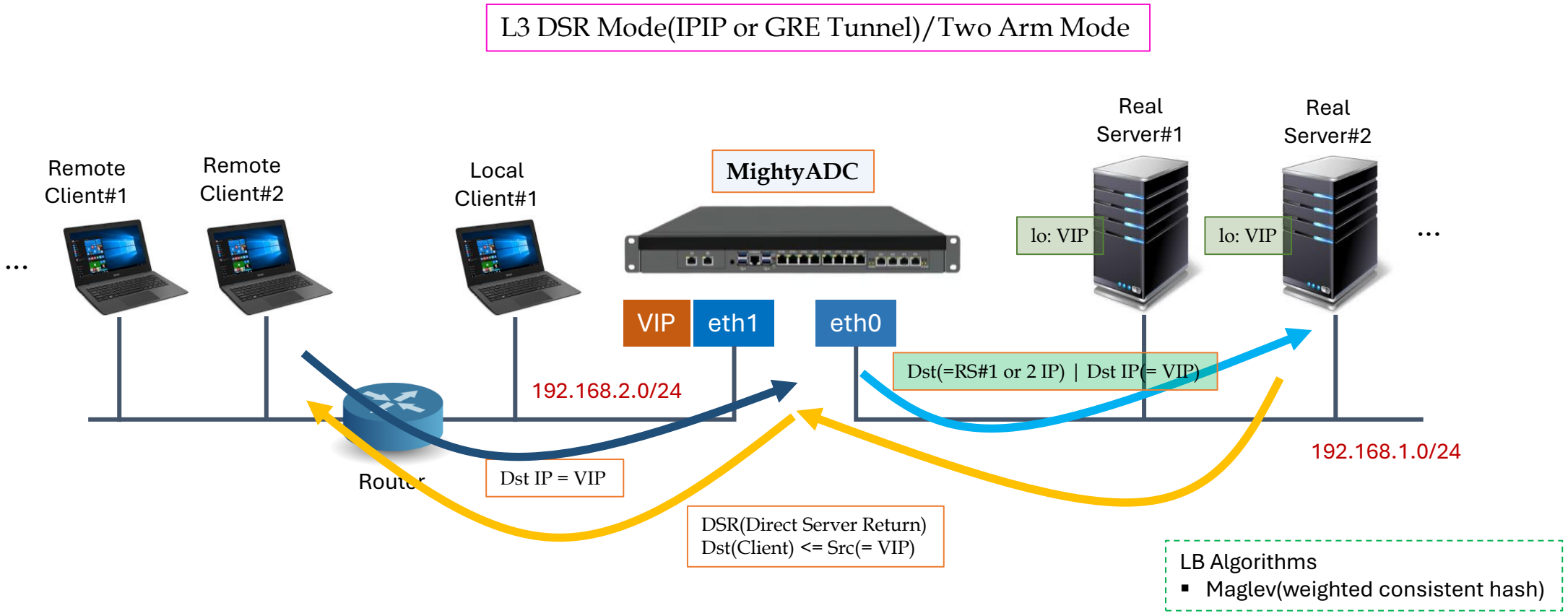
2. L4/L7 Load Balancer(4) – L4 LB Modes(2)



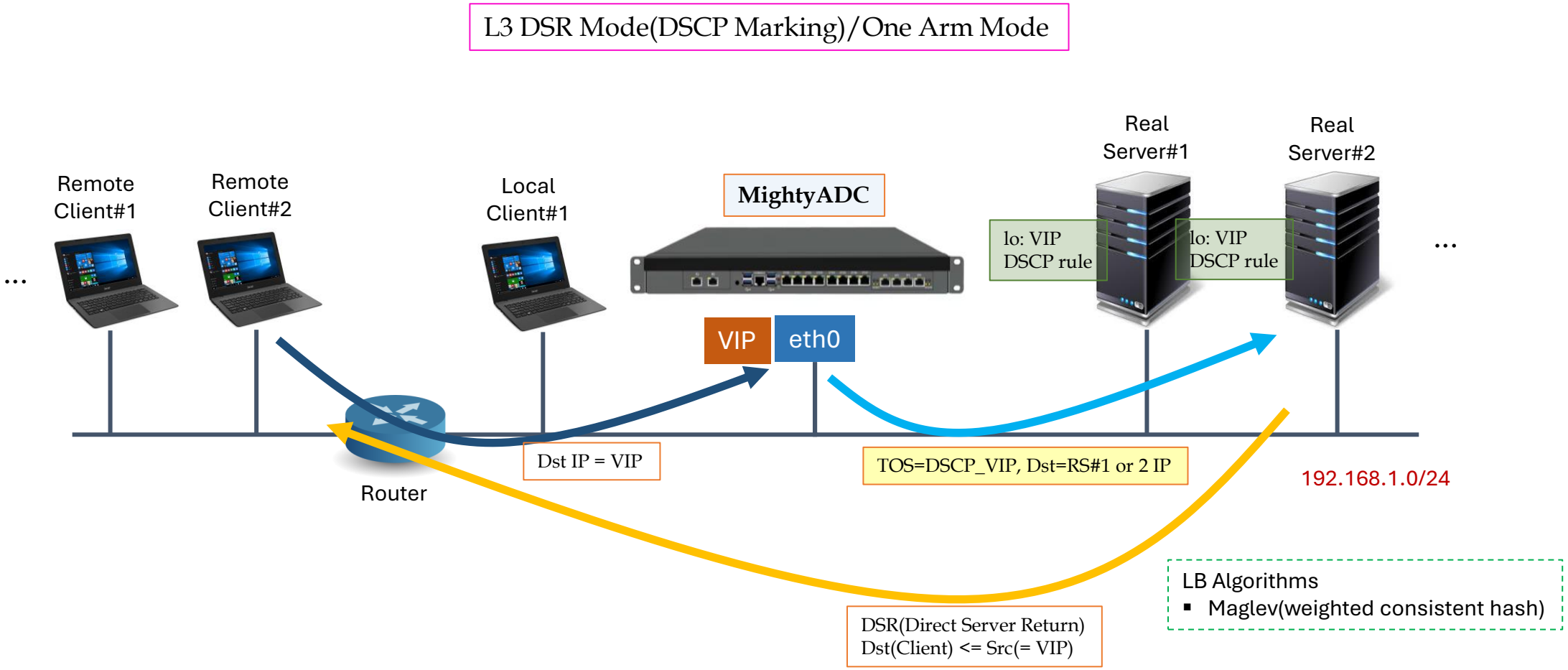
2. L4/L7 Load Balancer(4) – L4 LB Modes(3-1)



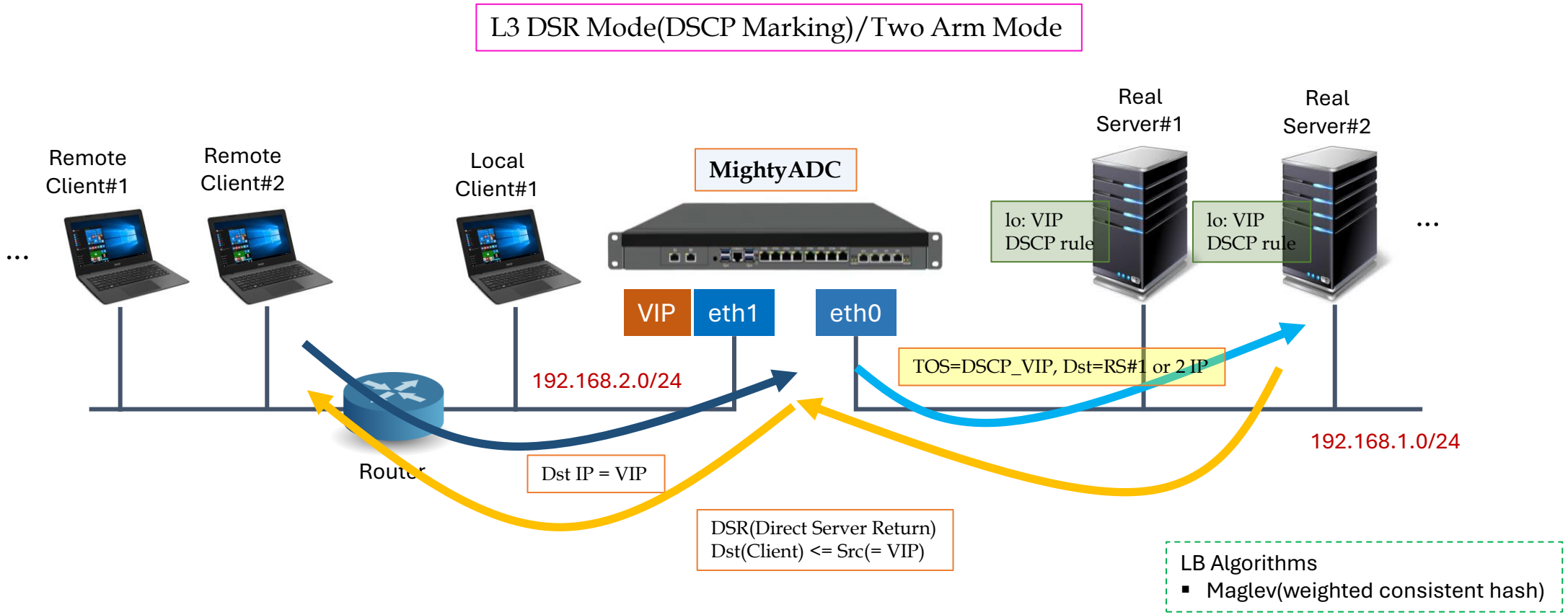
2. L4/L7 Load Balancer(4) – L4 LB Modes(3-2)



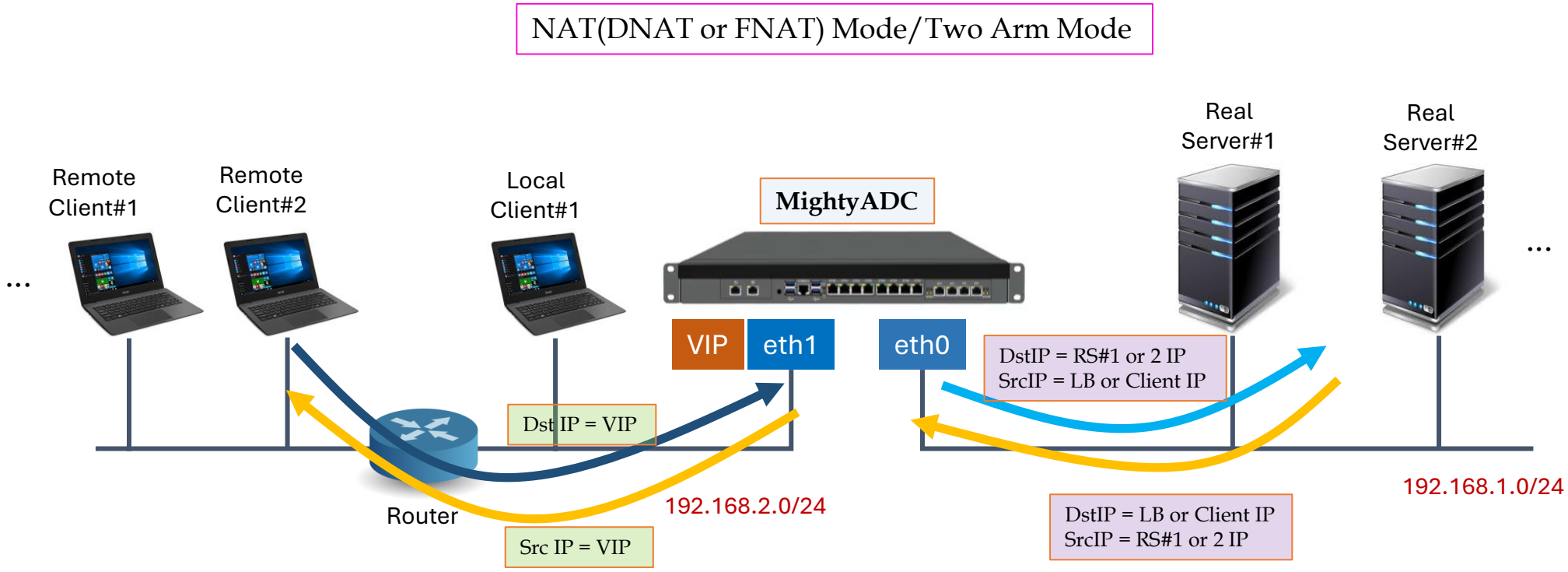
2. L4/L7 Load Balancer(4) – L4 LB Modes(4-1)



2. L4/L7 Load Balancer(4) – L4 LB Modes(4-2)



2. L4/L7 Load Balancer(4) – L4 LB Modes(5)

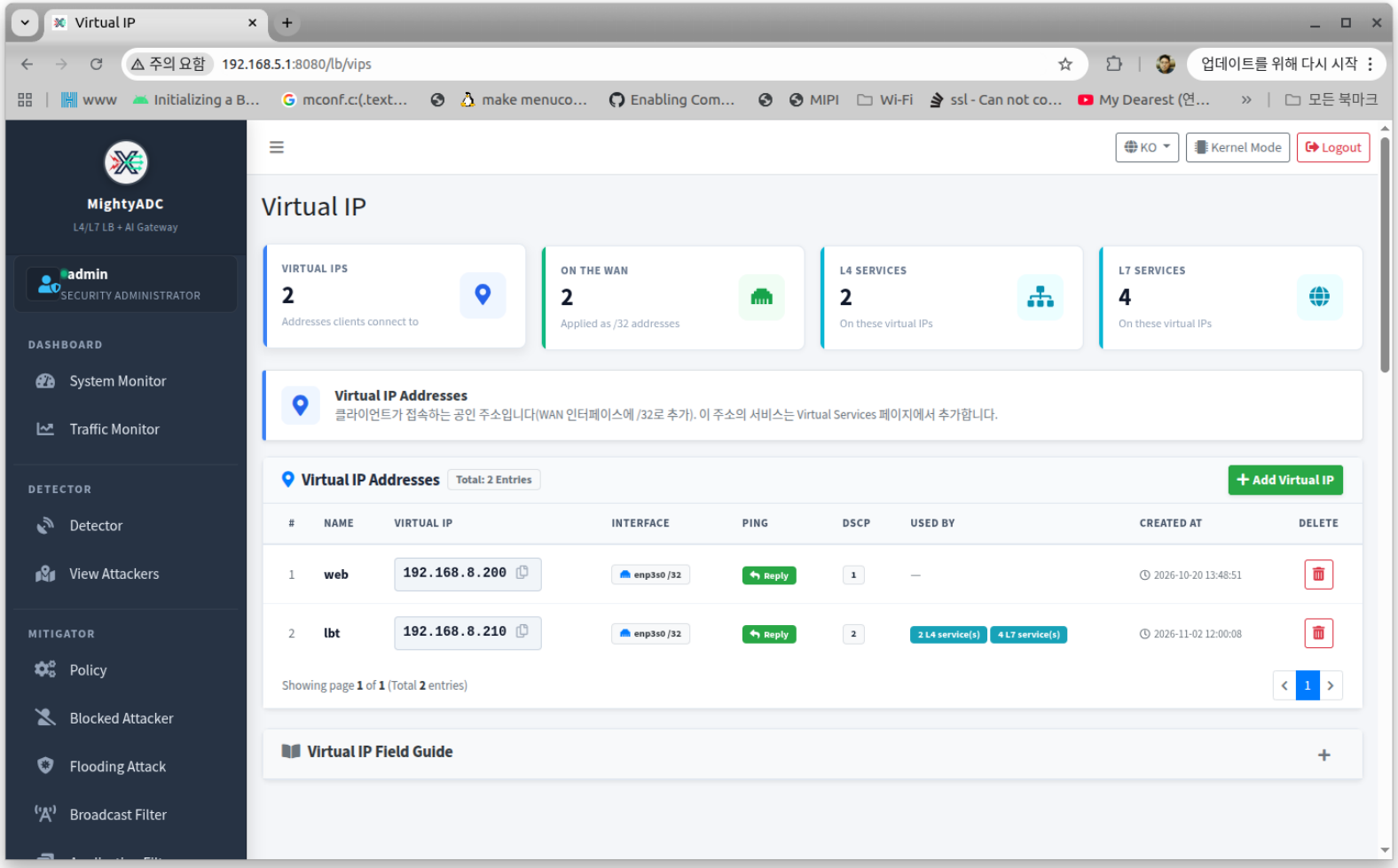


- DNAT: Destination NAT
- FNAT: Src and Destination NAT
  - ✓ FNAT provides the capability to configure a Source Address Pool.

LB Algorithms
- Maglev(weighted consistent hash)

2. L4/L7 Load Balancer(4) – L4 LB Modes(6)

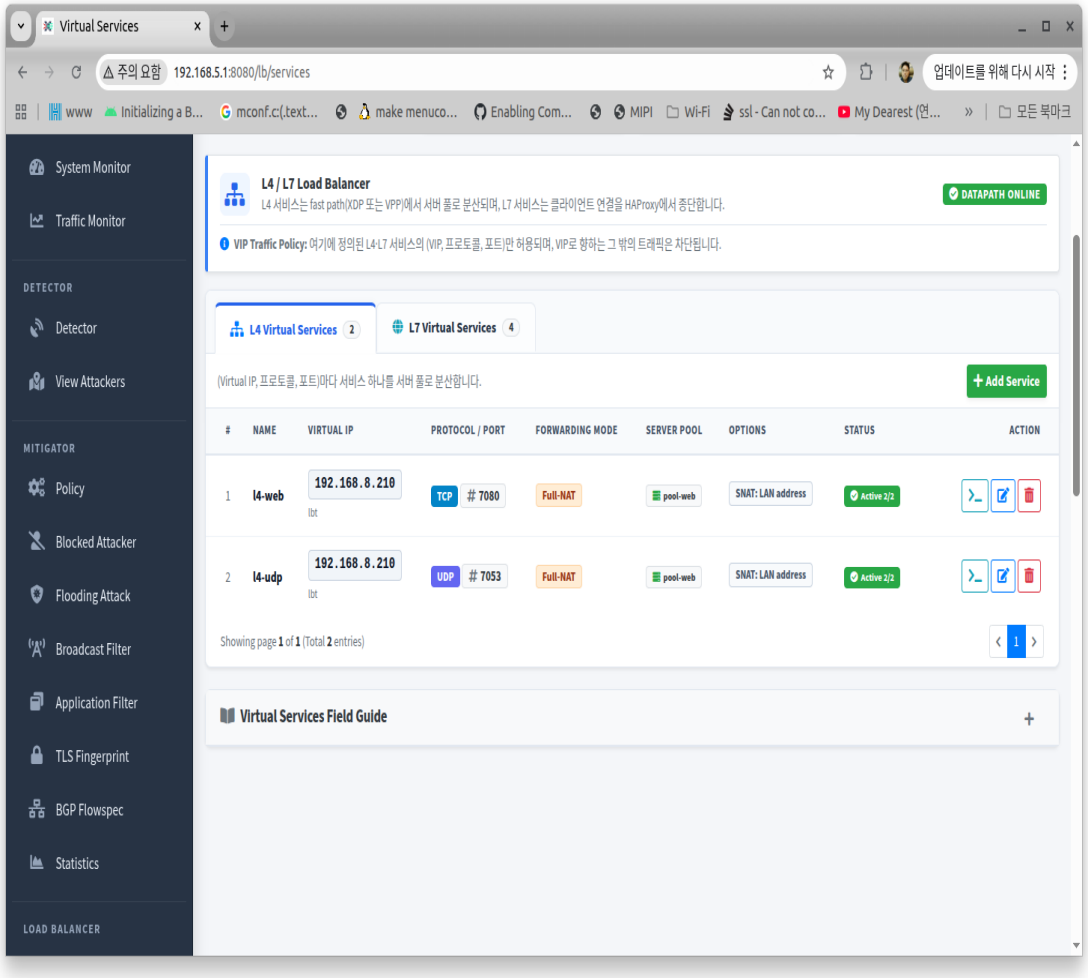
LB Virtual IP Page



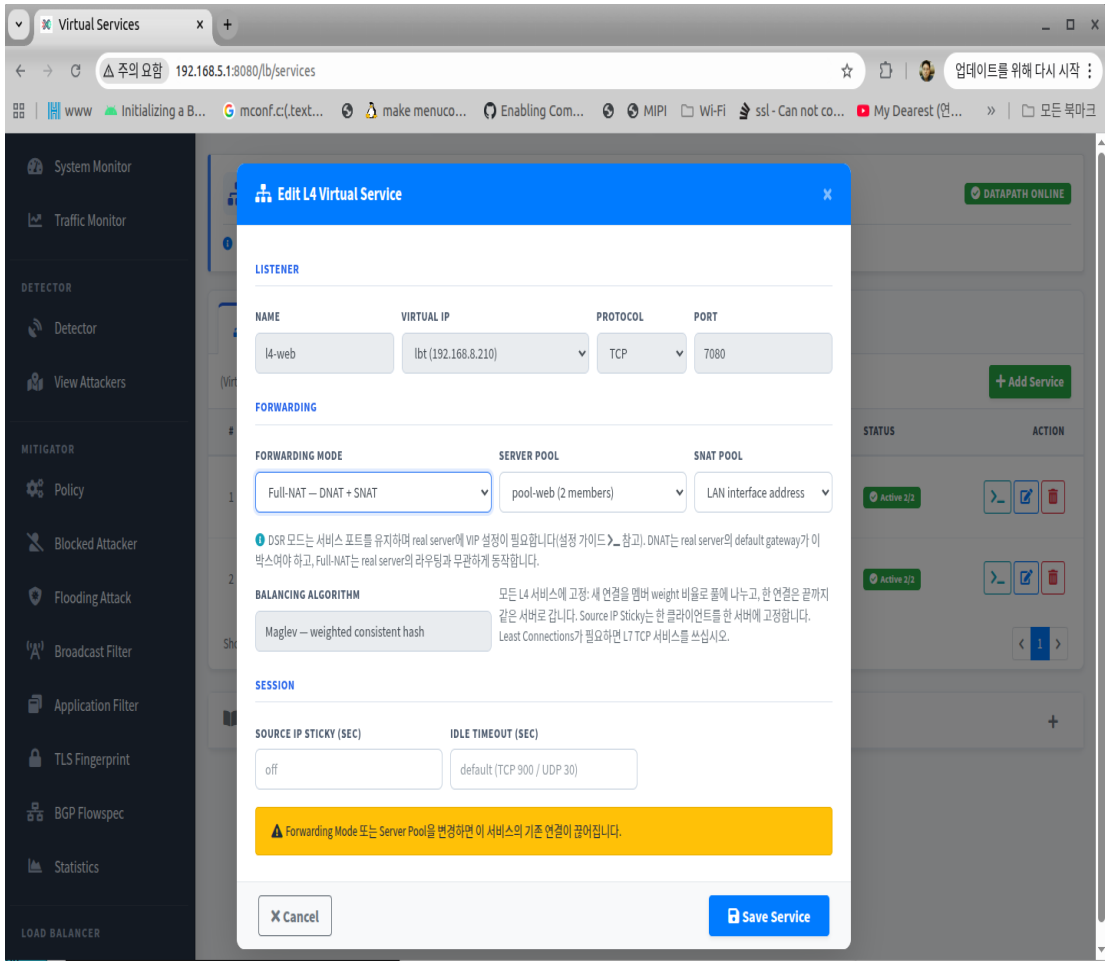


2. L4/L7 Load Balancer(4) – L4 LB Modes(7)

L4 Virtual Services Page - List

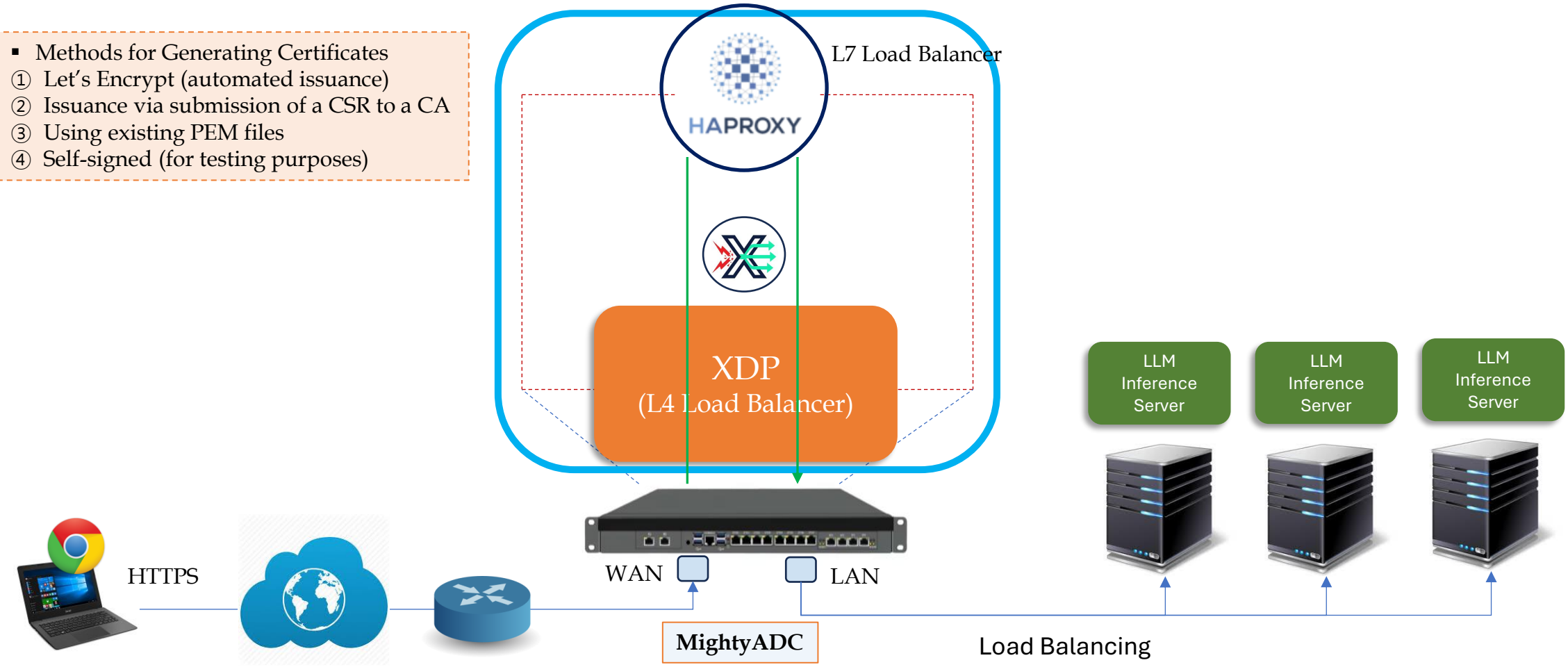


L4 Virtual Services Page – Settings Popup



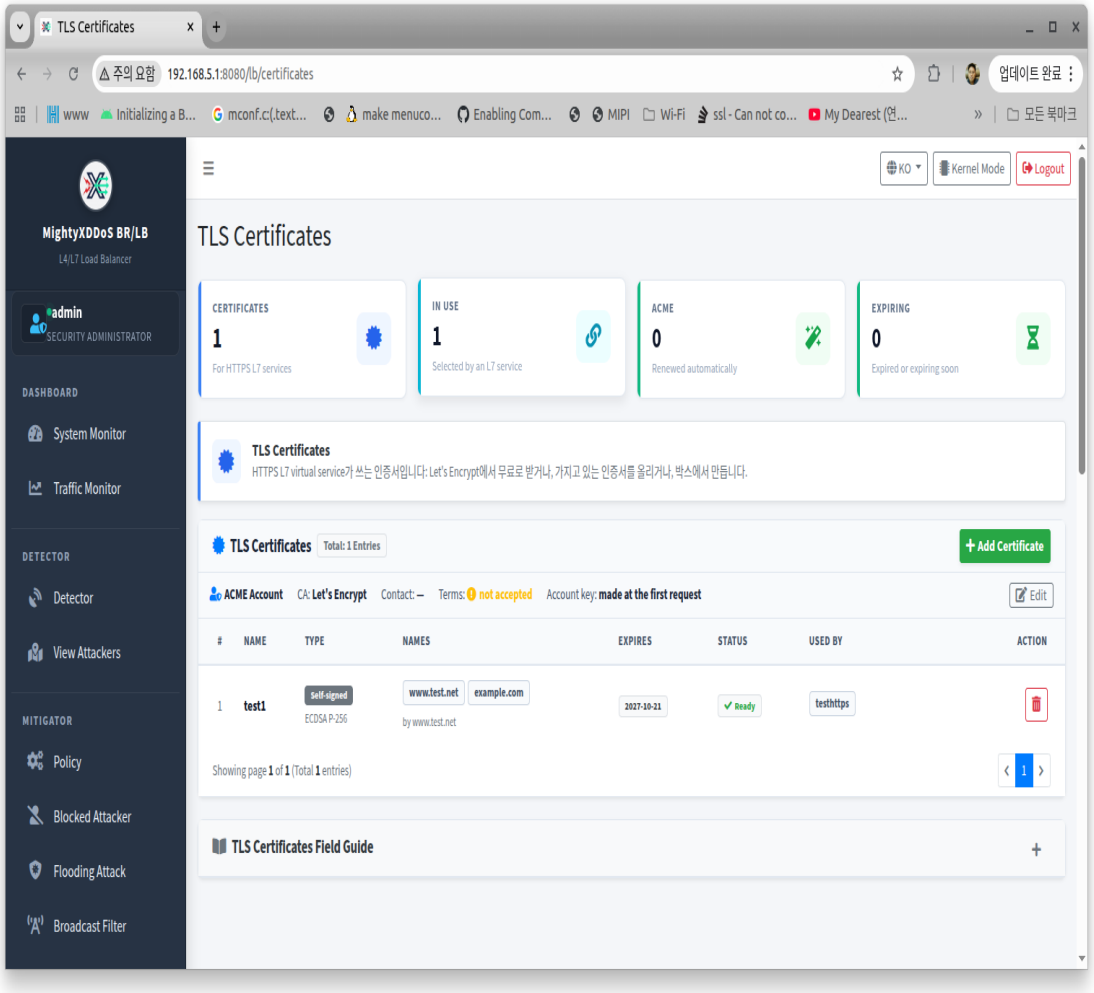
2. L4/L7 Load Balancer(5) – L7 Proxy & Certificate(1)

- Methods for Generating Certificates
① Let's Encrypt (automated issuance)
② Issuance via submission of a CSR to a CA
③ Using existing PEM files
④ Self-signed (for testing purposes)

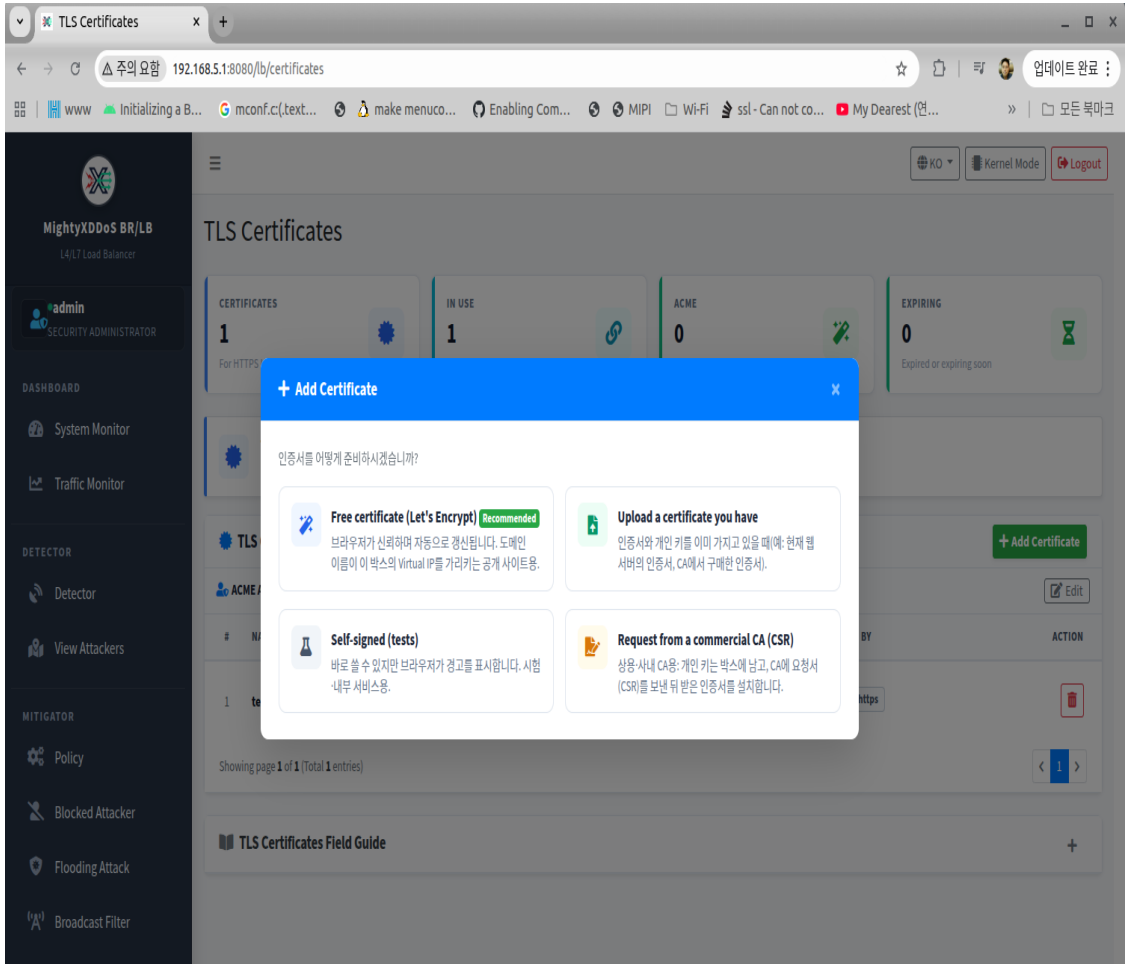


2. L4/L7 Load Balancer(5) – L7 Proxy & Certificate(2)

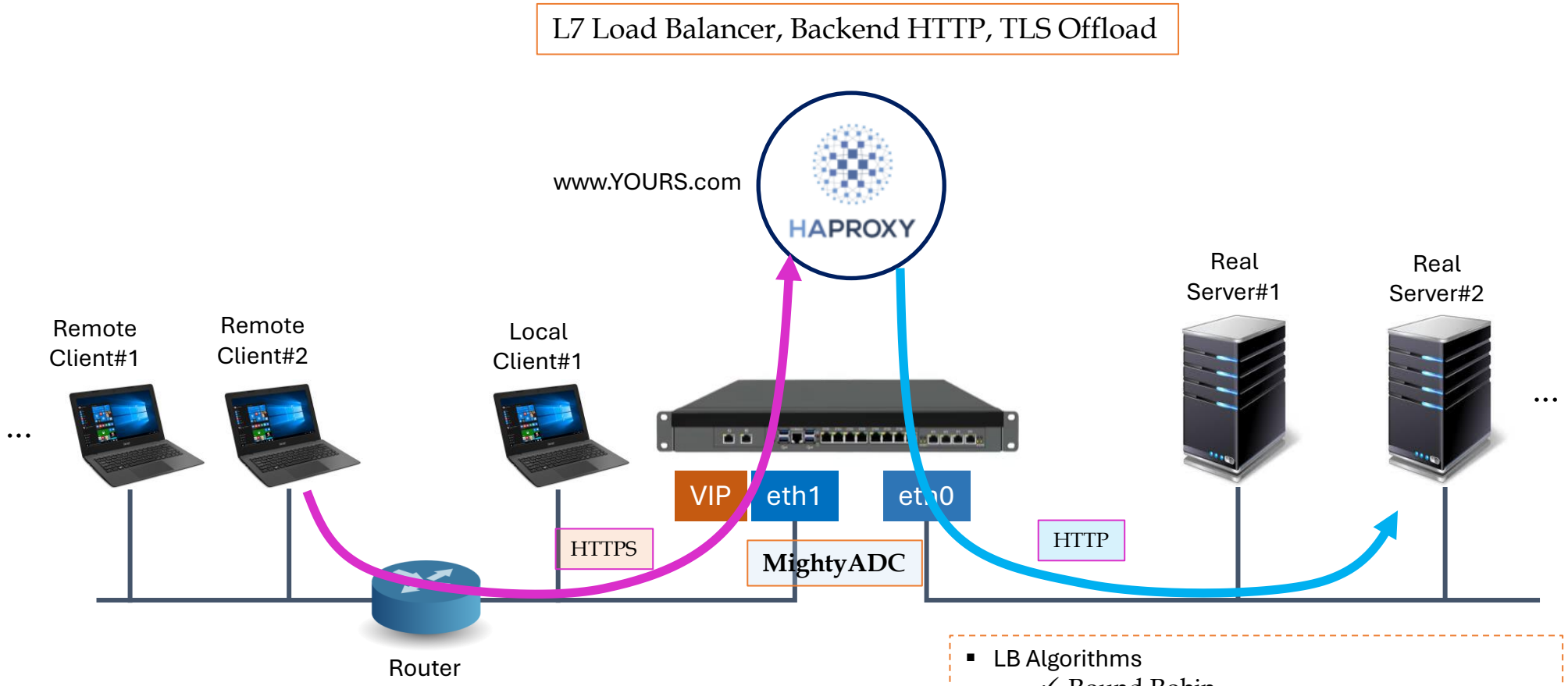
Settings -> TLS Certificates Page(1)



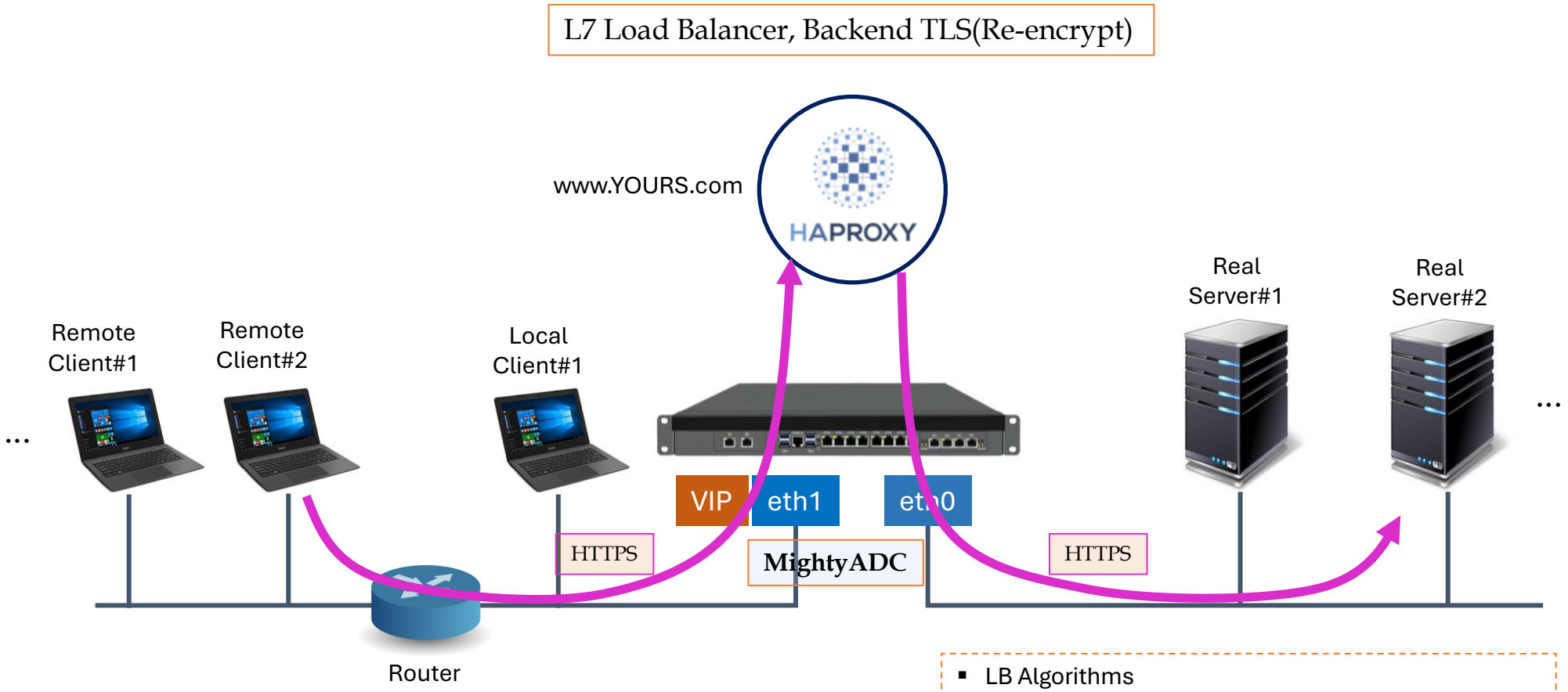
Settings -> TLS Certificates Page(2)



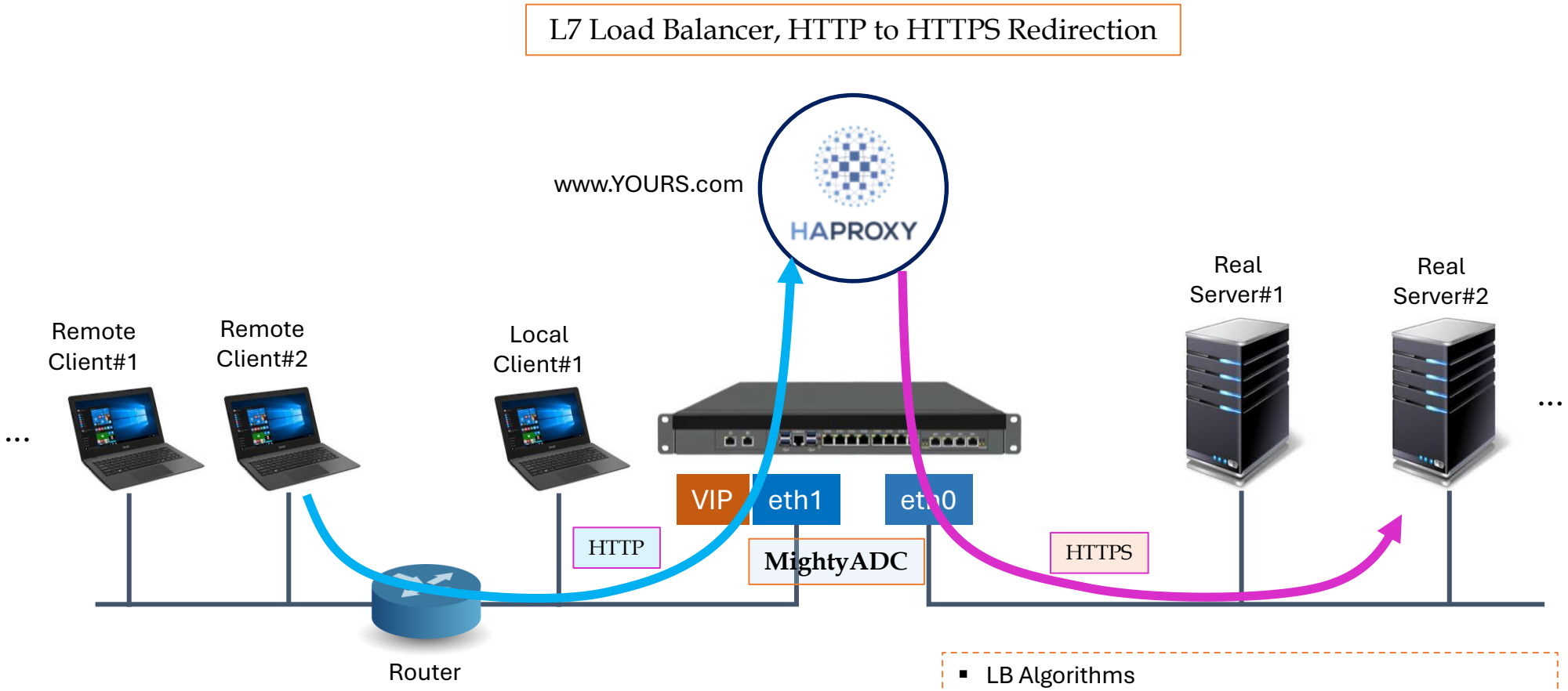
2. L4/L7 Load Balancer(6) – L7 Proxy Mode(1)



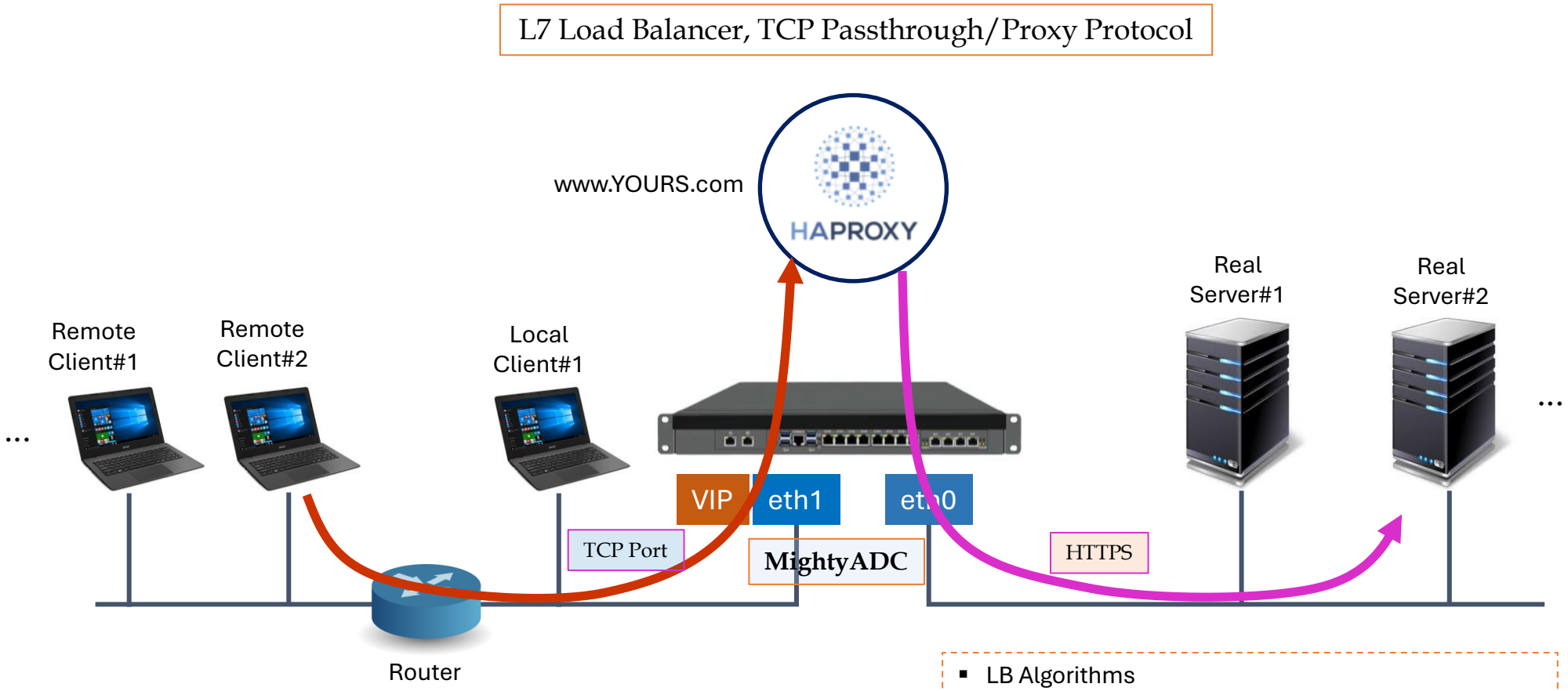
2. L4/L7 Load Balancer(6) – L7 Proxy Mode(2)



2. L4/L7 Load Balancer(6) – L7 Proxy Mode(3)

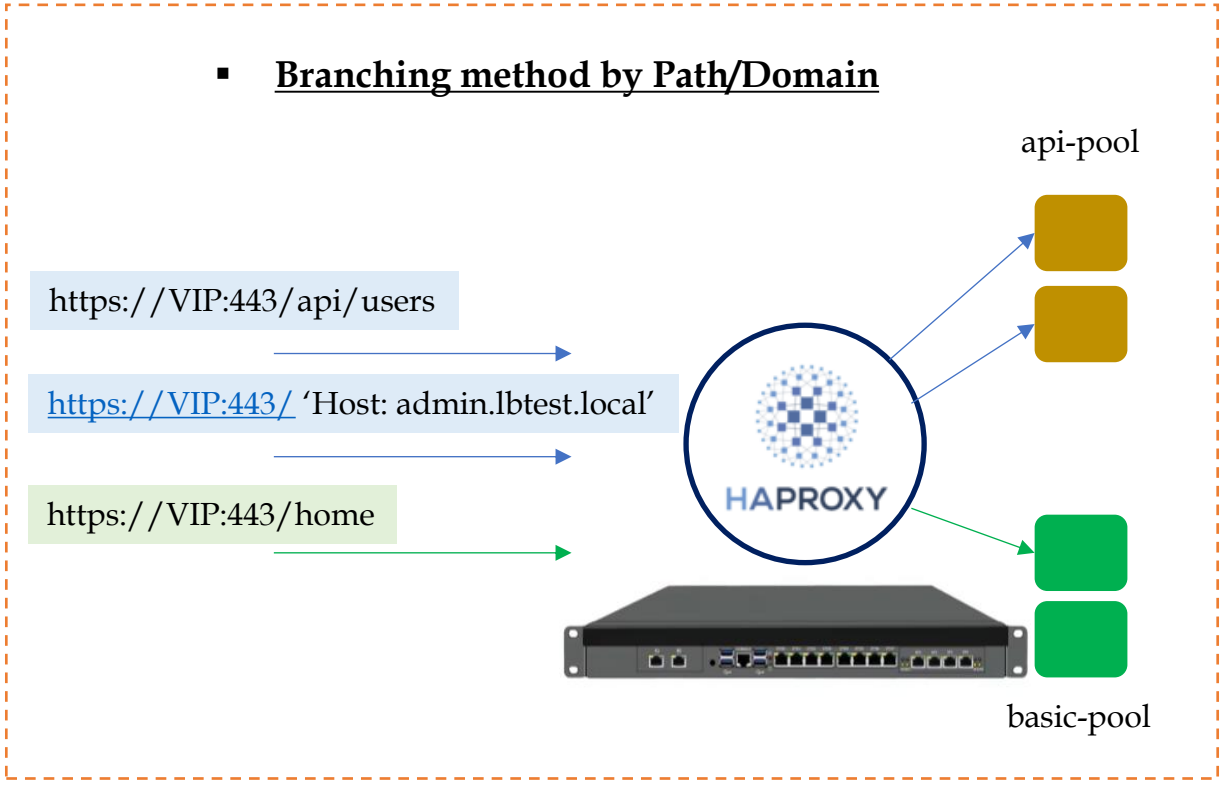


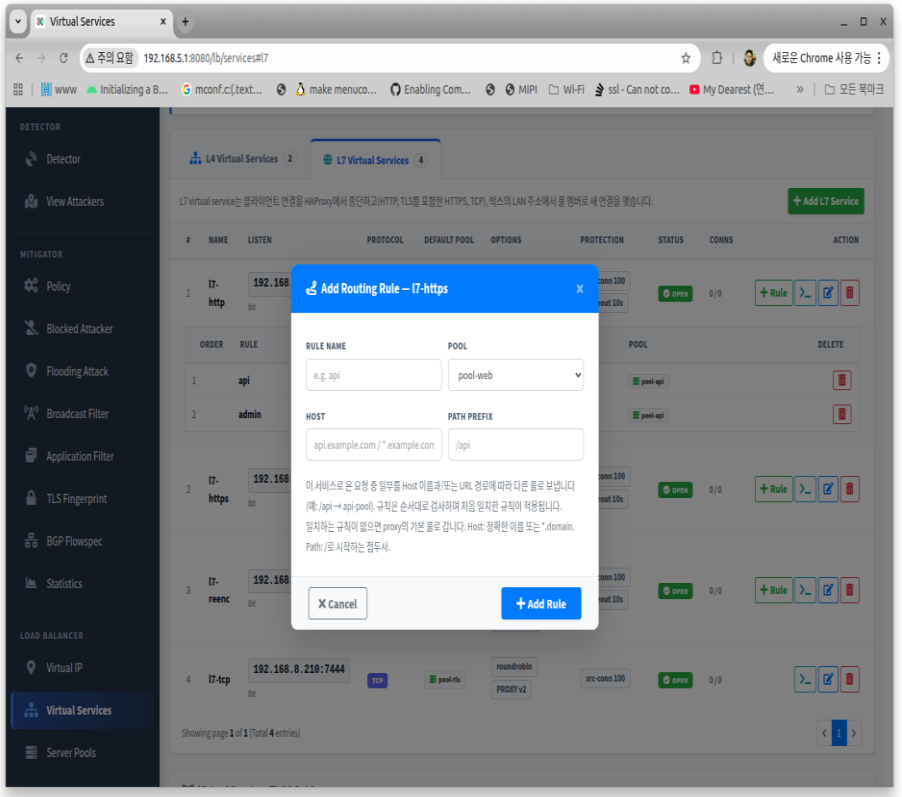
2. L4/L7 Load Balancer(6) – L7 Proxy Mode(4)



2. L4/L7 Load Balancer(6) – L7 Proxy Mode(5)

- **Branching method by Path/Domain**



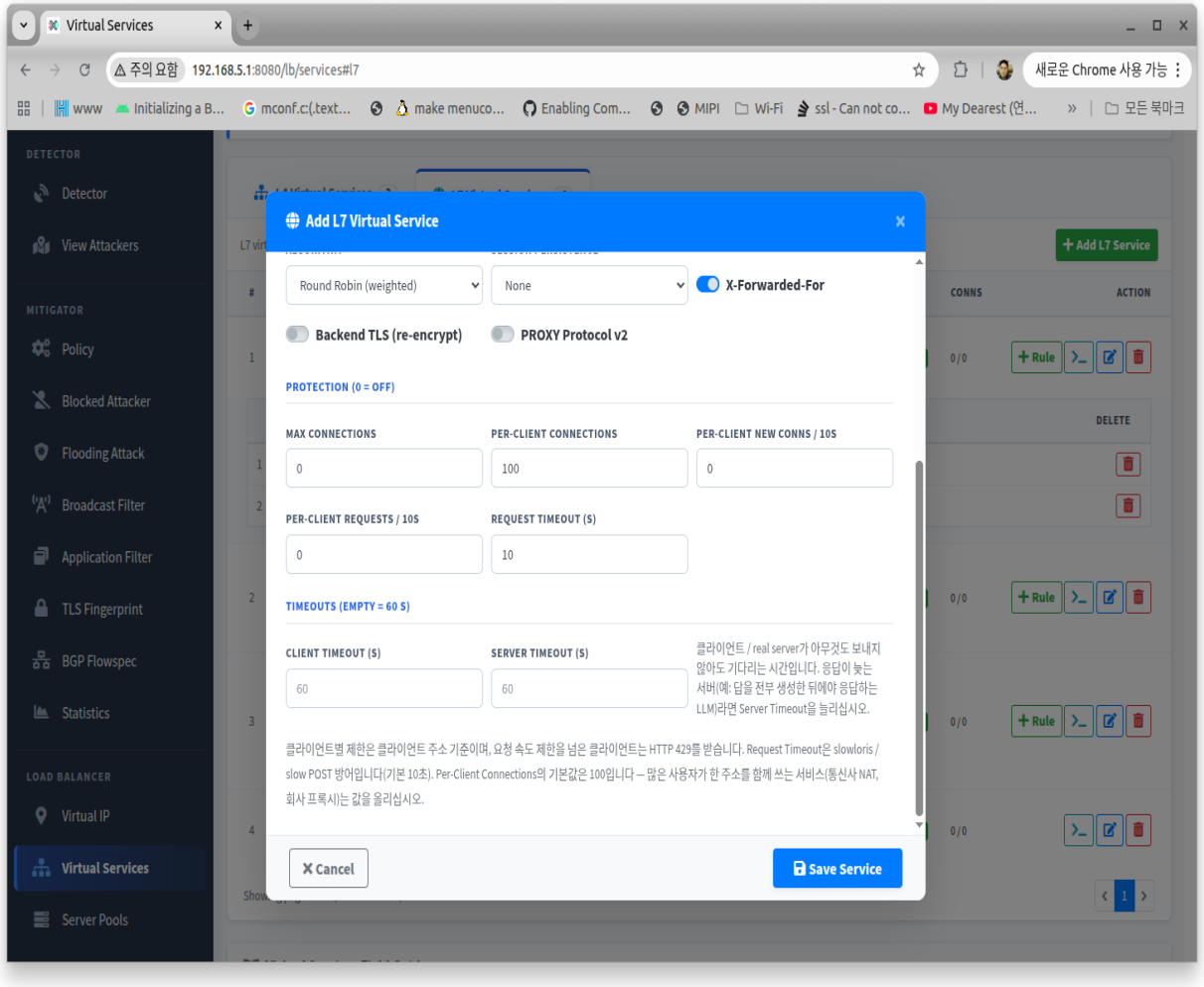


2. L4/L7 Load Balancer(6) – L7 Proxy Mode(6)

HTTP(S) Attack Protections

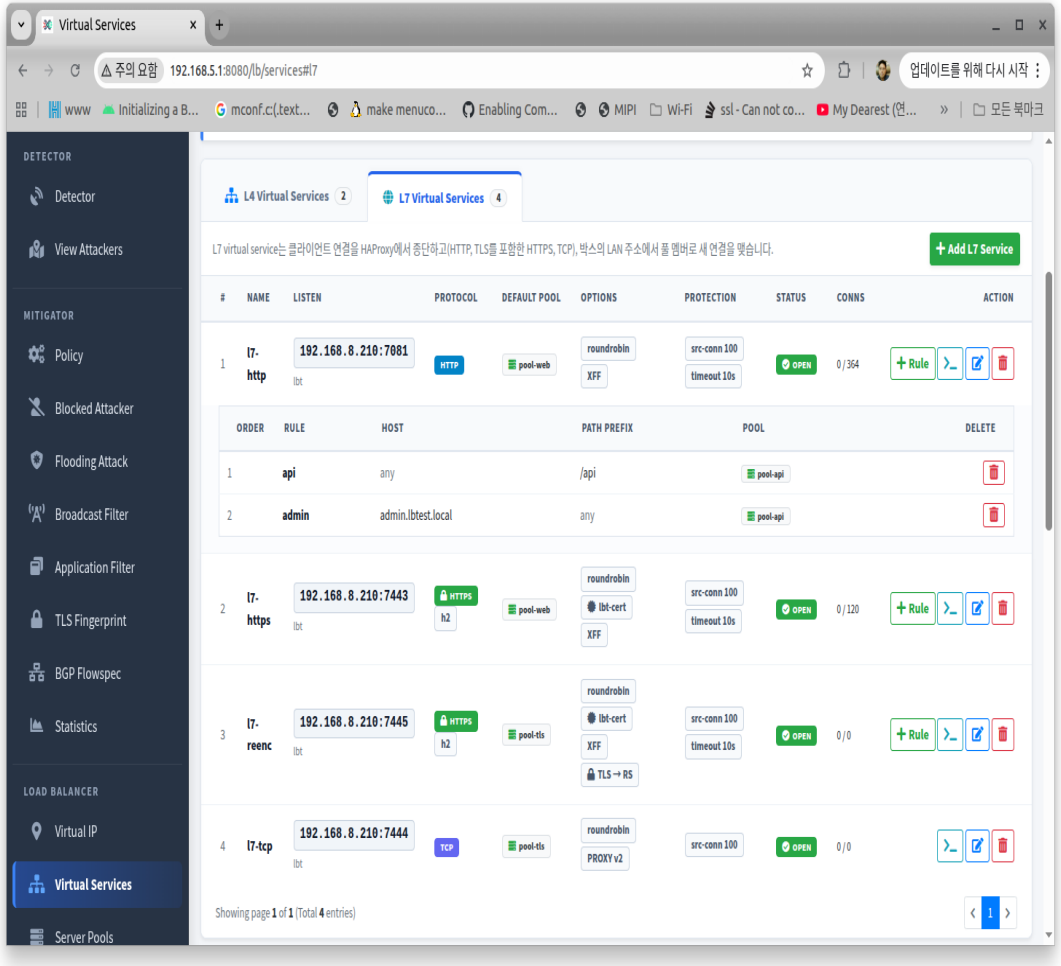
- Per-Clients Requests
  - ✓ req-rate-limit
- Per-Clients Connections
  - ✓ src-conn-limit
- Request Timeout
  - ✓ Slowloris protection



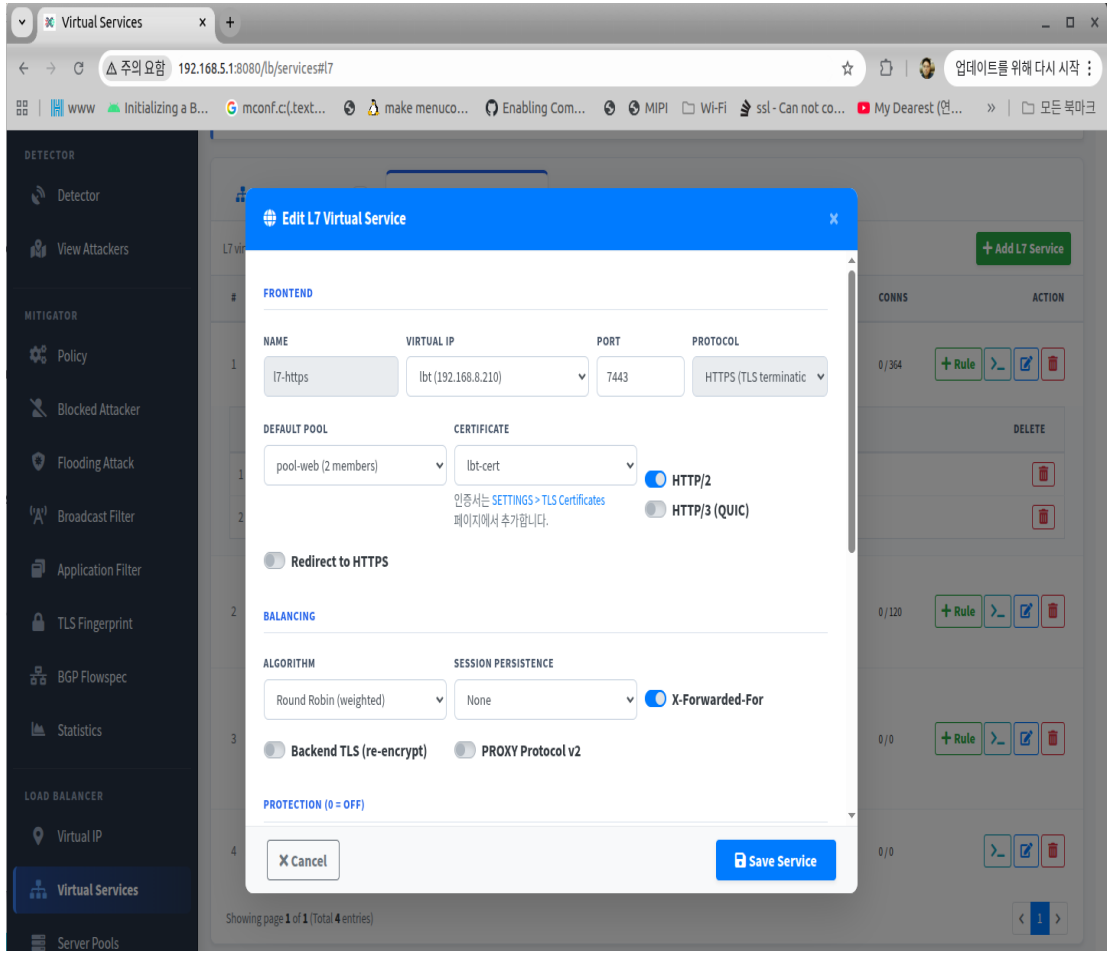


2. L4/L7 Load Balancer(6) – L7 Proxy Mode(7)

L7 Virtual Services Page - List

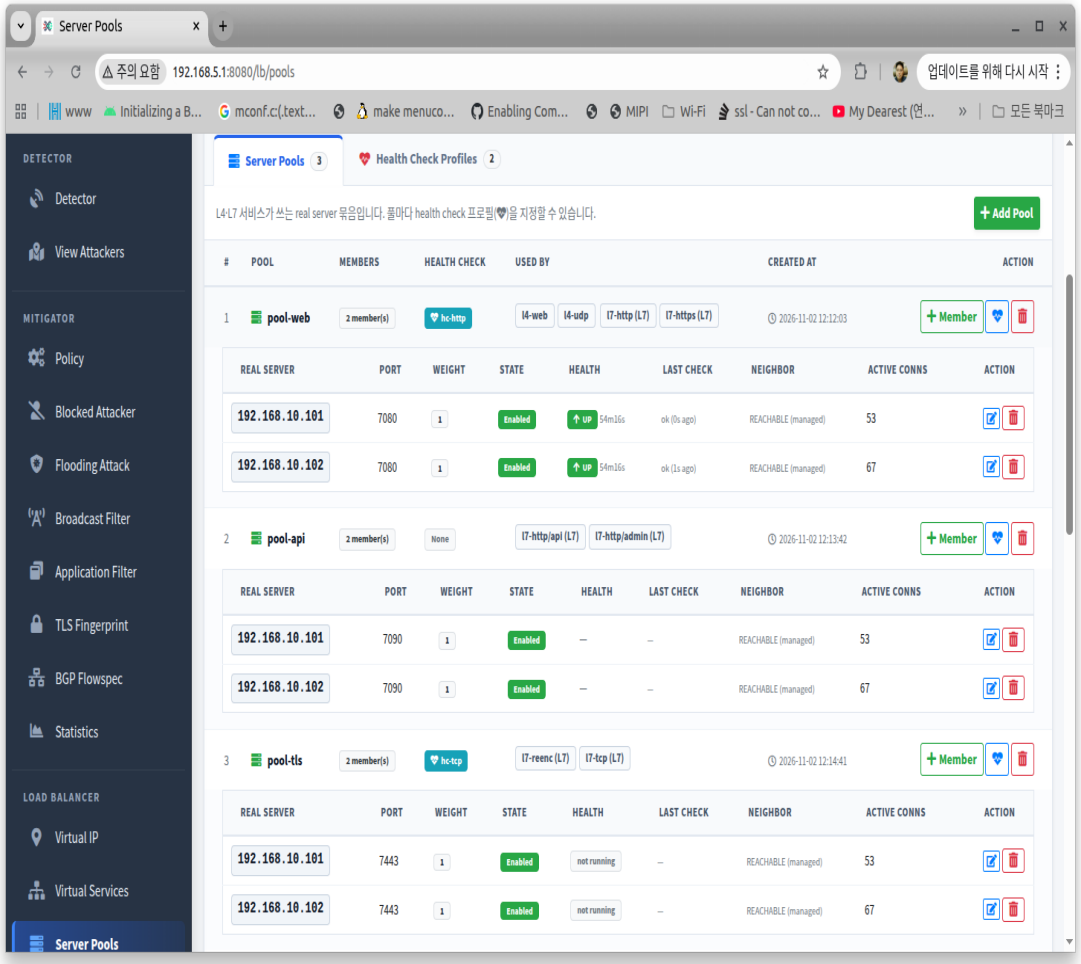


L7 Virtual Services Page – Settings Popup

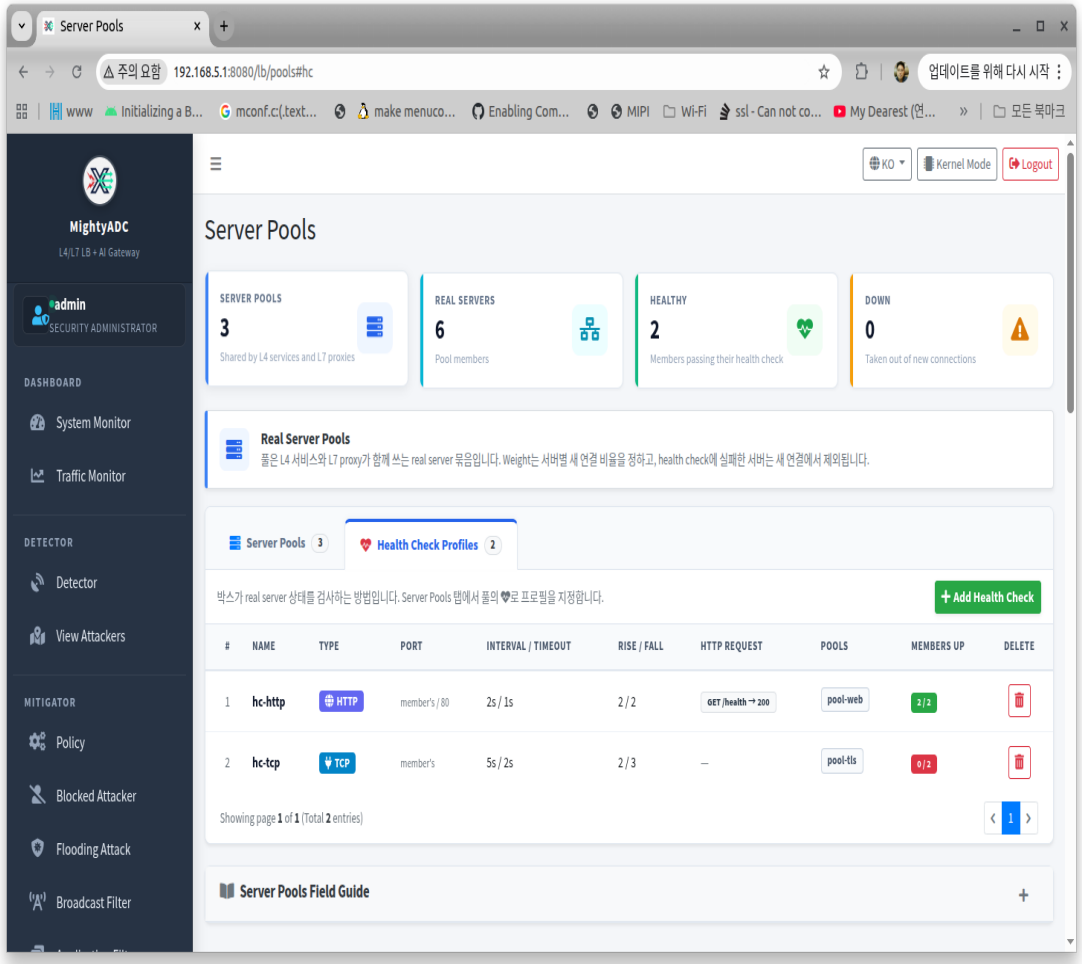


2. L4/L7 Load Balancer(6) – L7 Proxy Mode(8)

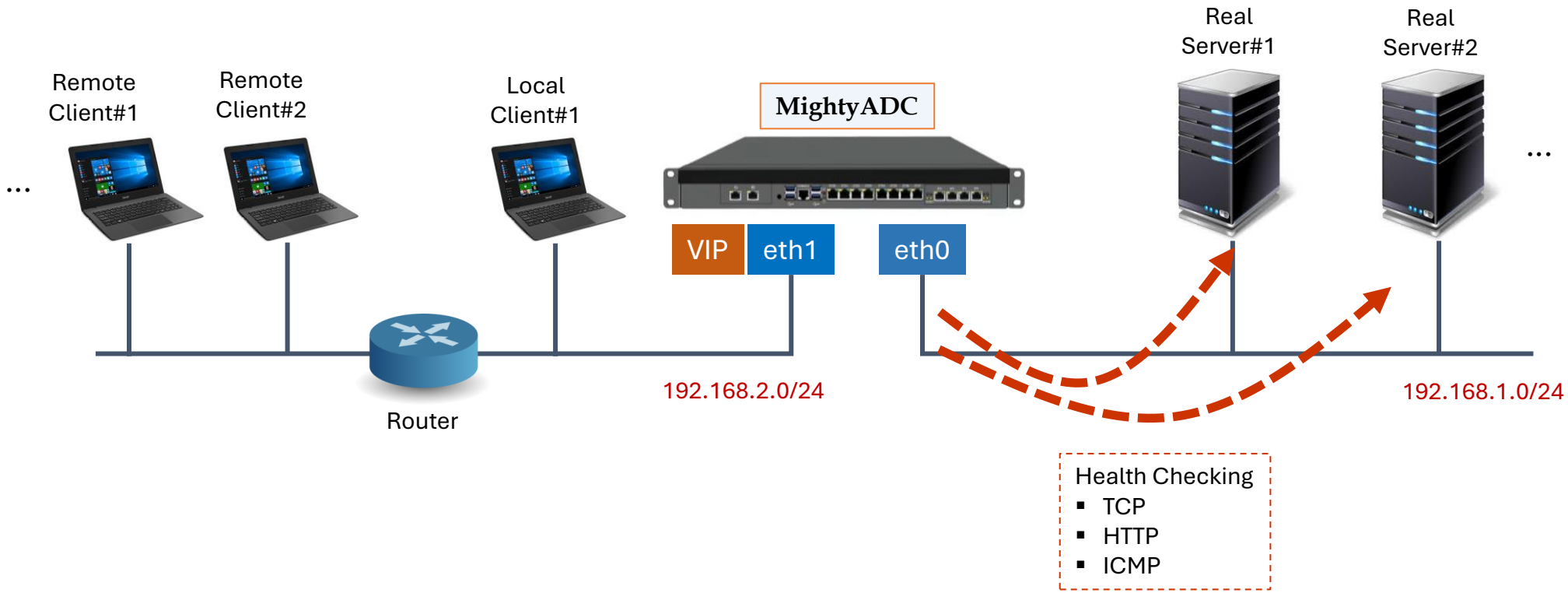
Real Server Pool - List



Real Server Pool – Health Check Rules

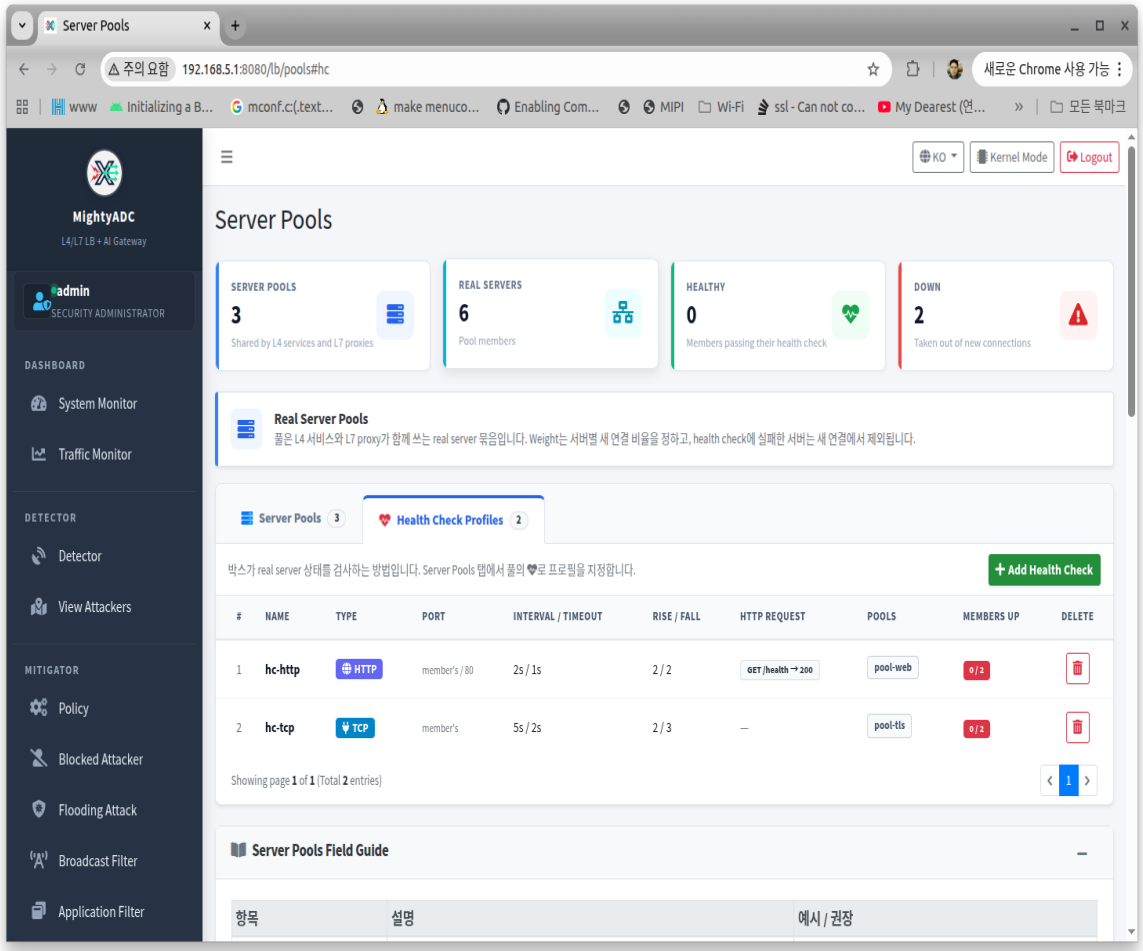


2. L4/L7 Load Balancer(7) – Health Check(1)

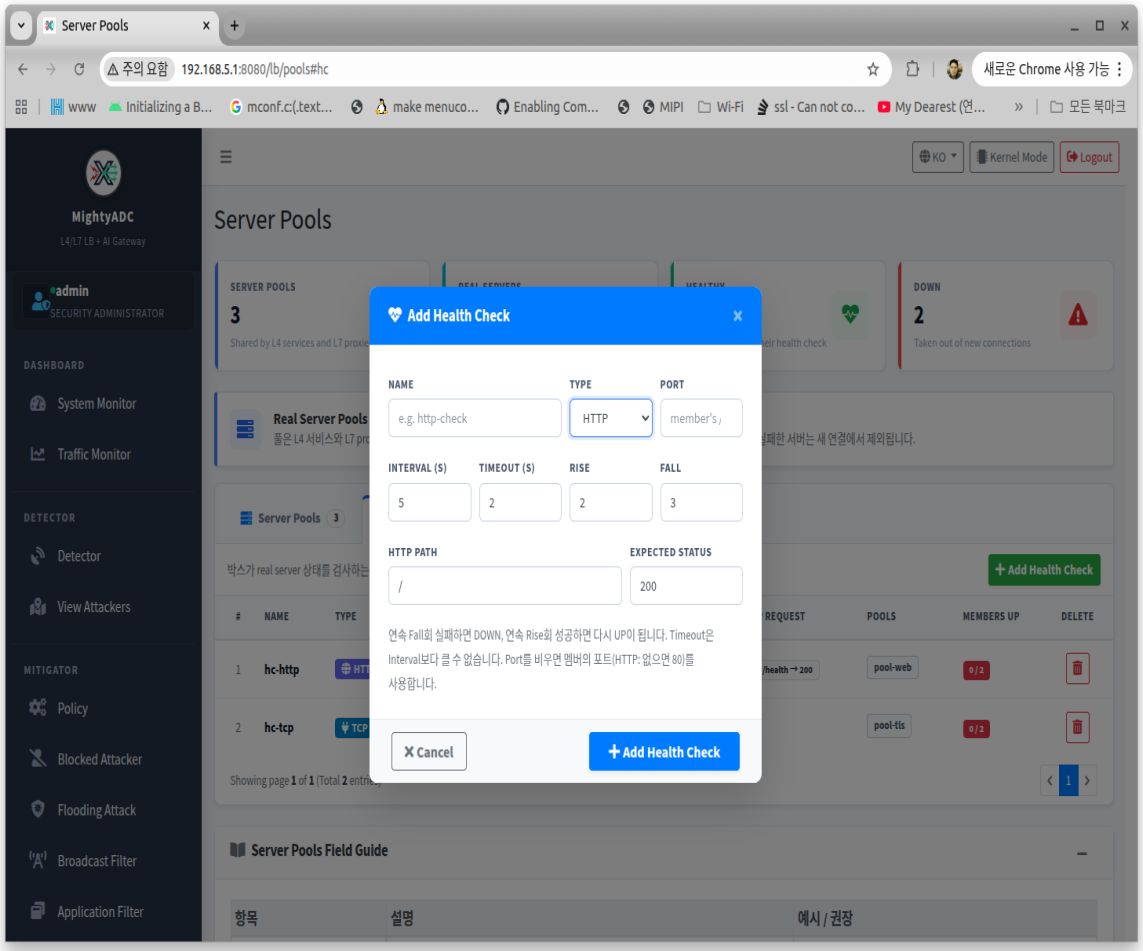


2. L4/L7 Load Balancer(7) – Health Check(2)

Load Balancer Server Pools -> Health Check Profiles Page(1)

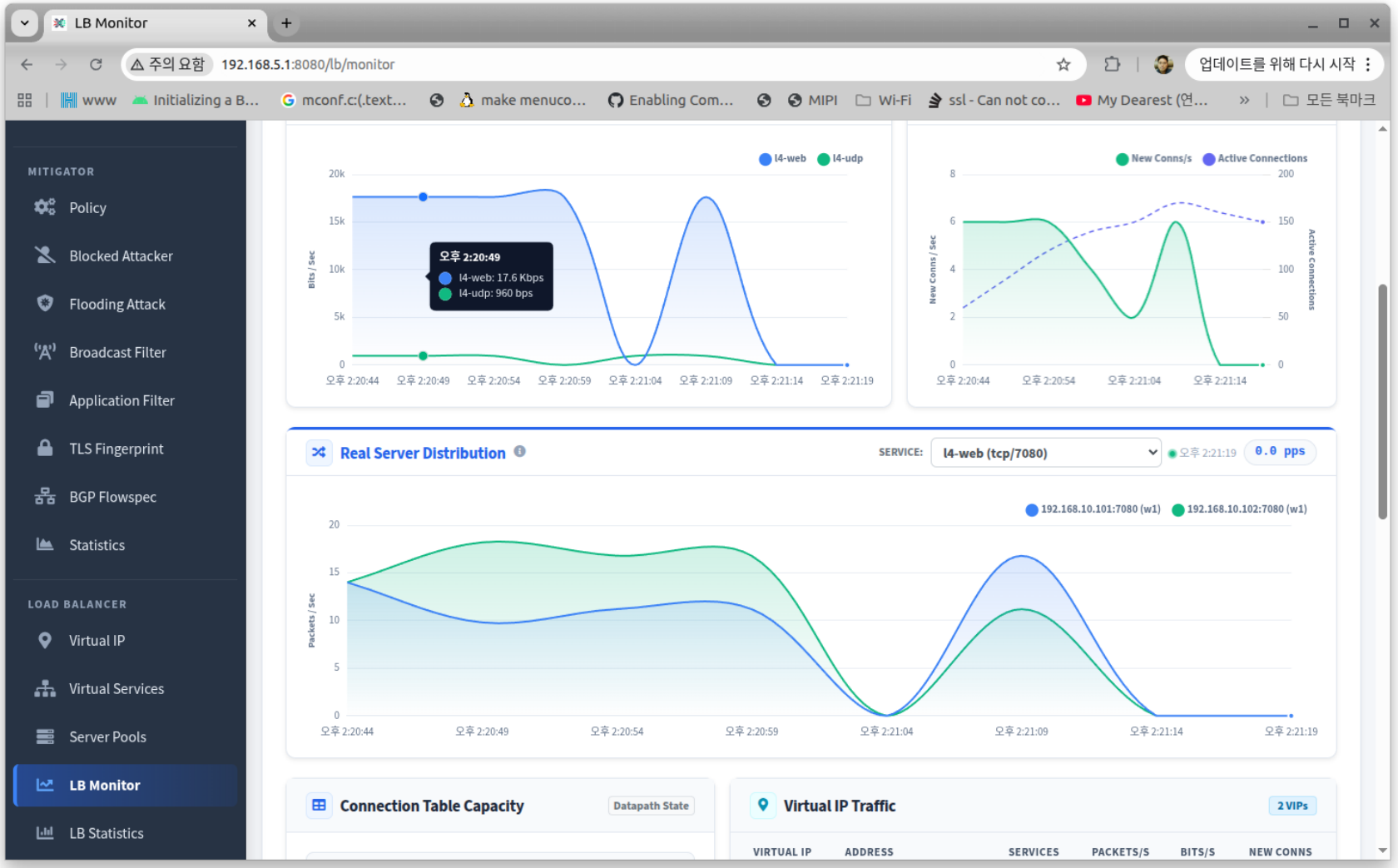


Load Balancer Server Pools -> Health Check Profiles Page(2)



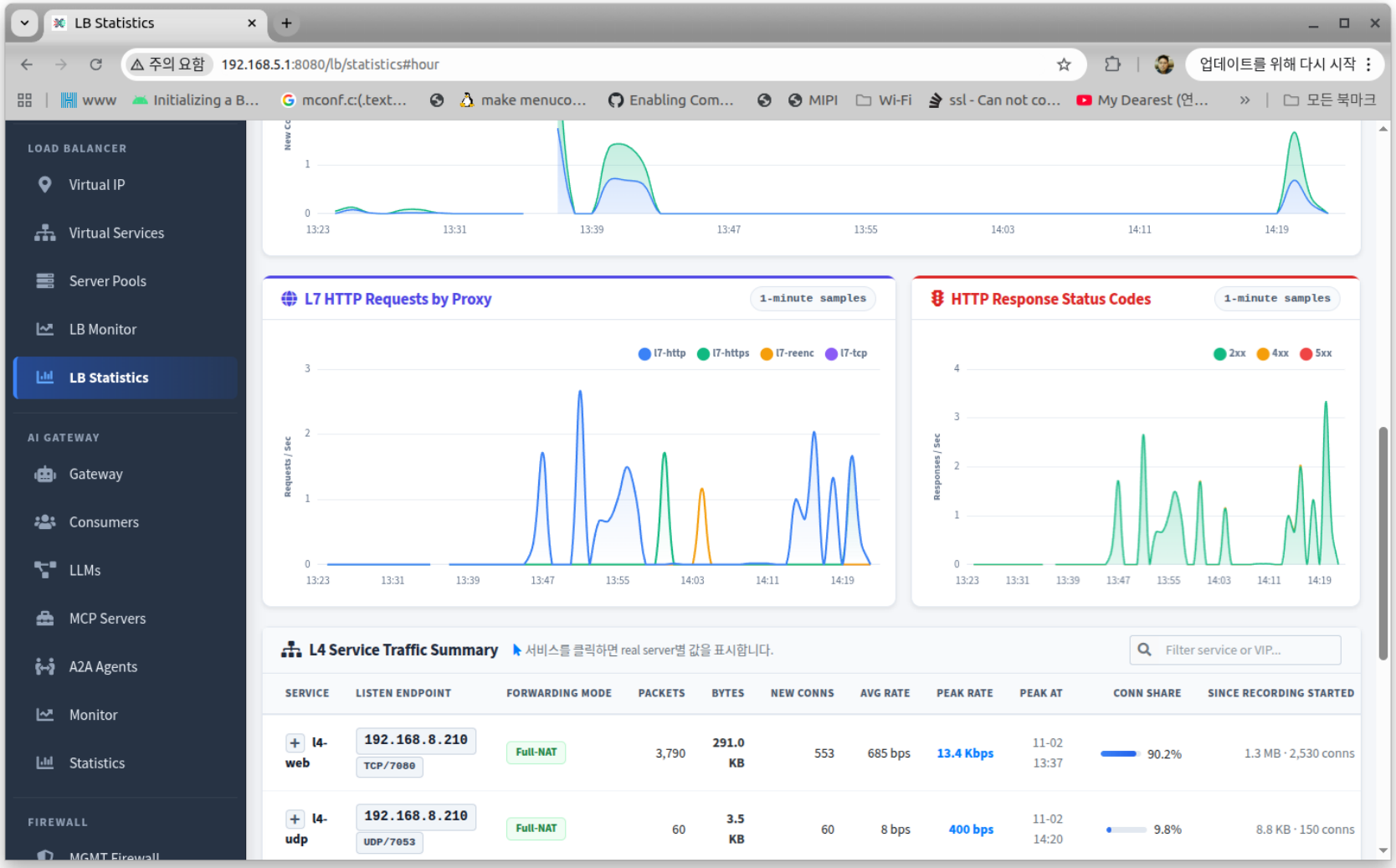
2. L4/L7 Load Balancer(8) – Monitoring & Statistics(1)

LB Monitor Page



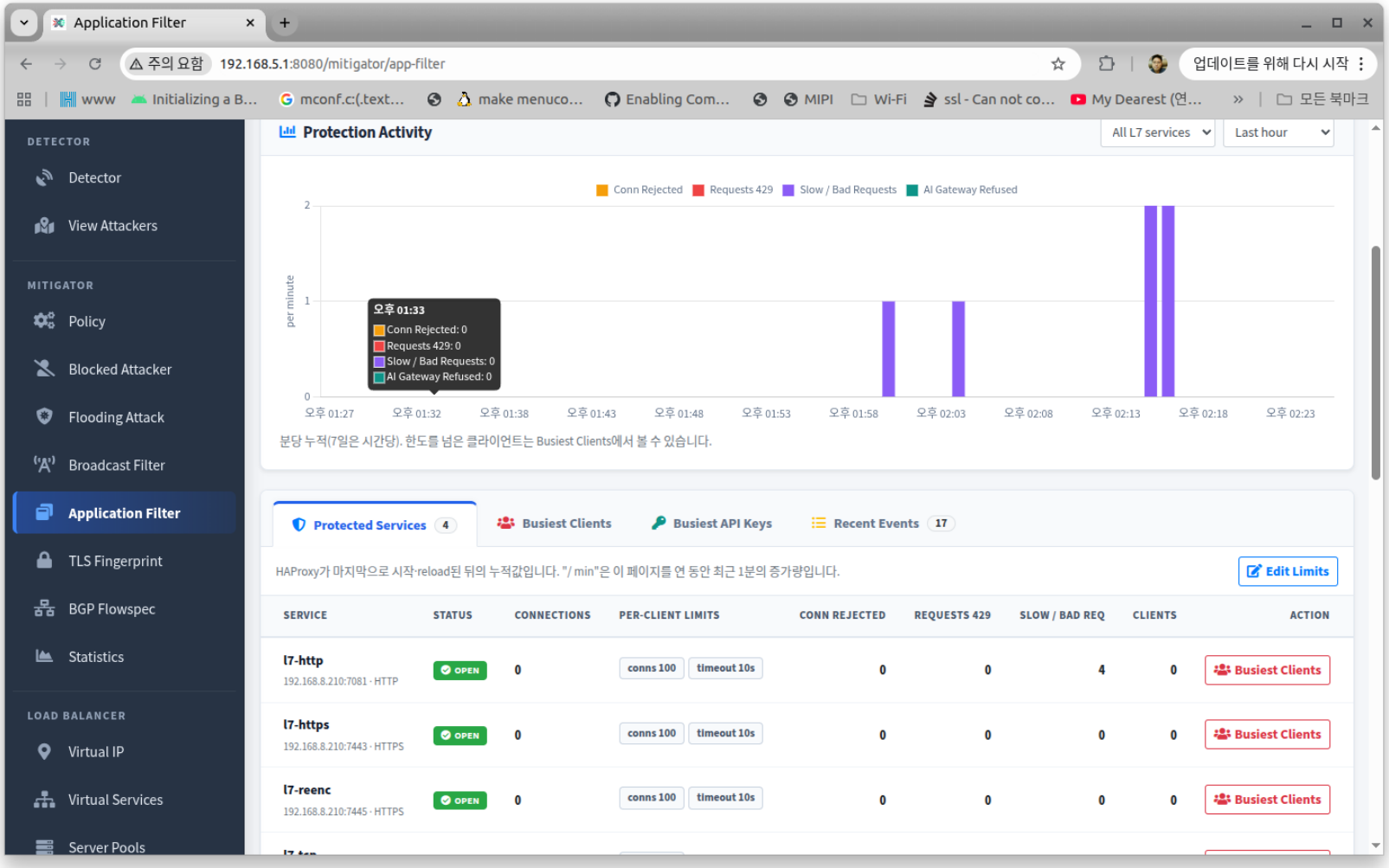
2. L4/L7 Load Balancer(8) – Monitoring & Statistics(2)

LB Statistics Page



2. L4/L7 Load Balancer(8) – Monitoring & Statistics(3)

Mitigator Application Filter Page



3. AI Gateway – LLM/MCP/A2A



- L7 Proxy(powered by agentgateway)
- L3 NAT(DNAT, FNAT)
- L3 DSR(IPIP, GRE, DSCP)
- L2 DSR

3. AI Gateway(1) – AgentGateway(1)

The image below was captured from agentgateway.dev.

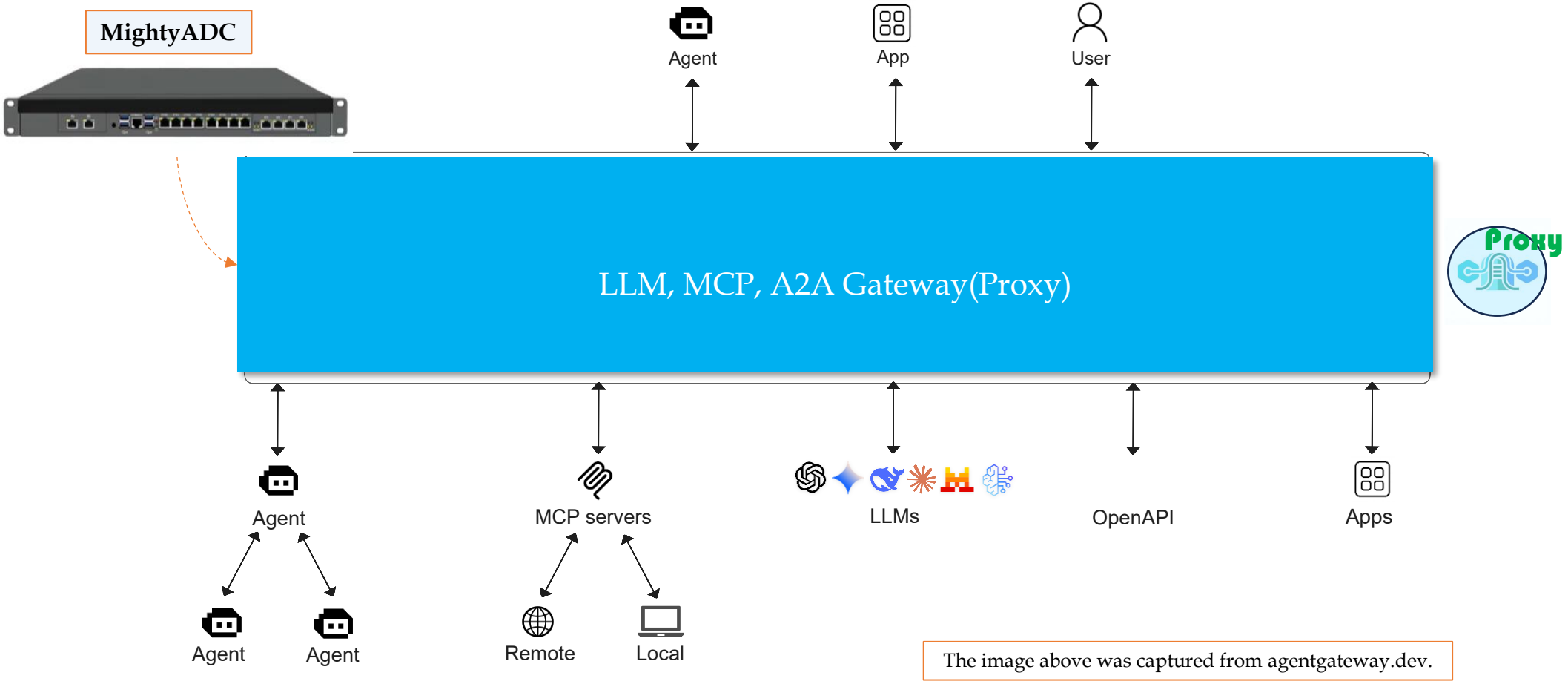


Ref) https://agentgateway.dev/



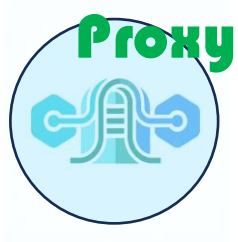
MightyADC

3. AI Gateway(1) – AgentGateway(2)



The image above was captured from agentgateway.dev.

3. AI Gateway(1) – AgentGateway(3)







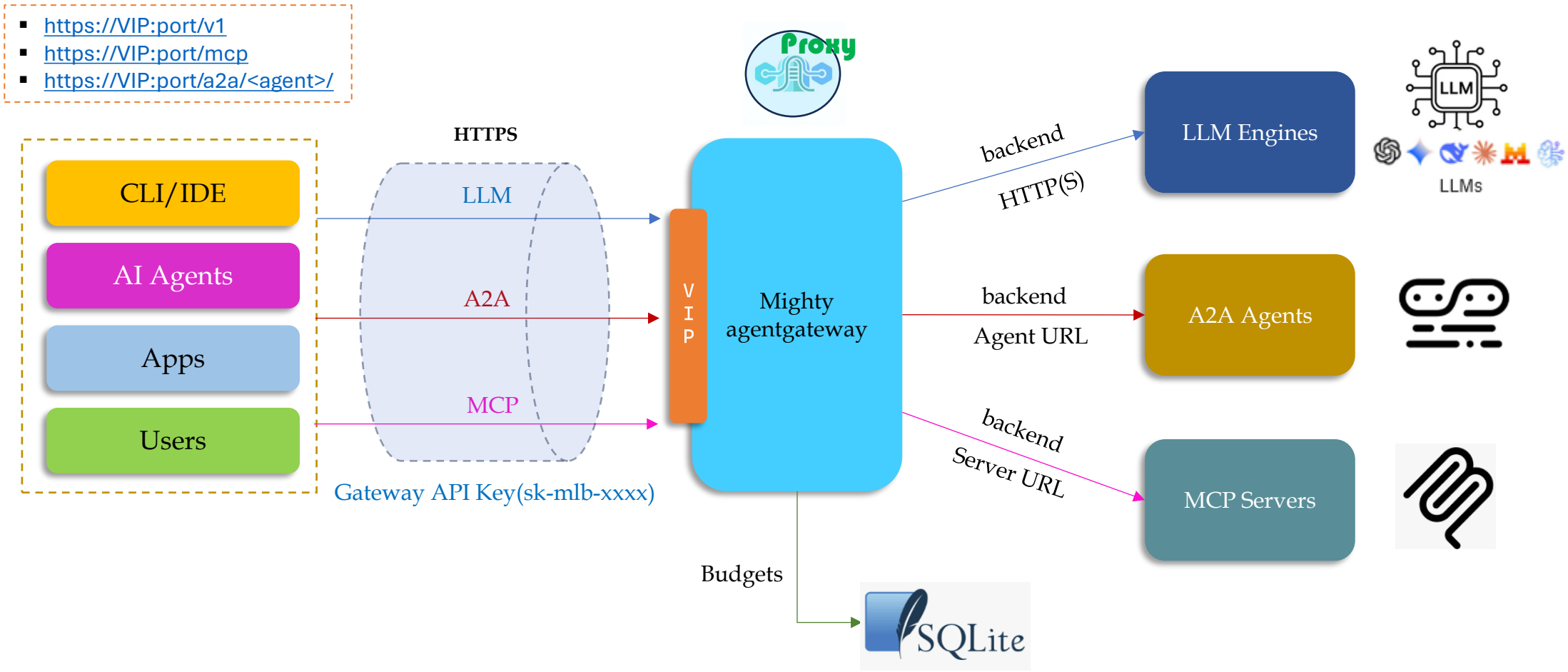


구분	LLM	MCP	A2A
client 접속 주소	`/v1` (OpenAI SDK), `/` (Anthropic·Gemini SDK)	`/mcp`	`/a2a/<agent>/`
연결 대상	자체 추론 서버, OpenAI·Anthropic·Gemini, OpenAI 호환 서비스 13종	MCP 서버(Streamable HTTP·SSE), REST API(OpenAPI → 도구)	A2A agent(프로토콜 1.0, v0.3 client 호환)
client 인증	Gateway API key	같은 API key	같은 API key
key별 권한	허용 모델	허용 서버·도구(기본 없음)	허용 agent
사용량 제한	key별 분당 요청 수, Tokens/USD budget	key별 분당 도구 호출 수	key별·agent별 분당 요청 수
내용 보호	Guardrails(개인정보·비밀정보 마스킹/차단)	—	—
장애 대응	backend 자동 제외·재시도·failover	서버 장애 시 나머지 도구 계속 사용	—
모니터링	요청·토큰·비용·지연(p50/p95)·첫 토큰 시간	도구 호출·오류·세션	메시지·스트림·task 호출·오류



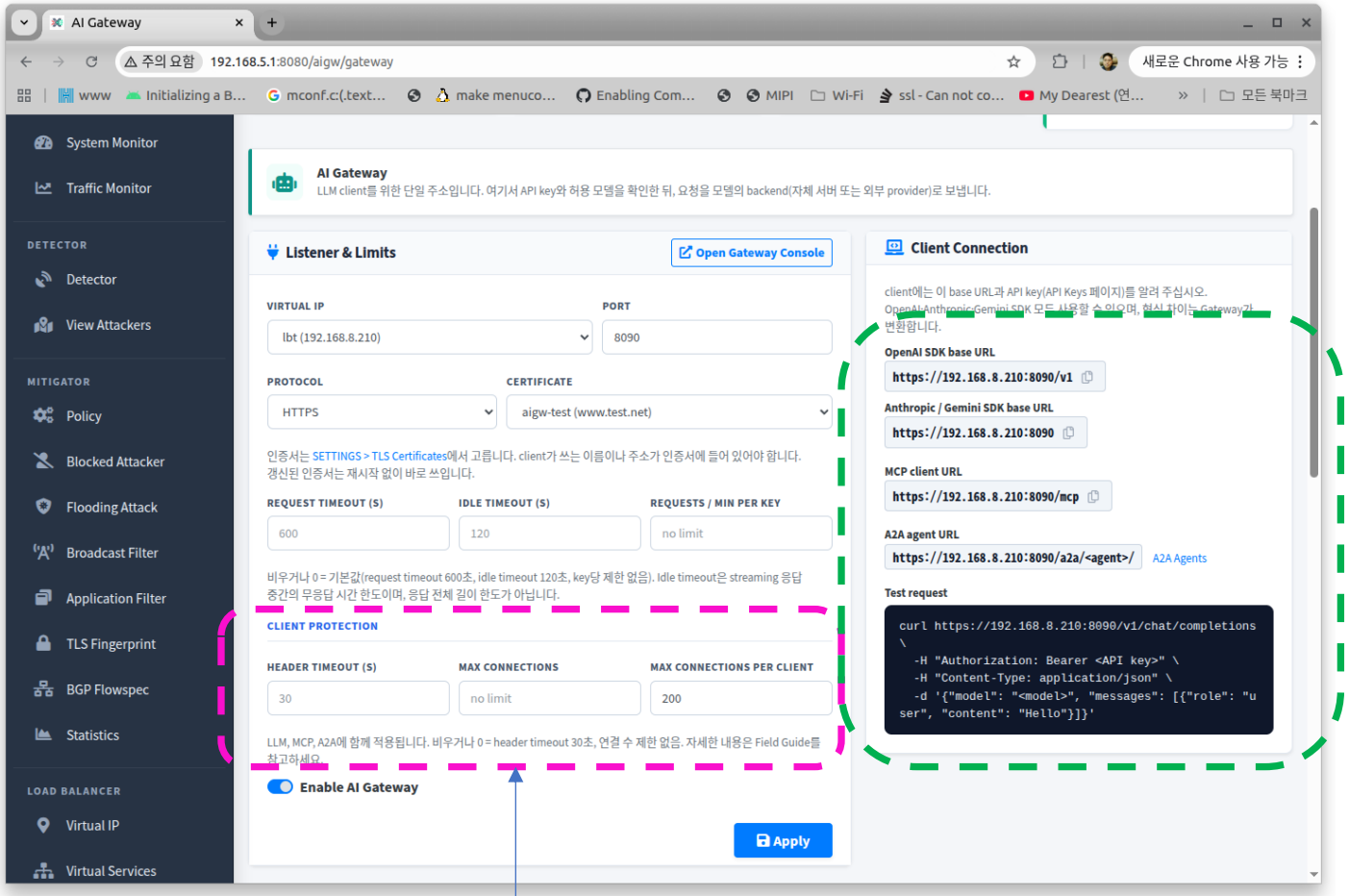
MightyADC

3. AI Gateway(1) – AgentGateway(4)



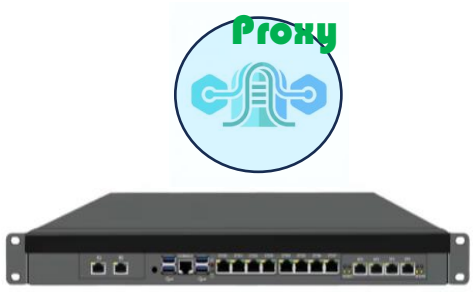
3. AI Gateway(1) – AgentGateway(5)

AI Gateway -> Gateway Page



Connection to AI Gateway

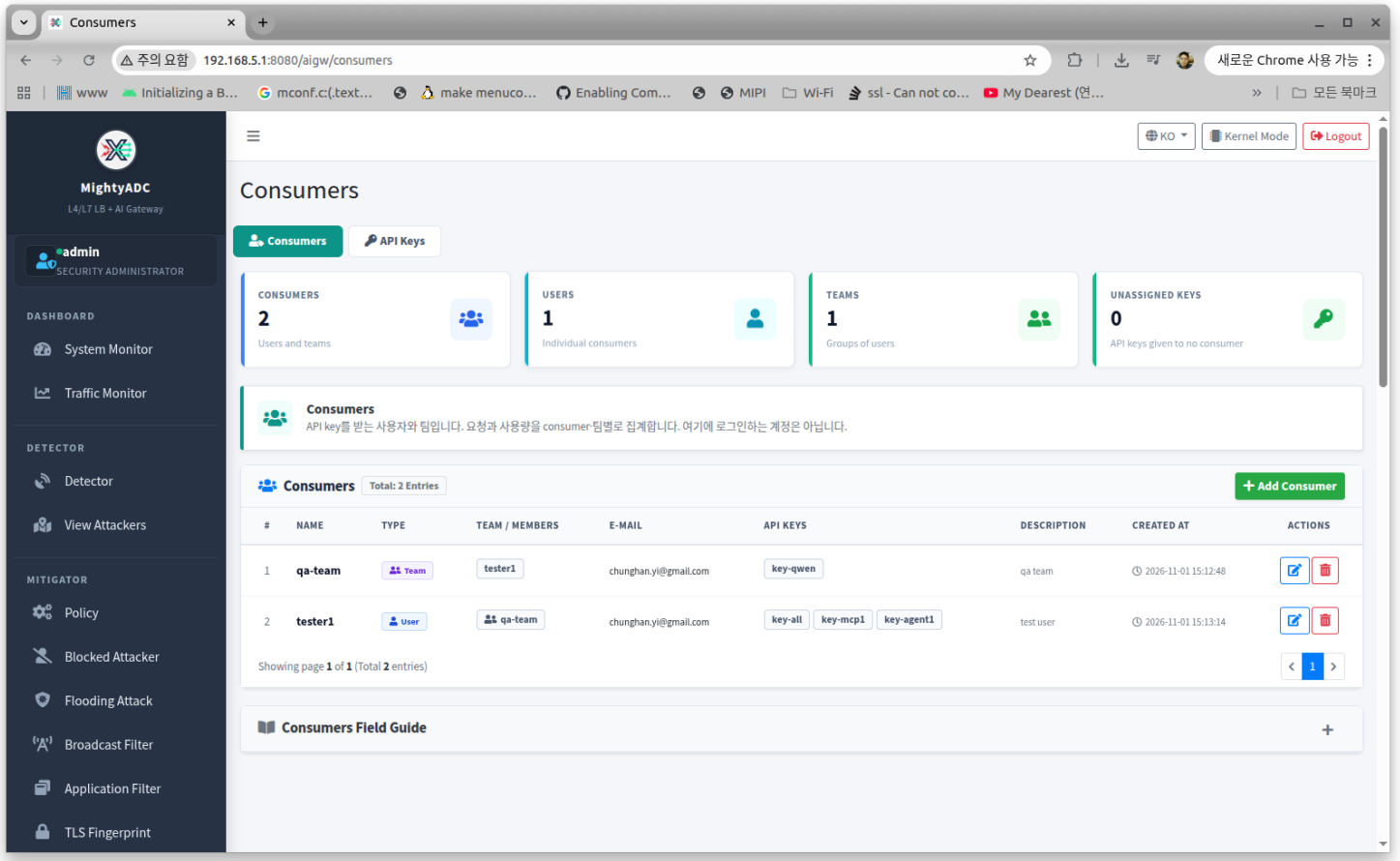
Application Filter



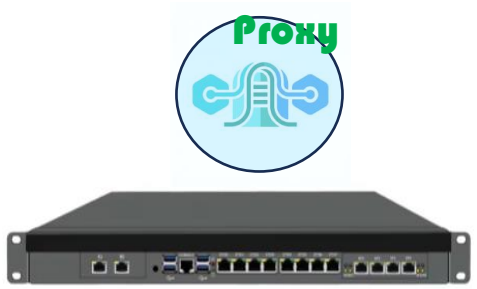
MightyADC

3. AI Gateway(1) – AgentGateway(6-1)

AI Gateway -> Consumers -> Consumers Page



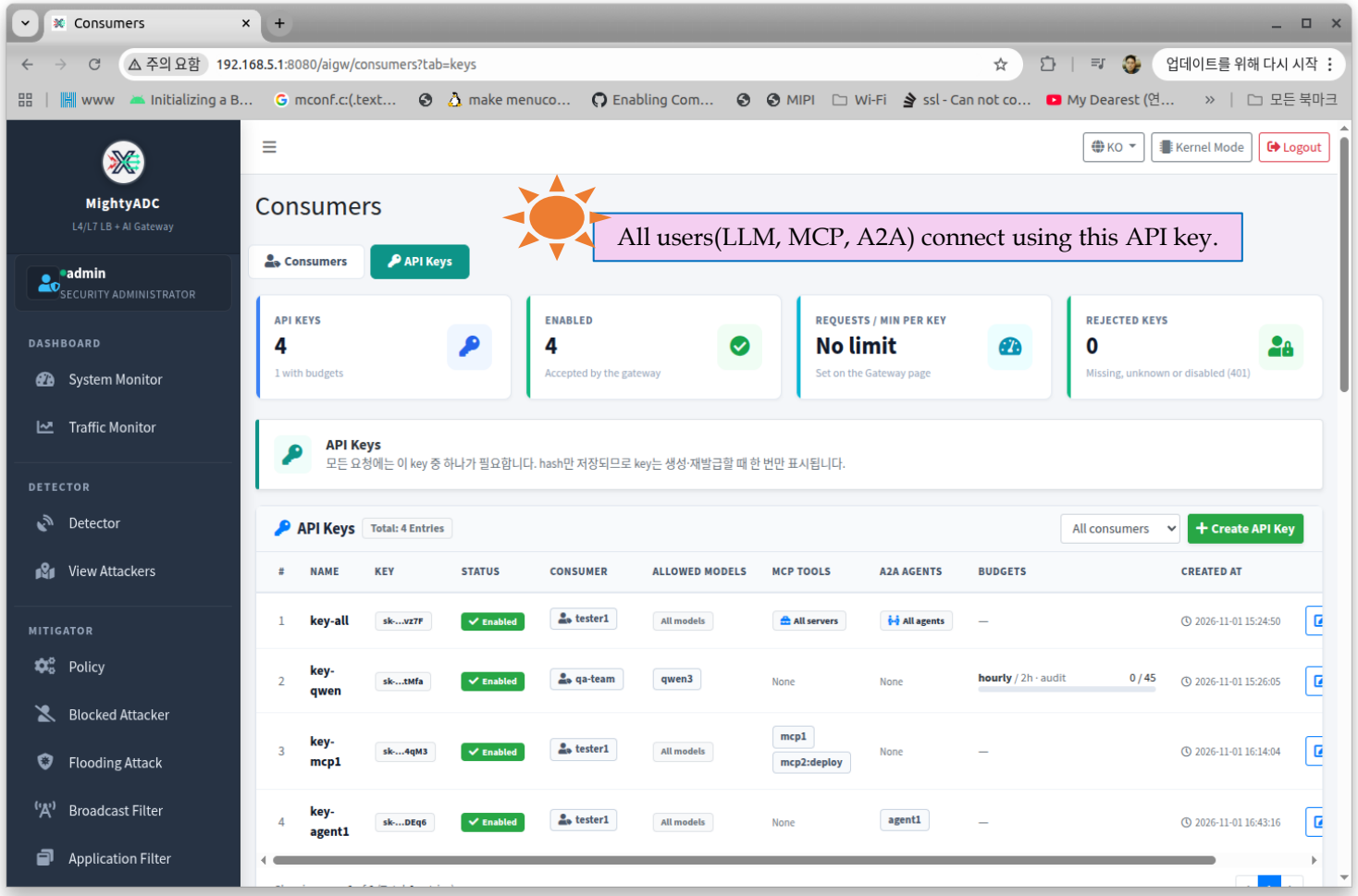
User Management Page



MightyADC

3. AI Gateway(1) – AgentGateway(6-2)

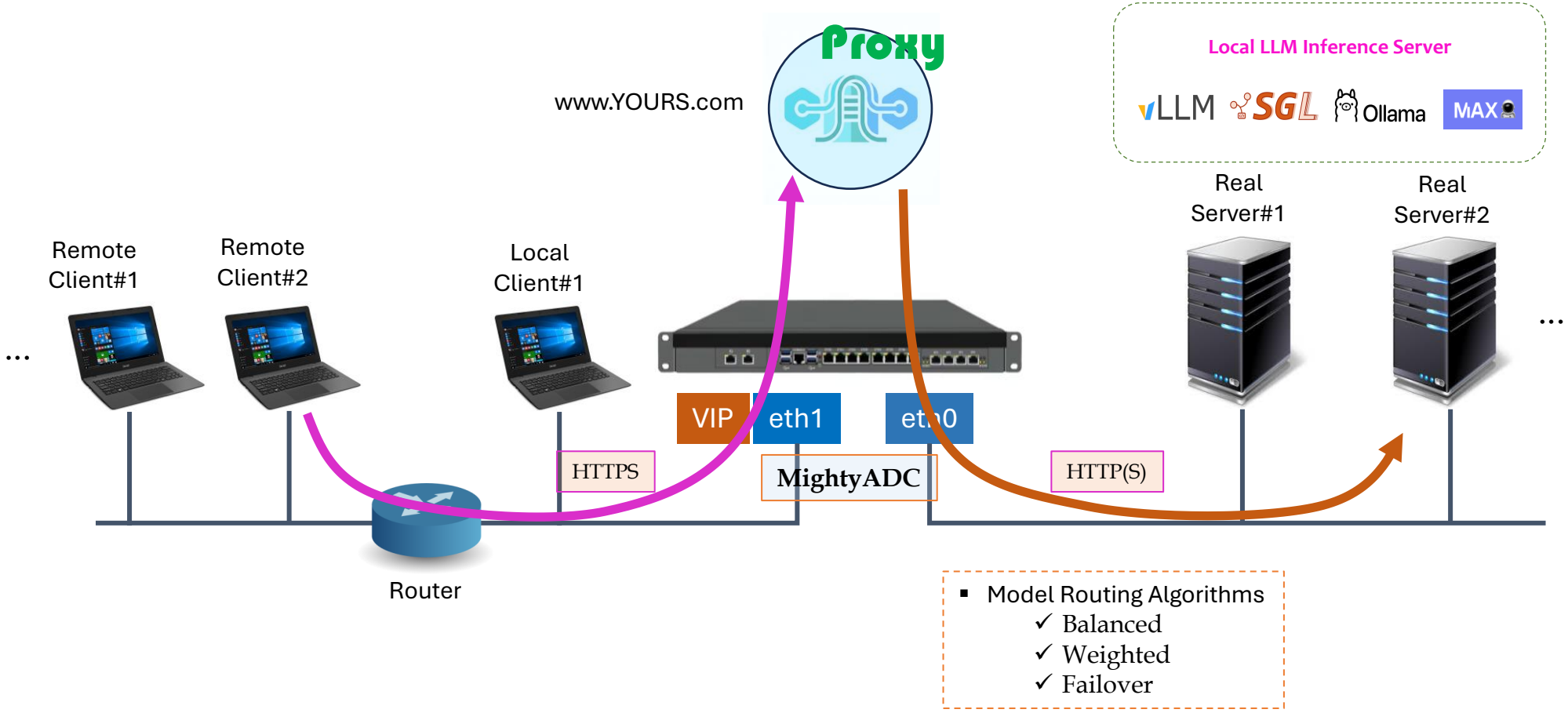
AI Gateway -> Consumers -> API Keys Page



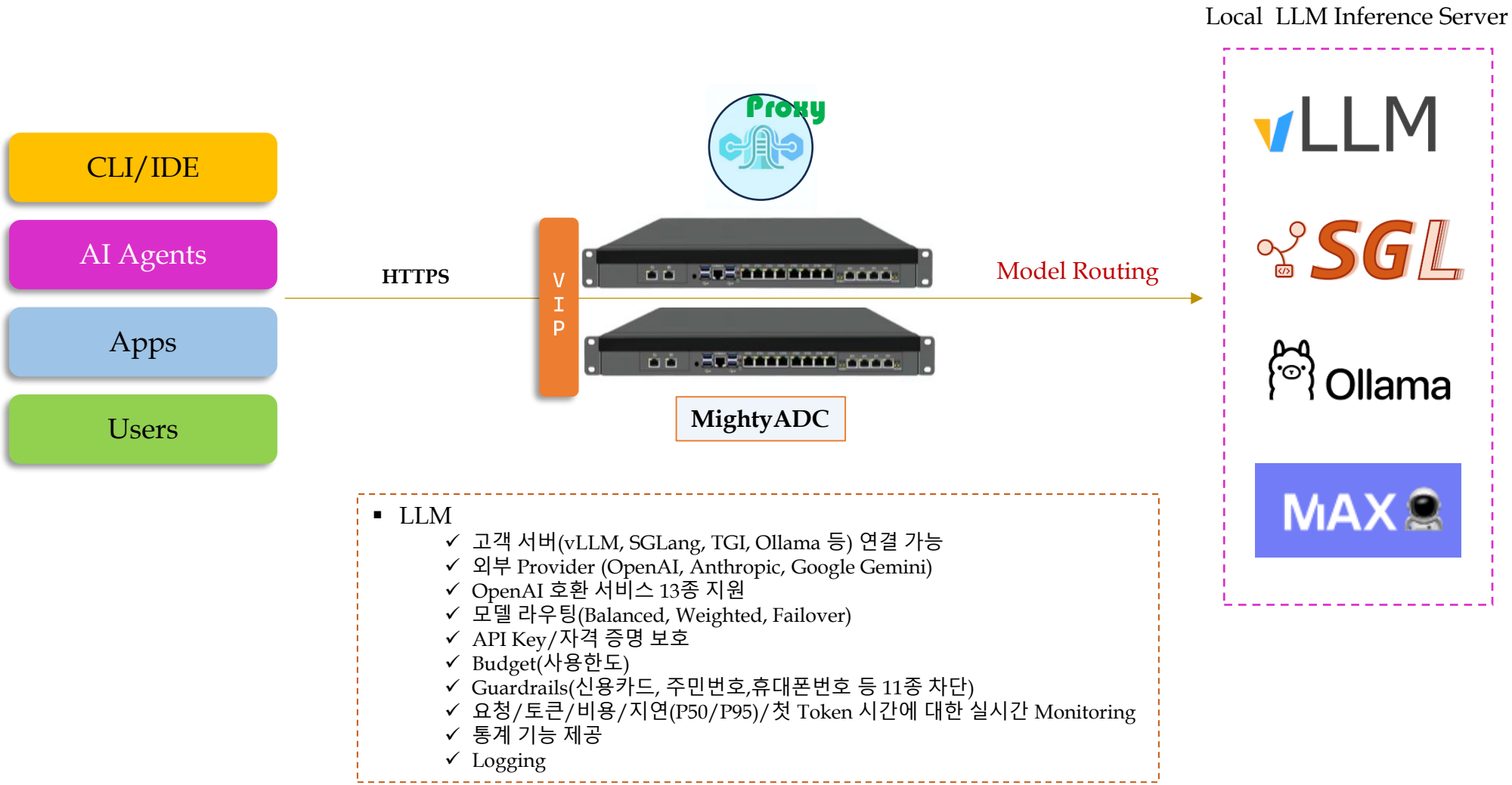


MightyADC

3. AI Gateway(2) – LLM Servers(1-1)

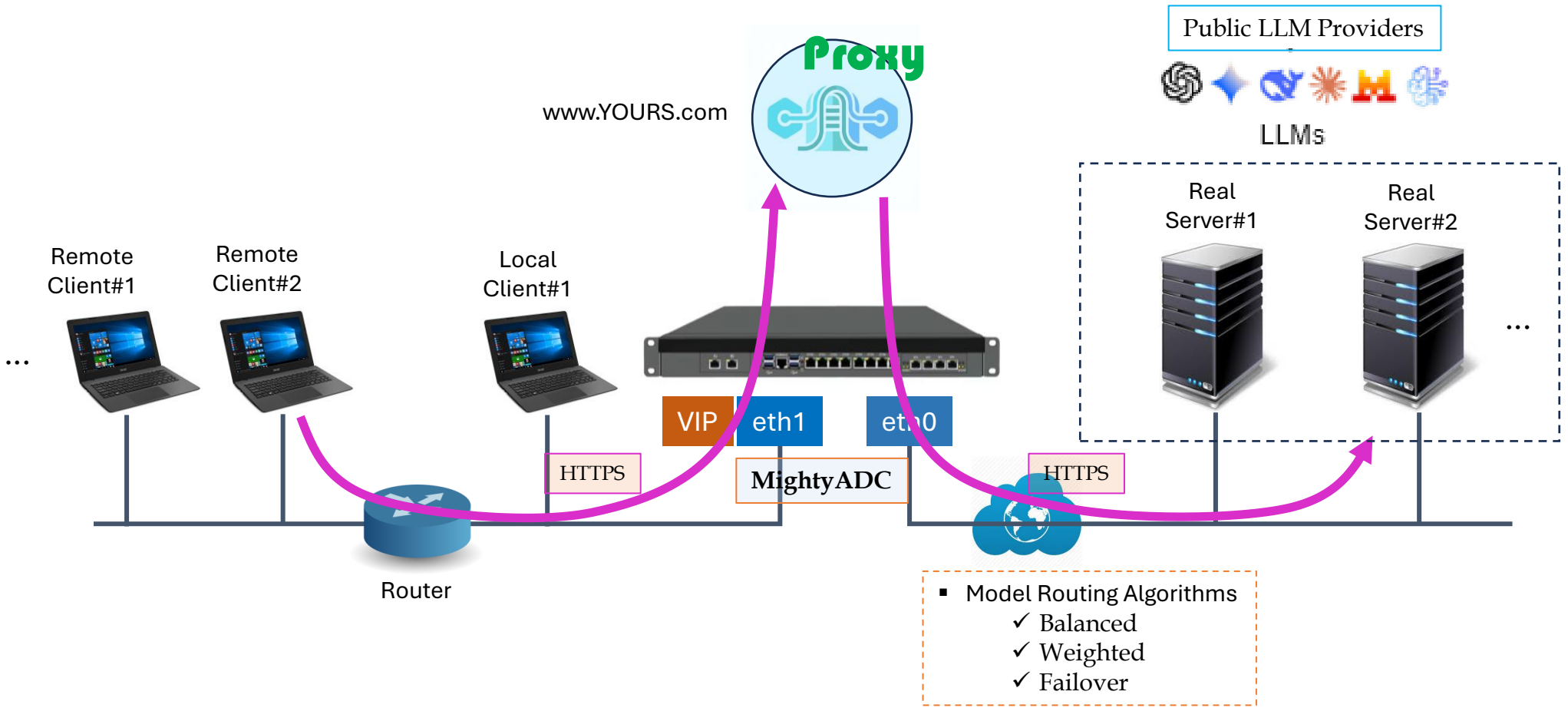


3. AI Gateway(2) – LLM Servers(1-2)

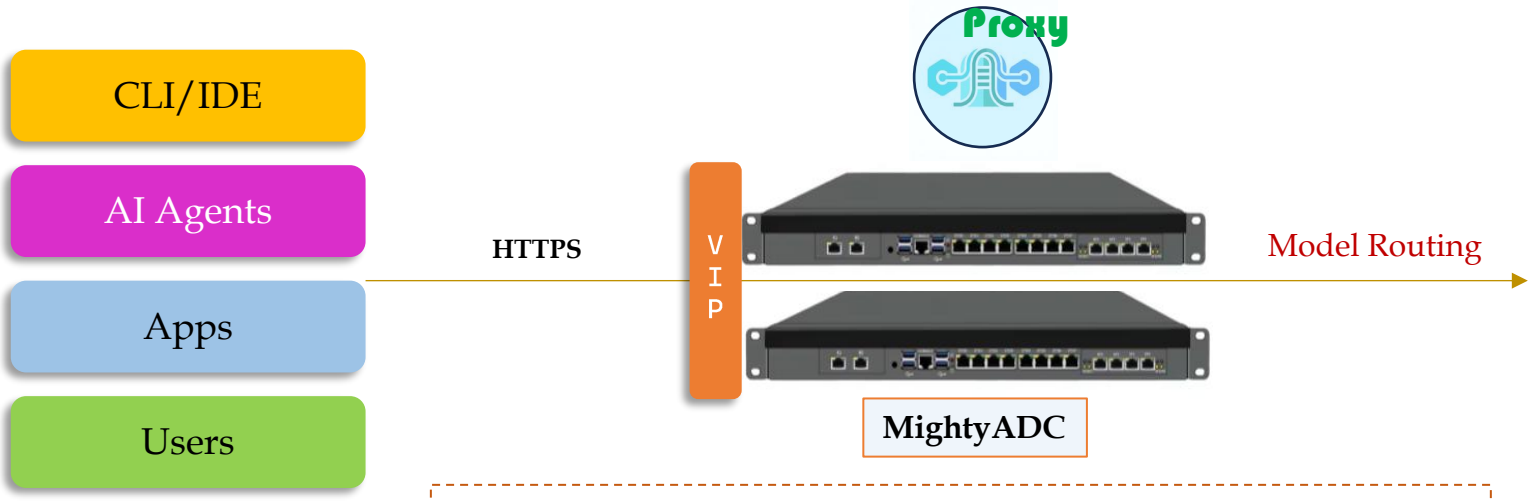


- LLM
  - ✓ 고객 서버(vLLM, SGLang, TGI, Ollama 등) 연결 가능
  - ✓ 외부 Provider (OpenAI, Anthropic, Google Gemini)
  - ✓ OpenAI 호환 서비스 13종 지원
  - ✓ 모델 라우팅(Balanced, Weighted, Failover)
  - ✓ API Key/자격 증명 보호
  - ✓ Budget(사용한도)
  - ✓ Guardrails(신용카드, 주민번호,휴대폰번호 등 11종 차단)
  - ✓ 요청/토큰/비용/지연(P50/P95)/첫 Token 시간에 대한 실시간 Monitoring
  - ✓ 통계 기능 제공
  - ✓ Logging

3. AI Gateway(2) – LLM Servers(2-1)



3. AI Gateway(2) – LLM Servers(2-2)



Public LLM Providers

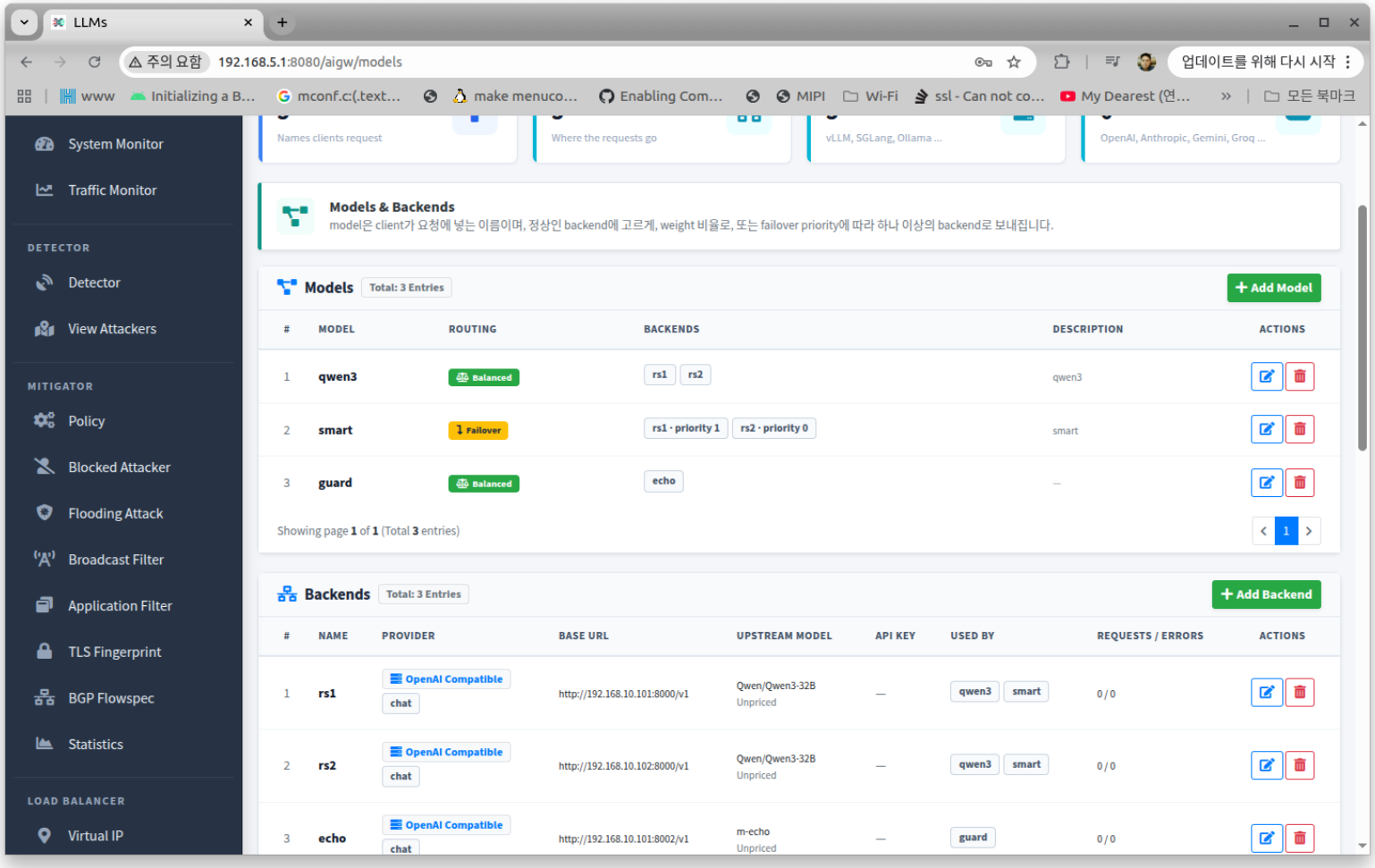
- OpenAI
- Anthropic
- Bedrock
- Azure
- Gemini
- Vertex AI
- Copilot
- Cohere
- Ollama
- Baseten
- Cerebras
- Deepinfra
- Deepseek
- Groq
- Hugging Face
- Mistral
- OpenRouter
- Together AI
- xAI
- Fireworks

- LLM
  - ✓ 고객 서버(vLLM, SGLang, TGI, Ollama 등) 연결 가능
  - ✓ 외부 Provider (OpenAI, Anthropic, Google Gemini)
  - ✓ OpenAI 호환 서비스 13종 지원
  - ✓ 모델 라우팅(Balanced, Weighted, Failover)
  - ✓ API Key/자격 증명 보호
  - ✓ Budget(사용한도)
  - ✓ Guardrails(신용카드, 주민번호,휴대폰번호 등 11종 차단)
  - ✓ 요청/토큰/비용/지연(P50/P95)/첫 Token 시간에 대한 실시간 Monitoring
  - ✓ 통계 기능 제공
  - ✓ Logging

The image above was captured from agentgateway.dev.

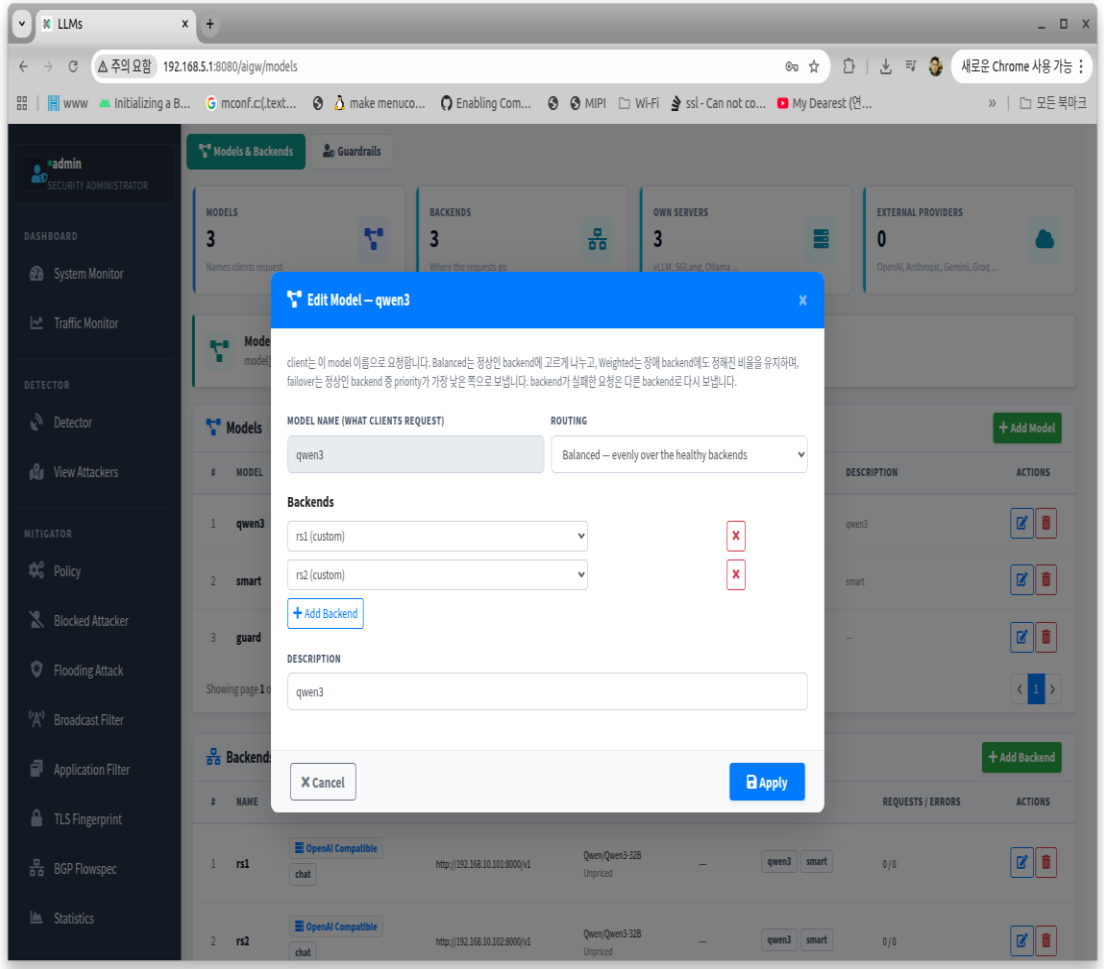
3. AI Gateway(2) – LLM Servers(3-1)

AI Gateway -> LLMs -> Models & Backends Page

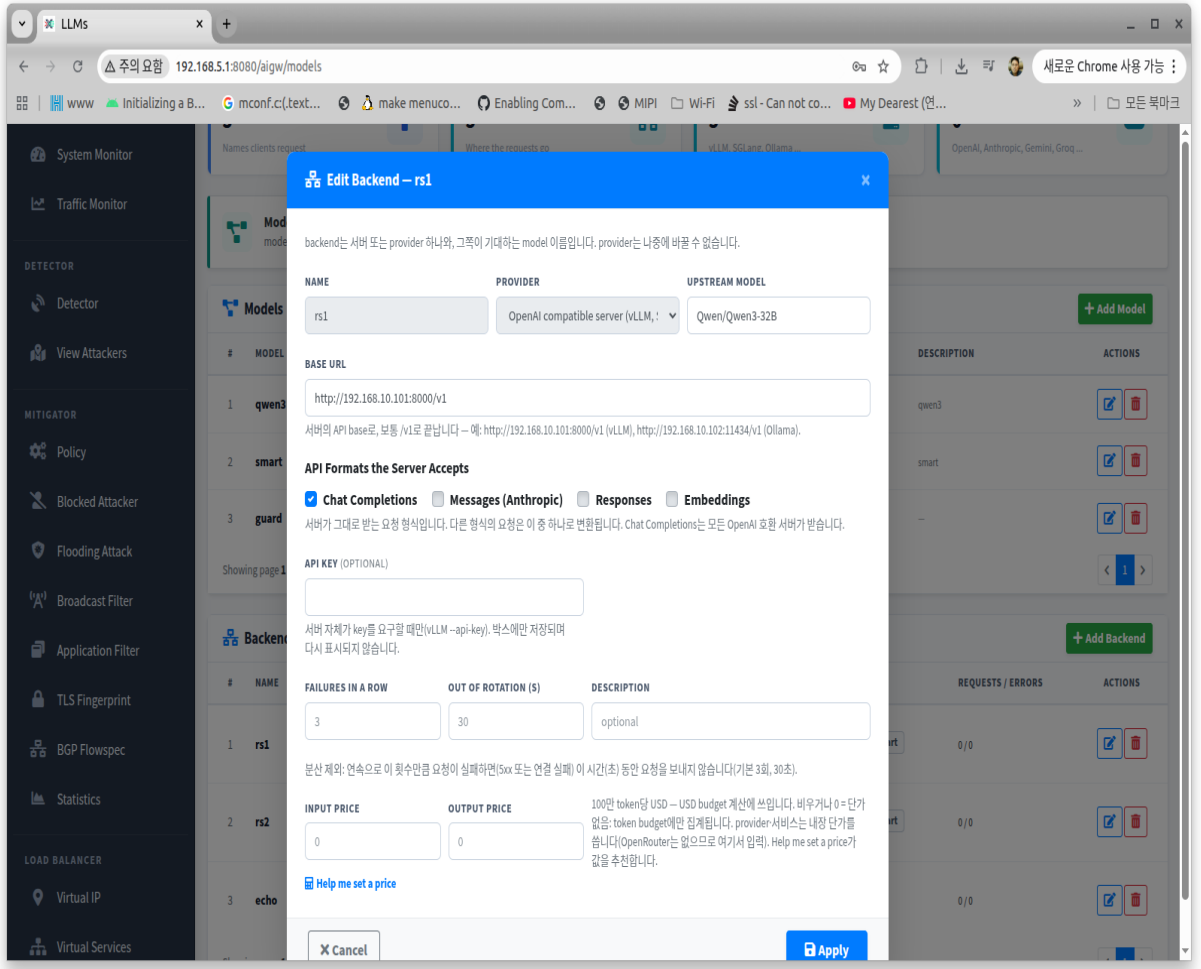


3. AI Gateway(2) – LLM Servers(3-2)

AI Gateway -> LLMs -> Models & Backends -> Model Page



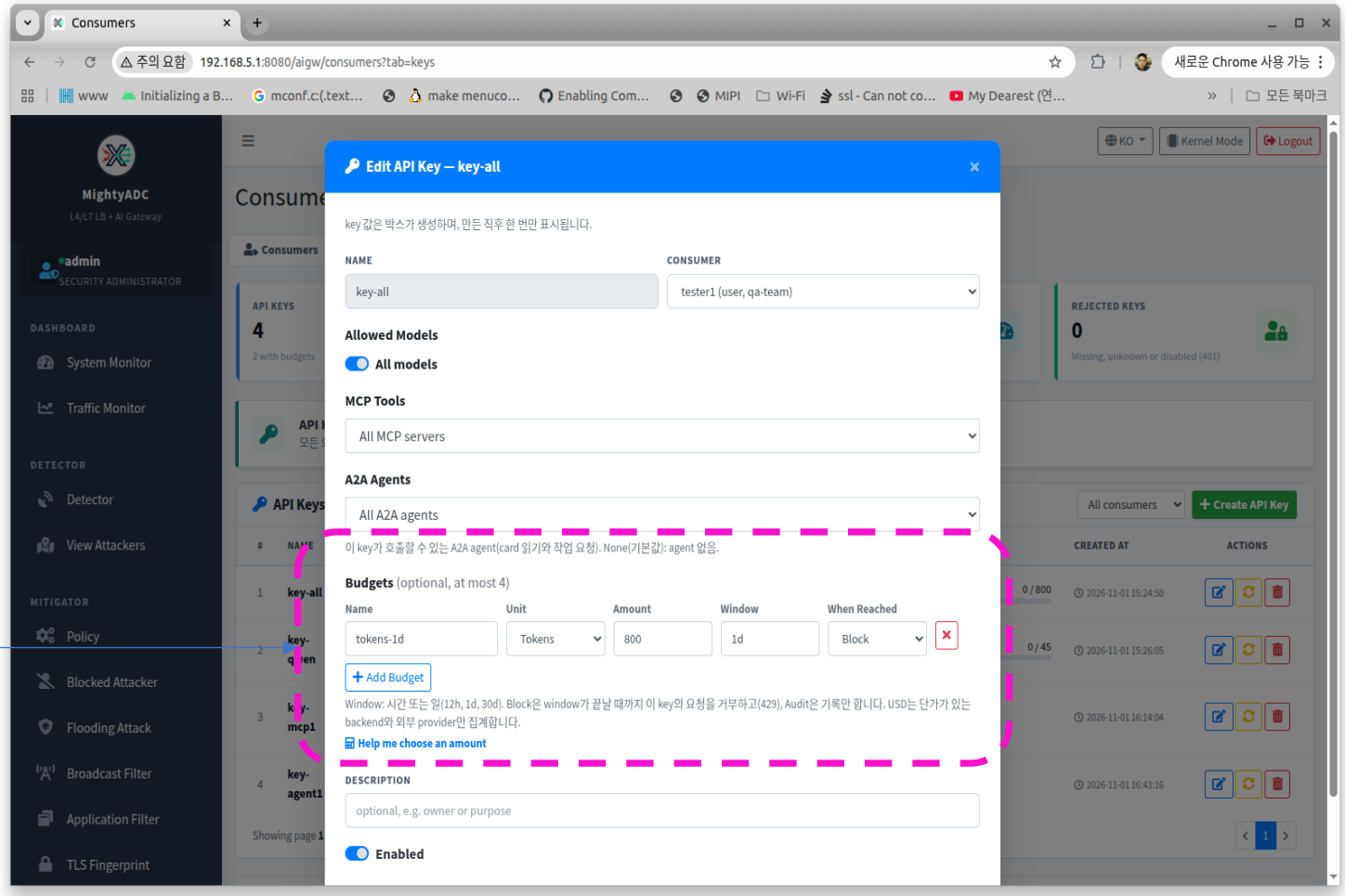
AI Gateway -> LLMs -> Models & Backends -> Backend Page



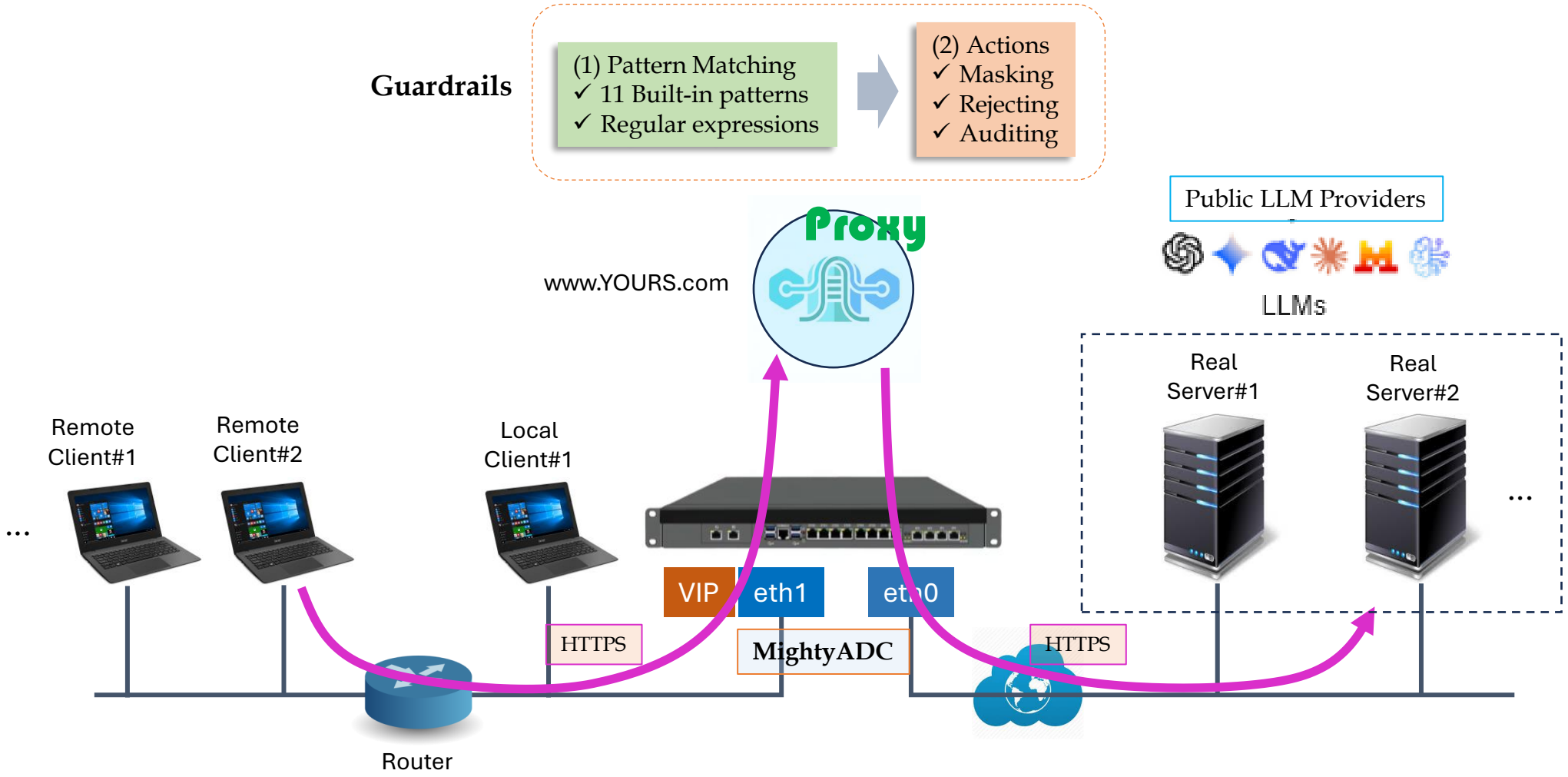
3. AI Gateway(2) – LLM Servers(4)

AI Gateway -> Consumers -> API Keys Page -> Budgets Settings

Budget settings



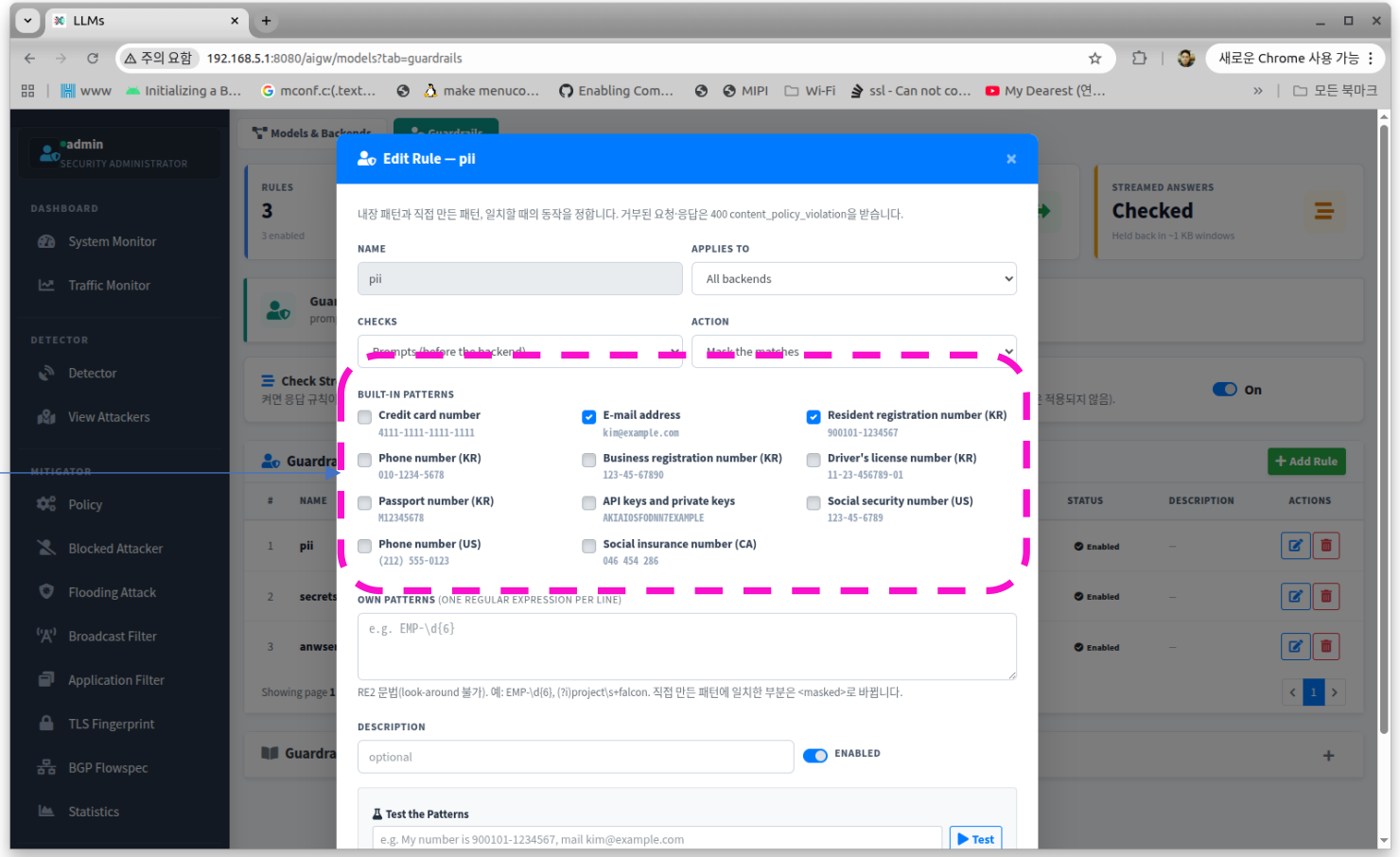
3. AI Gateway(2) – LLM Servers(5-1)



3. AI Gateway(2) – LLM Servers(5-2)

AI Gateway -> LLMs -> Guardrails Page

Guardrail settings



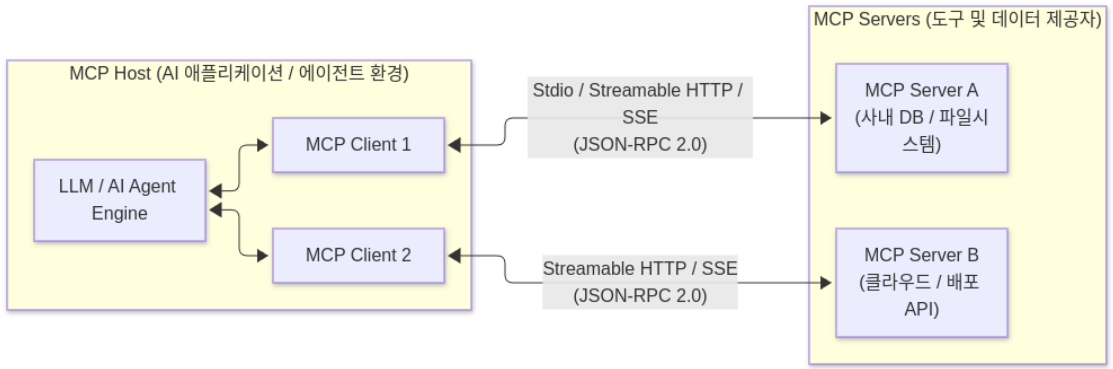
3. AI Gateway(3) – MCP Servers(1)

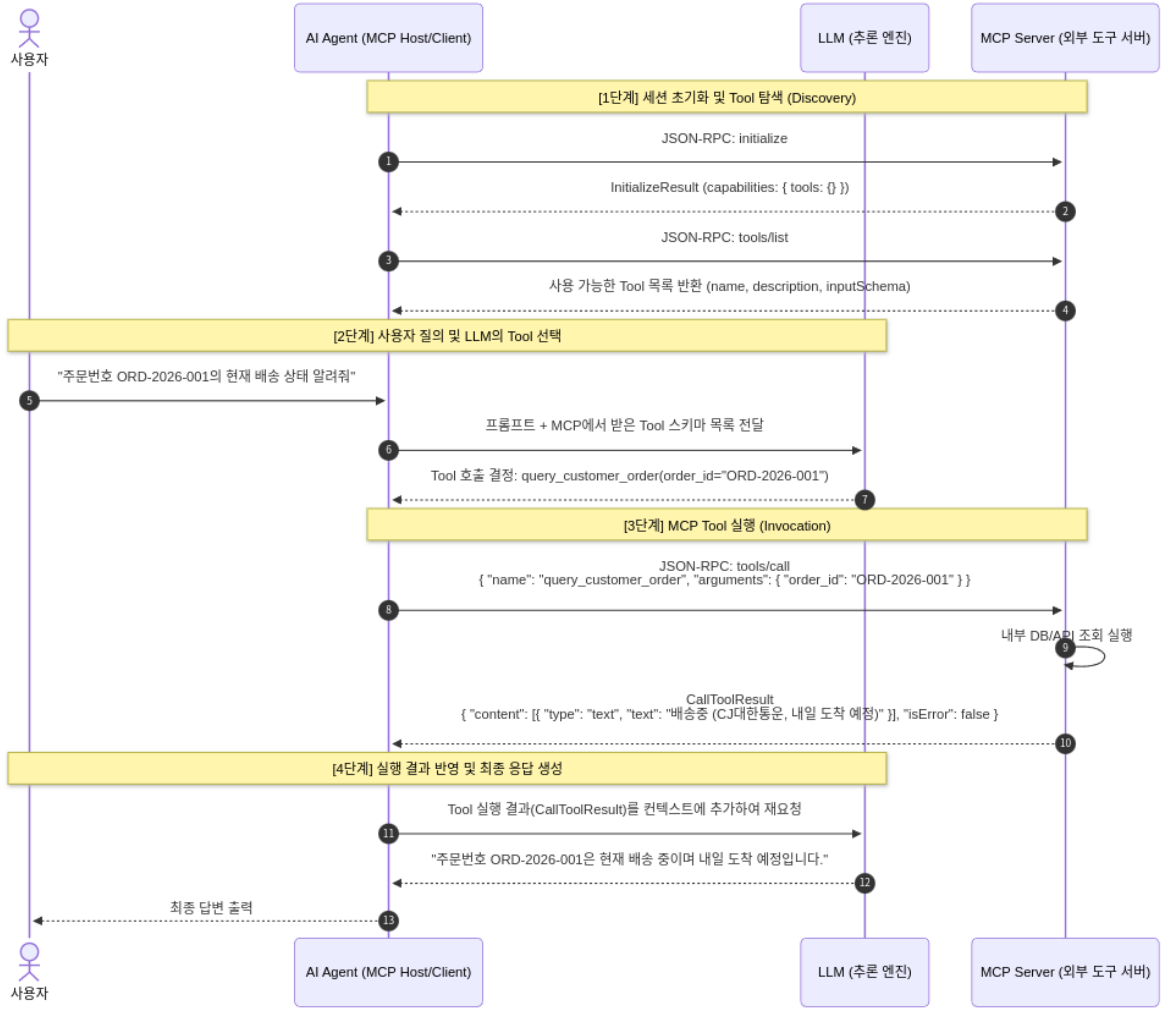




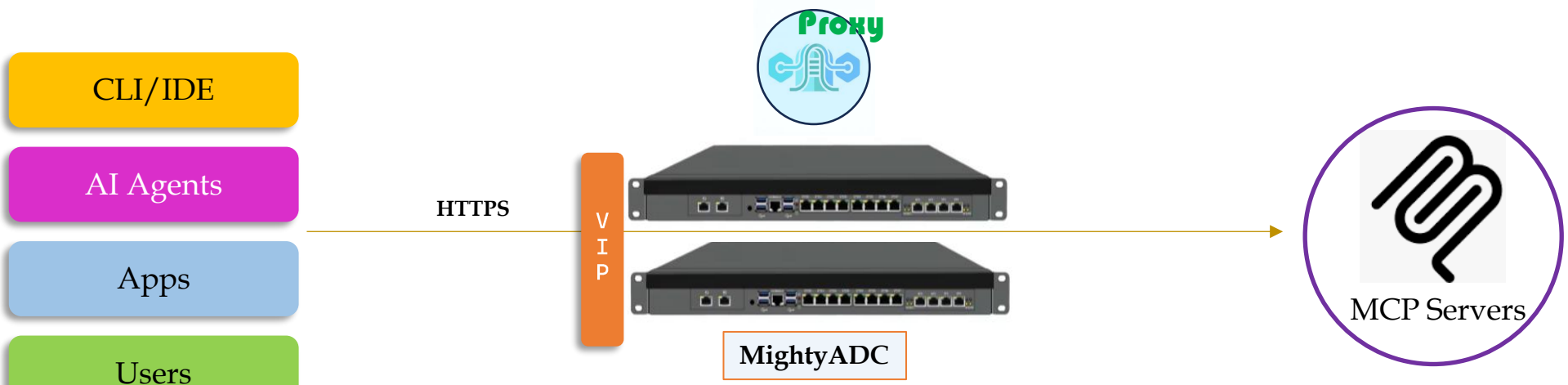
비교 항목	MCP (Model Context Protocol)	A2A (Agent-to-Agent Protocol)
연결 대상	AI 에이전트 ↔ **도구(Tool) / 리소스(Resource)**	**AI 에이전트(Client)** ↔ **AI 에이전트(Remote Agent)**
상대방의 성격	상태가 없거나 단순한 결정론적 함수/API (`inputSchema` → 결과)	자체 LLM, 메모리, 계획 능력을 가진 **자율적 추론 주체**
기능 탐색 방법	`tools/list` (함수 이름과 JSON 파라미터 스키마 목록)	**`Agent Card` (`/.well-known/agent-card.json`)** (에이전트 소개, Skills, 입출력 모달리티, 엔드포인트)
상호작용 방식	단발성 함수 호출 (`tools/call`) 및 결과 반환	**장기 실행 작업(`Task`)**, 상태 전이(`working` , `input-required` , `completed`), 멀티턴 대화(`contextId`)
비유	작업자가 **계산기, 망치, DB 쿼리 도구**를 직접 사용하는 것	프로젝트 매니저가 **전문 외주 업체나 타 부서 담당자**에게 업무를 의뢰하고 중간 보고를 받는 것

3. AI Gateway(3) – MCP Servers(2)





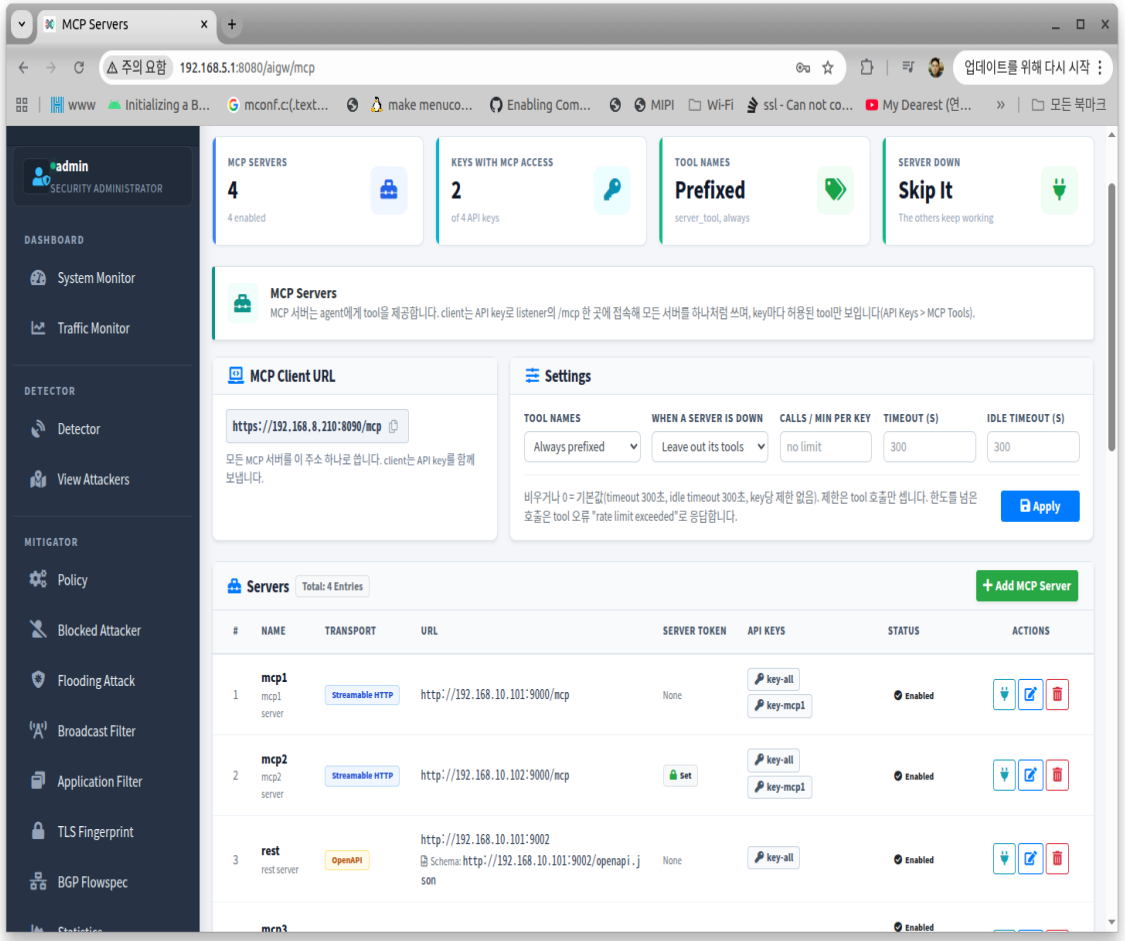
3. AI Gateway(3) – MCP Servers(3)



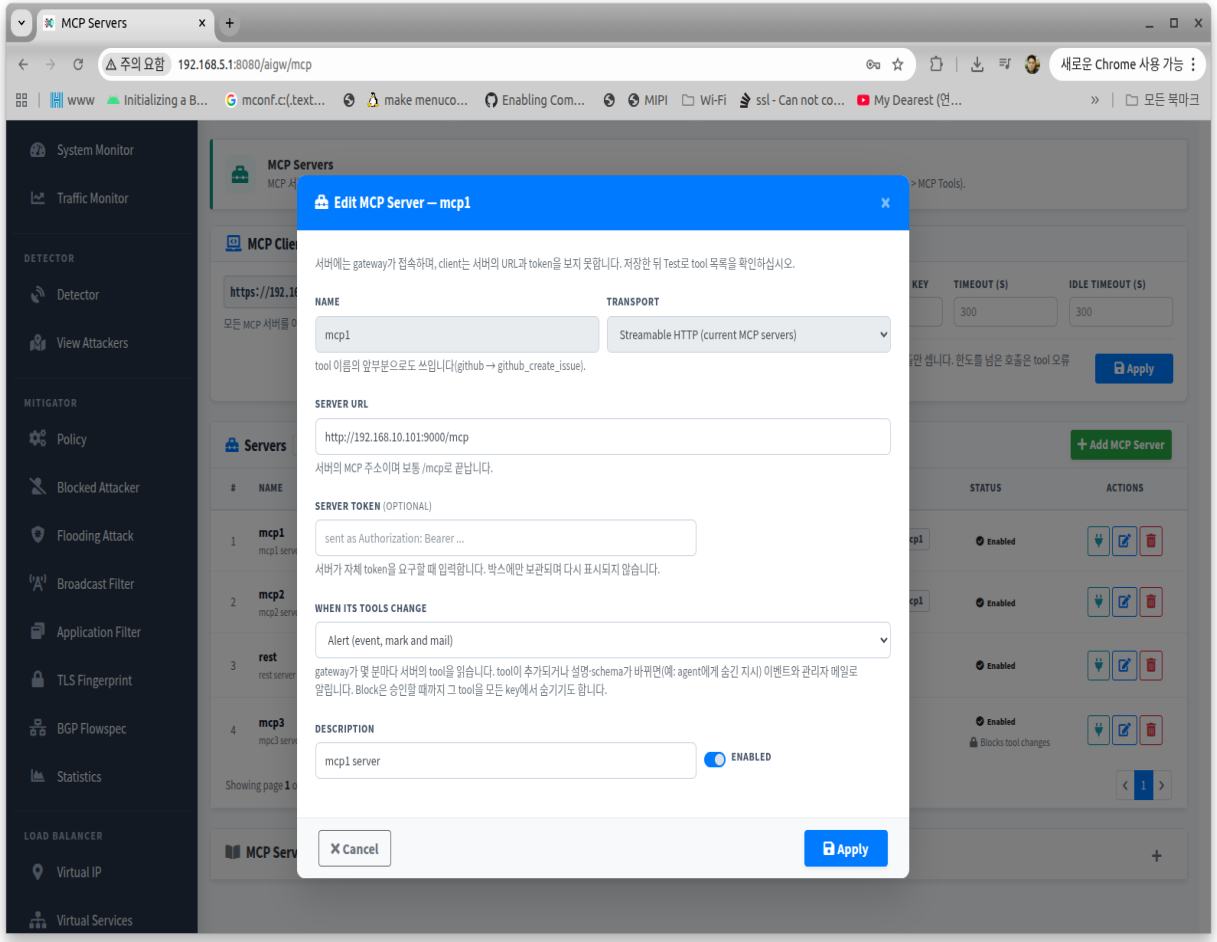
- MCP
  - ✓ API Keys/자격 증명 보호
  - ✓ Transport(Streamable HTTP, SSE, OpenAPI)
  - ✓ API Key별 도구(Tool) 권한
  - ✓ API Key별 사용량 제한, Timeout 기능
  - ✓ MCP tool 변경 감지 기능(tool poisoning 차단)
  - ✓ 도구 호출/오류/세션에 대한 실시간 Monitoring 및 통계 기능
  - ✓ Logging

3. AI Gateway(3) – MCP Servers(4)

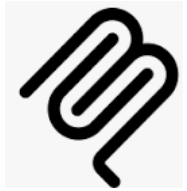
AI Gateway -> MCP Servers Page(1)



AI Gateway -> MCP Servers Page(2)



3. AI Gateway(4) – A2A Agents(1)

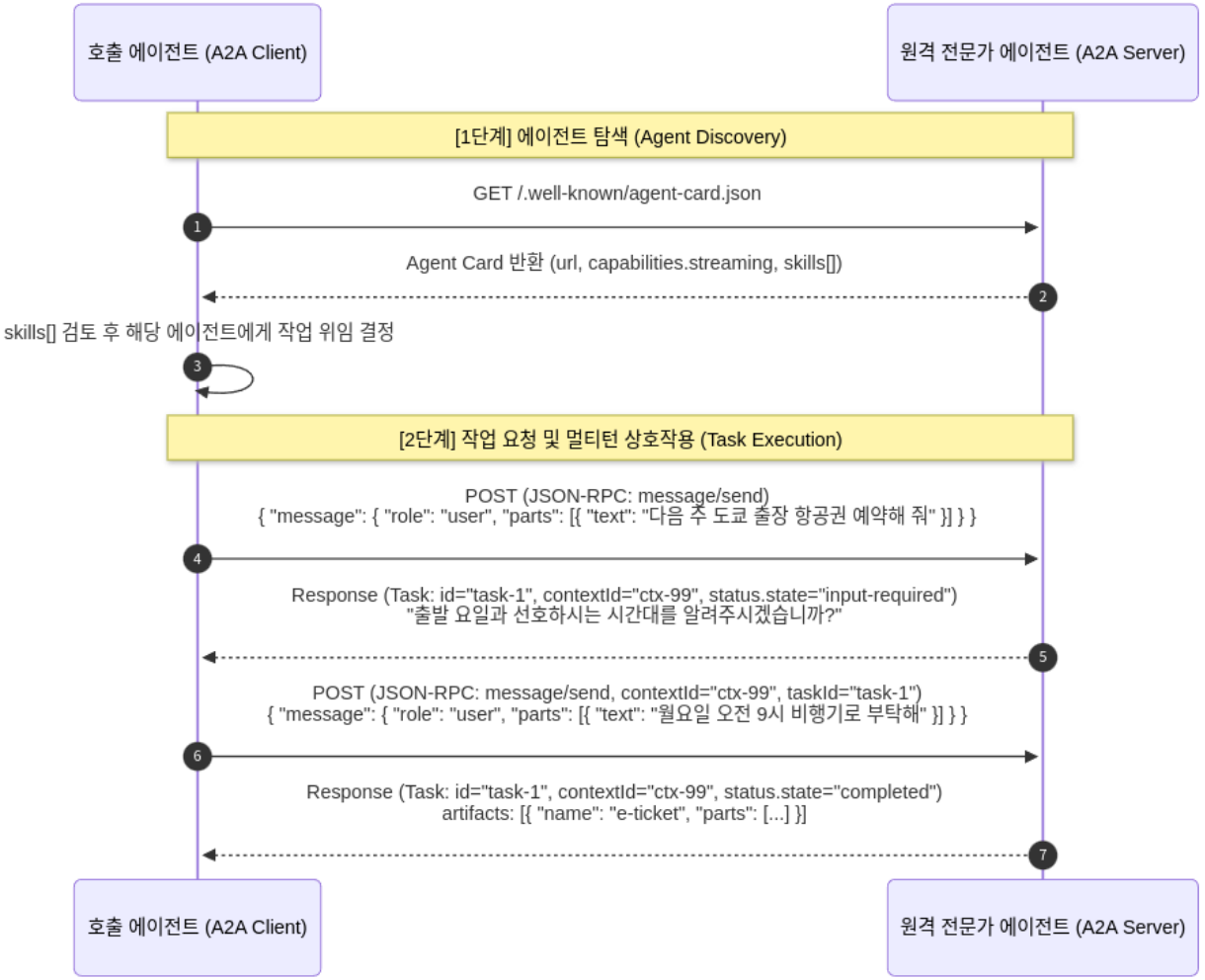




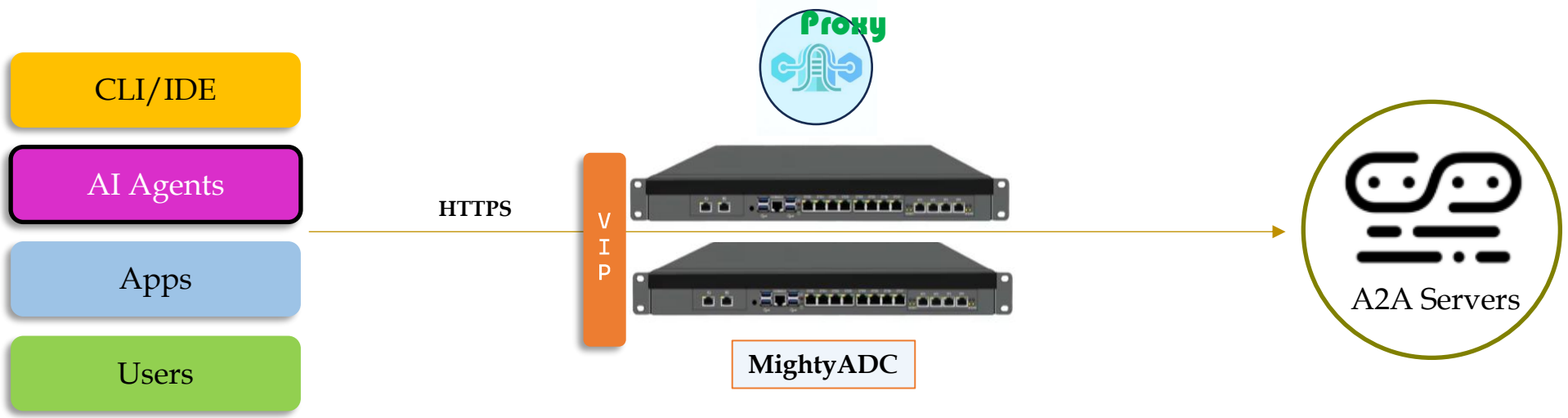
비교 항목	MCP (Model Context Protocol)	A2A (Agent-to-Agent Protocol)
연결 대상	AI 에이전트 ↔ **도구(Tool) / 리소스(Resource)**	**AI 에이전트(Client)** ↔ **AI 에이전트(Remote Agent)**
상대방의 성격	상태가 없거나 단순한 결정론적 함수/API (`inputSchema` → 결과)	자체 LLM, 메모리, 계획 능력을 가진 **자율적 추론 주체**
기능 탐색 방법	`tools/list` (함수 이름과 JSON 파라미터 스키마 목록)	**`Agent Card` (`/.well-known/agent-card.json`)** (에이전트 소개, Skills, 입출력 모달리티, 엔드포인트)
상호작용 방식	단발성 함수 호출 (`tools/call`) 및 결과 반환	**장기 실행 작업(`Task`)**, 상태 전이(`working` , `input-required` , `completed`), 멀티턴 대화(`contextId`)
비유	작업자가 **계산기, 망치, DB 쿼리 도구**를 직접 사용하는 것	프로젝트 매니저가 **전문 외주 업체나 타 부서 담당자**에게 업무를 의뢰하고 중간 보고를 받는 것

3. AI Gateway(4) – A2A Agents(2)





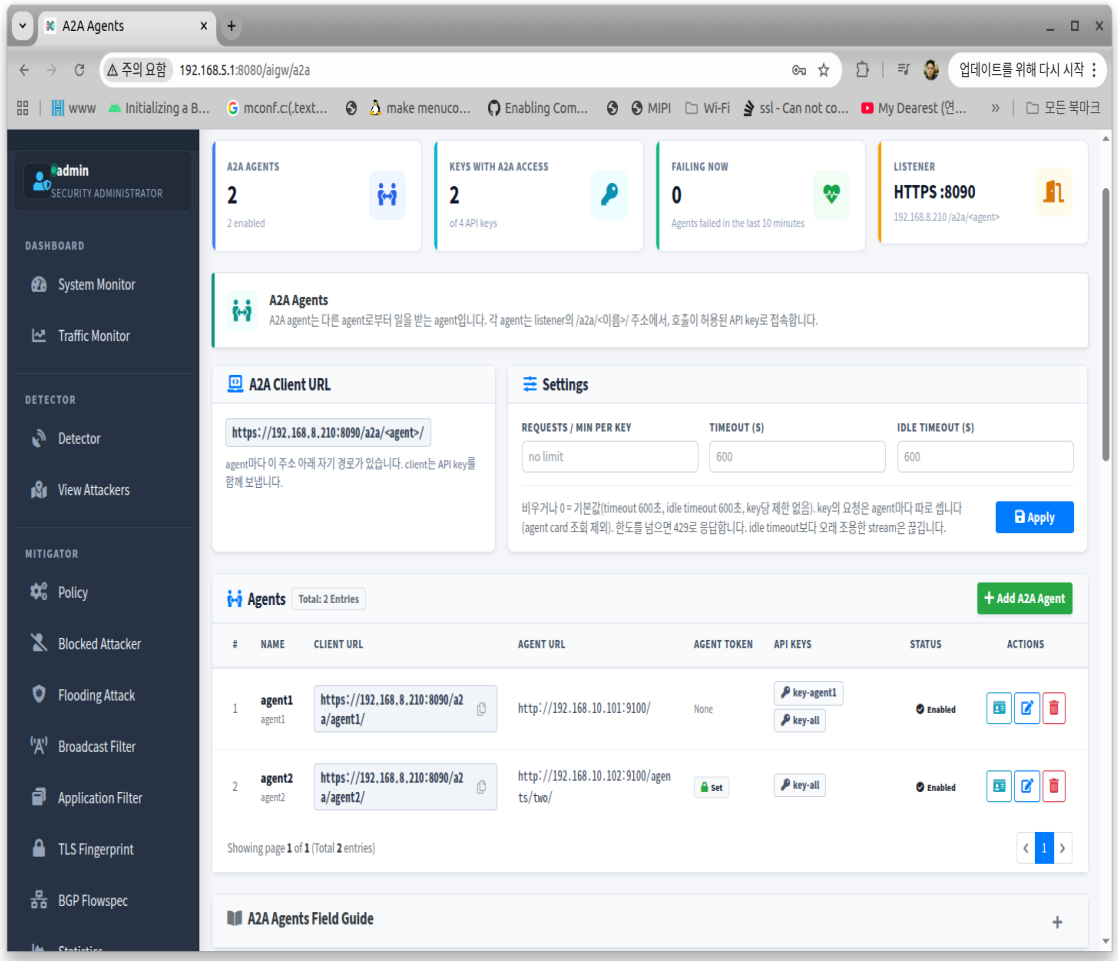
3. AI Gateway(4) – A2A Agents(3)



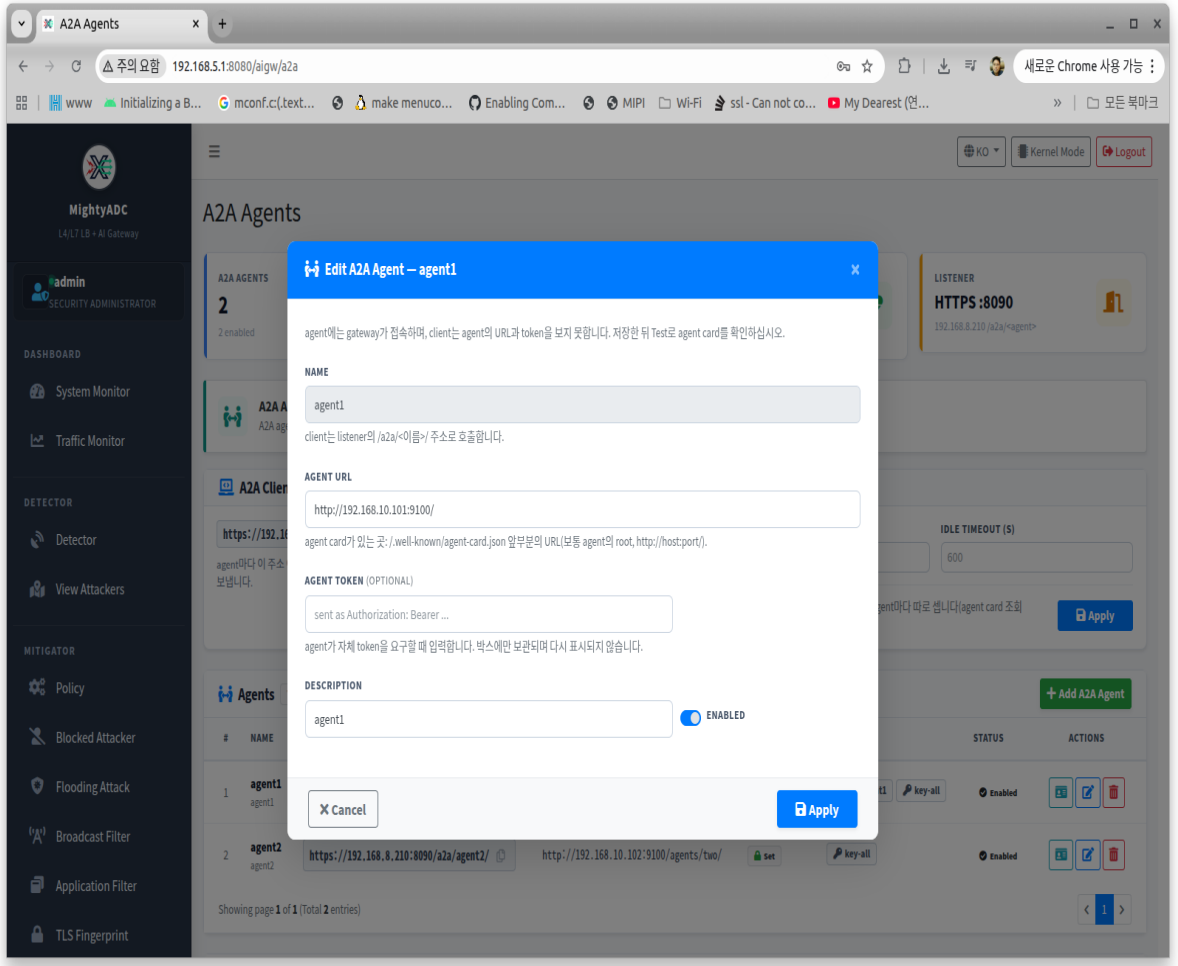
- A2A Agents
  - ✓ API Keys/자격 증명 보호
  - ✓ API Key별 허용 agent
  - ✓ API Key별 사용량 제한, Timeout 기능
  - ✓ 메시지/스트림/Task 호출/오류에 대한 실시간 Monitoring 및 통계 기능
  - ✓ Logging

3. AI Gateway(4) – A2A Agents(4)

AI Gateway -> A2A Agents Page(1)

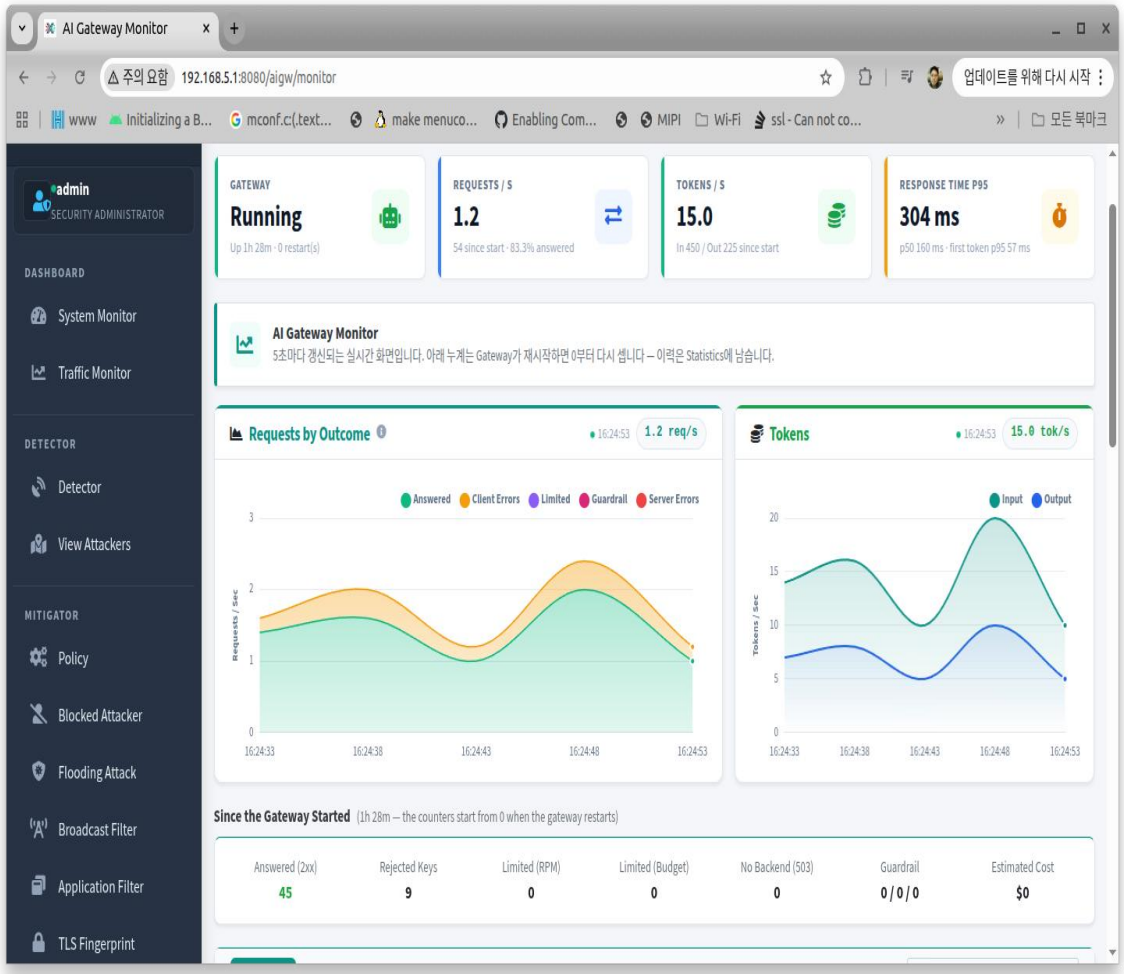


AI Gateway -> A2A Agents Page(2)

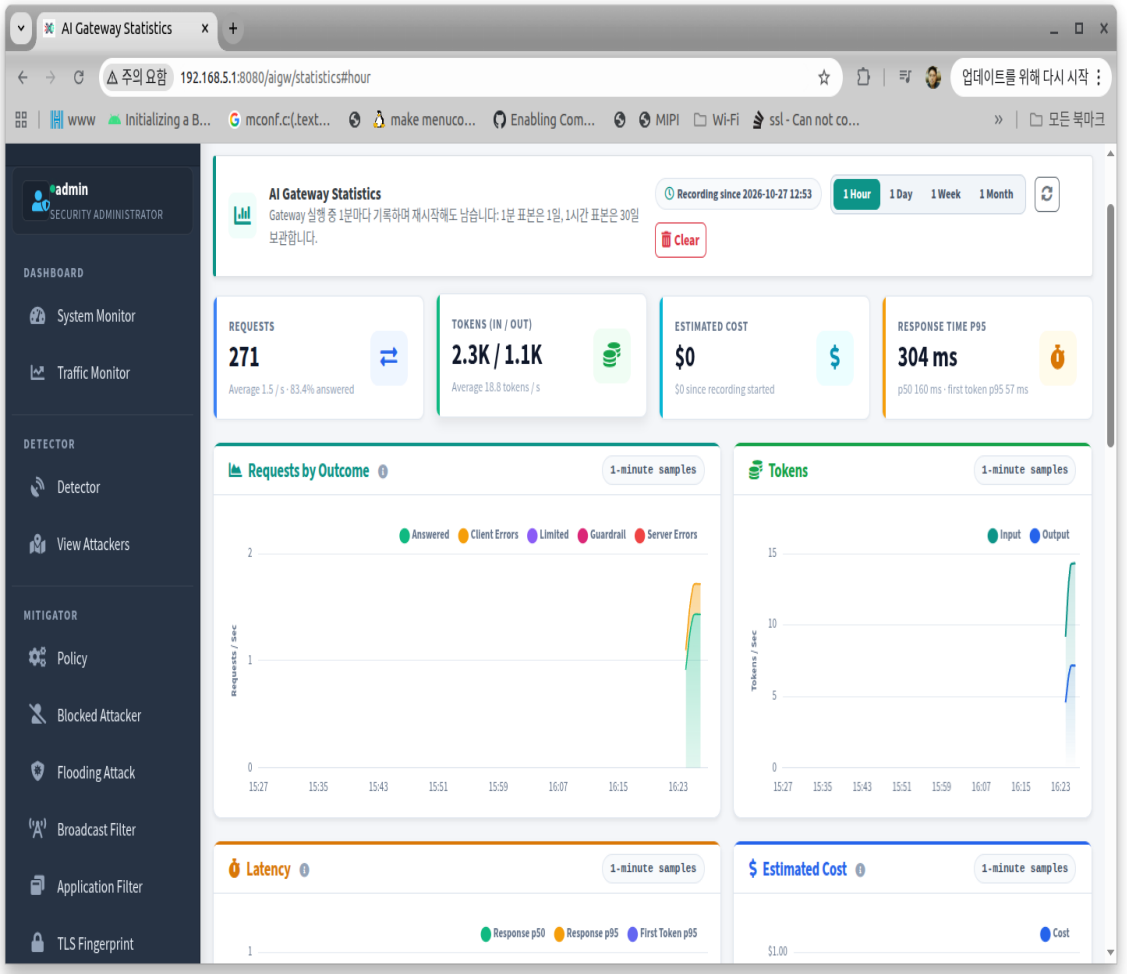


3. AI Gateway(5) – Monitor/Statistics

AI Gateway -> Monitor Page

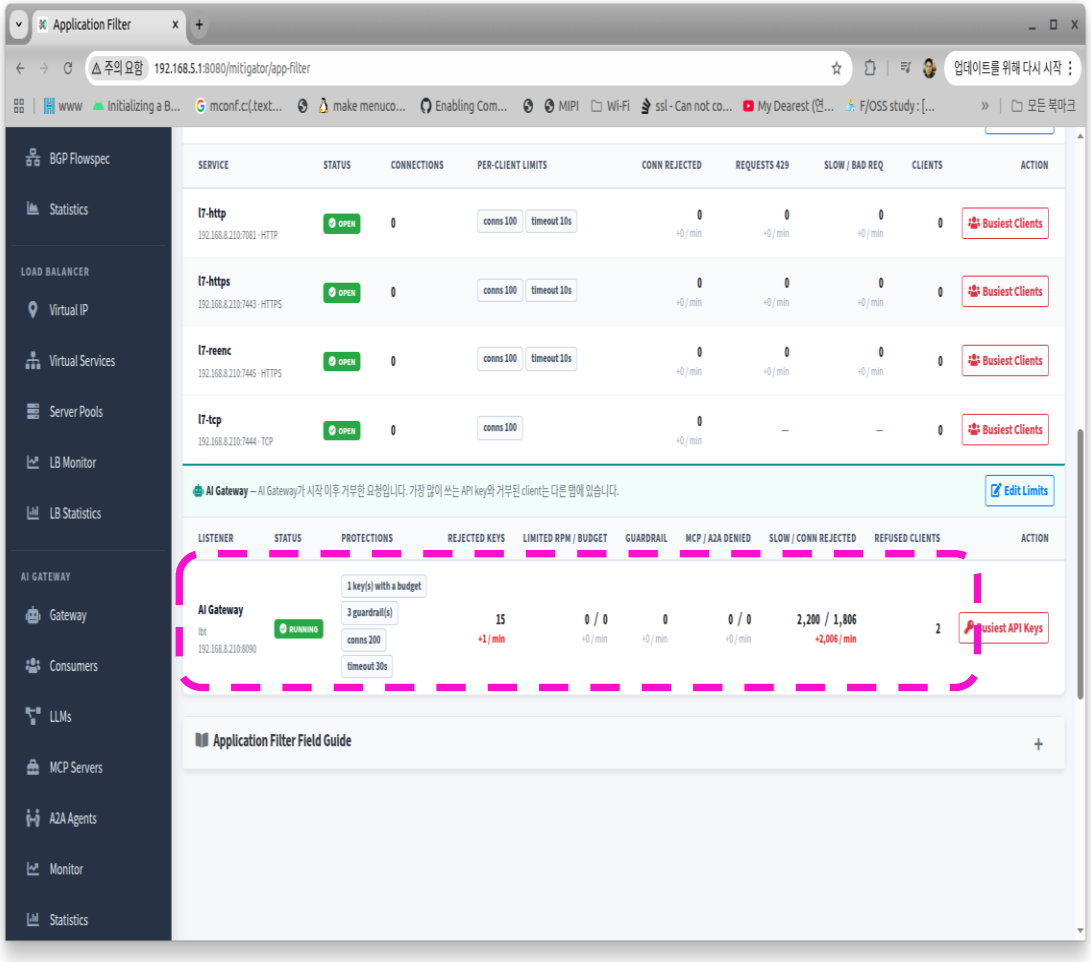


AI Gateway -> Statistics Page

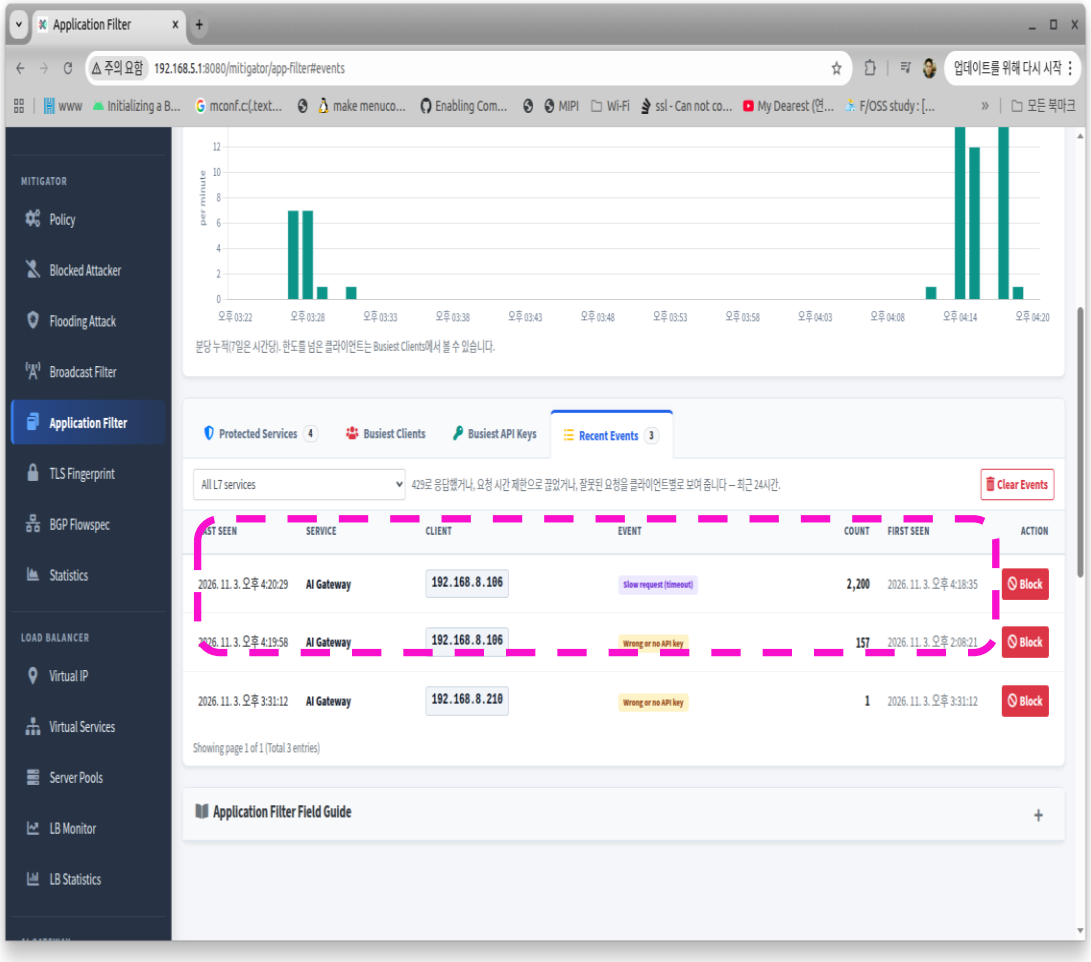


3. AI Gateway(6) – Slowloris Attack Protection

Mitigator -> Application Filter -> AI Gatway Page



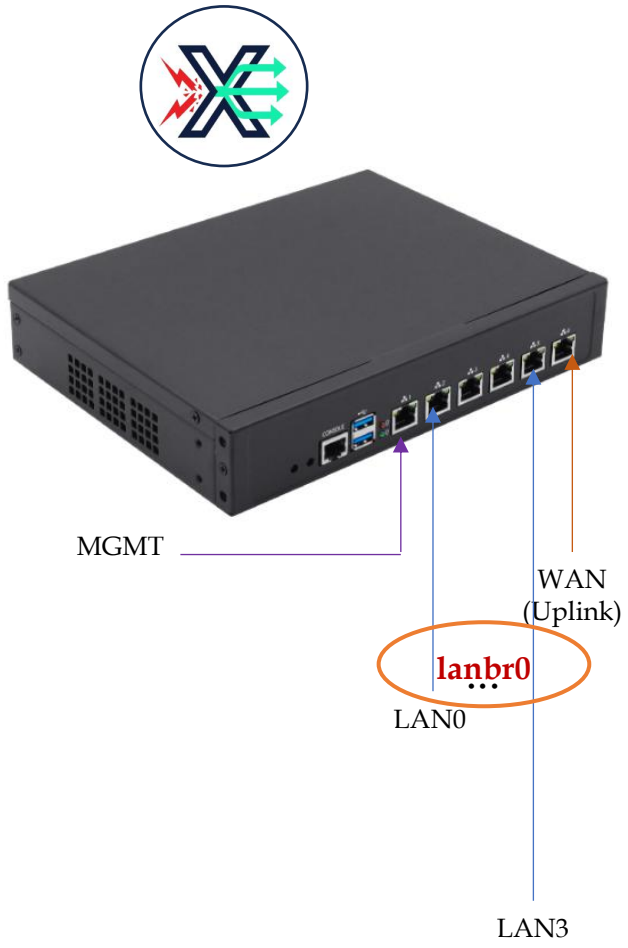
Mitigator -> Application Filter -> Recent Events Page

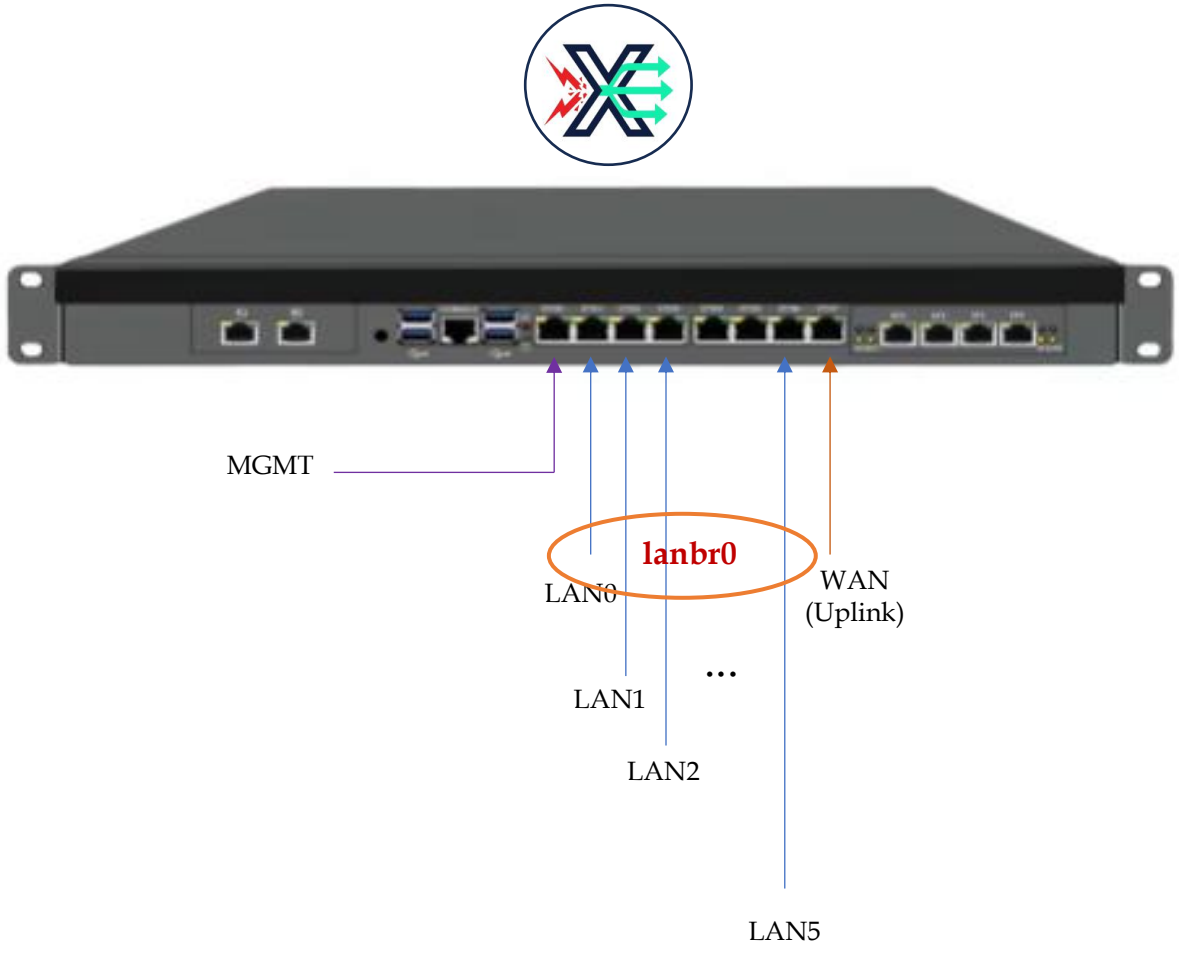


4. MightyADC – XDDoS + L4/L7 LB + AI Gateway



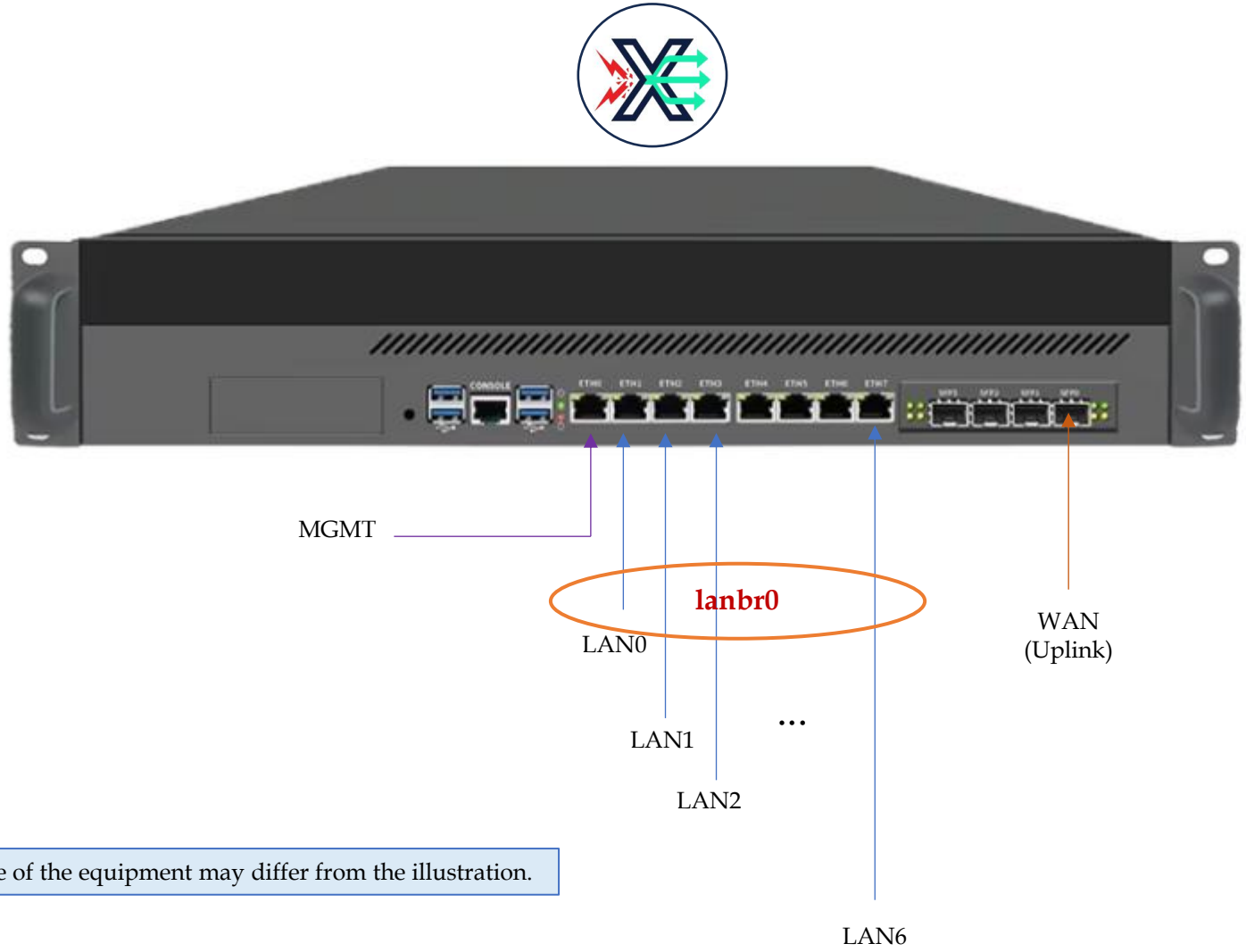
4. MightyADC(1) – H/W Box(1)





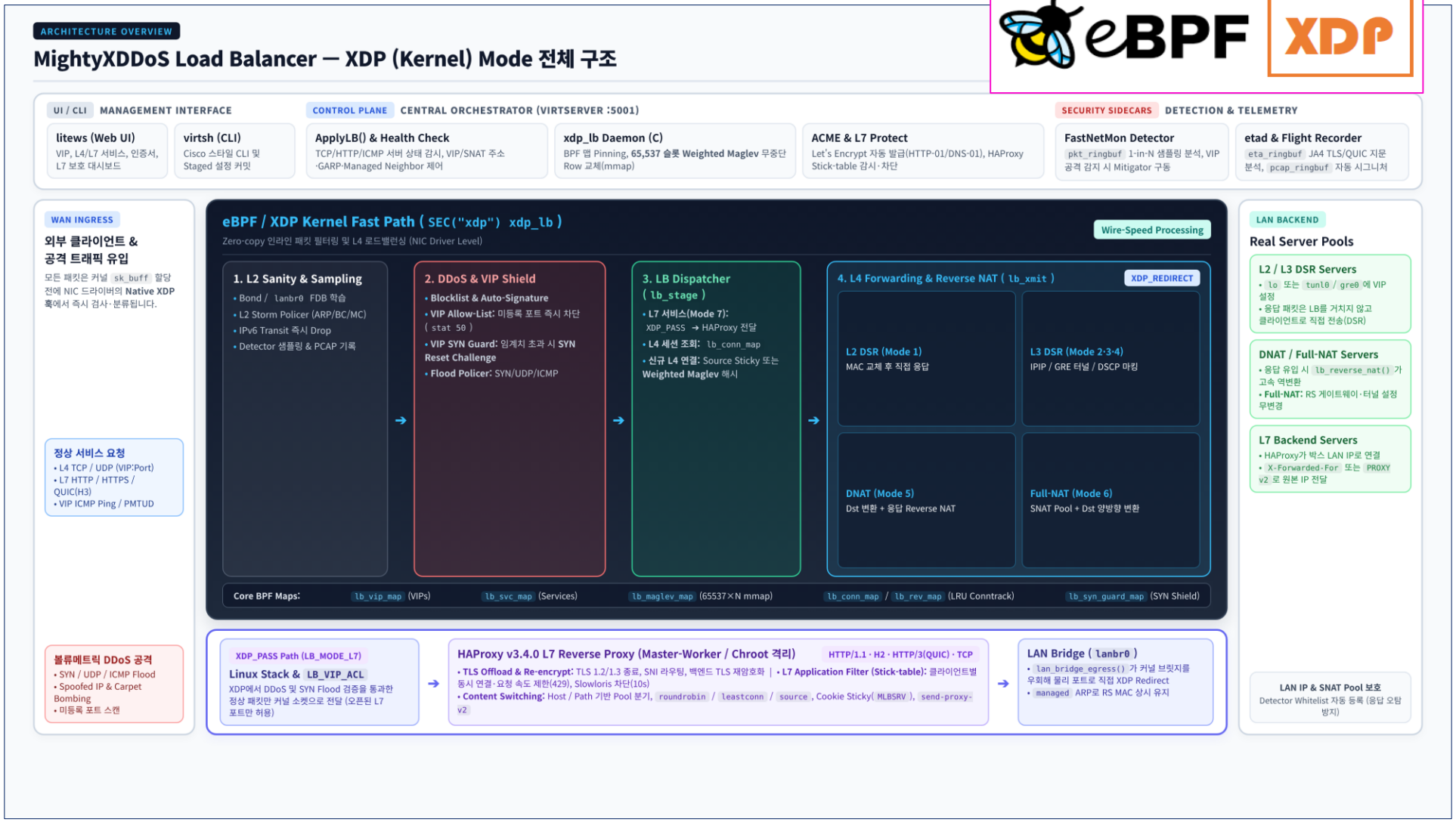
Note: The appearance of the equipment may differ from the illustration.

4. MightyADC(1) – H/W Box(2)

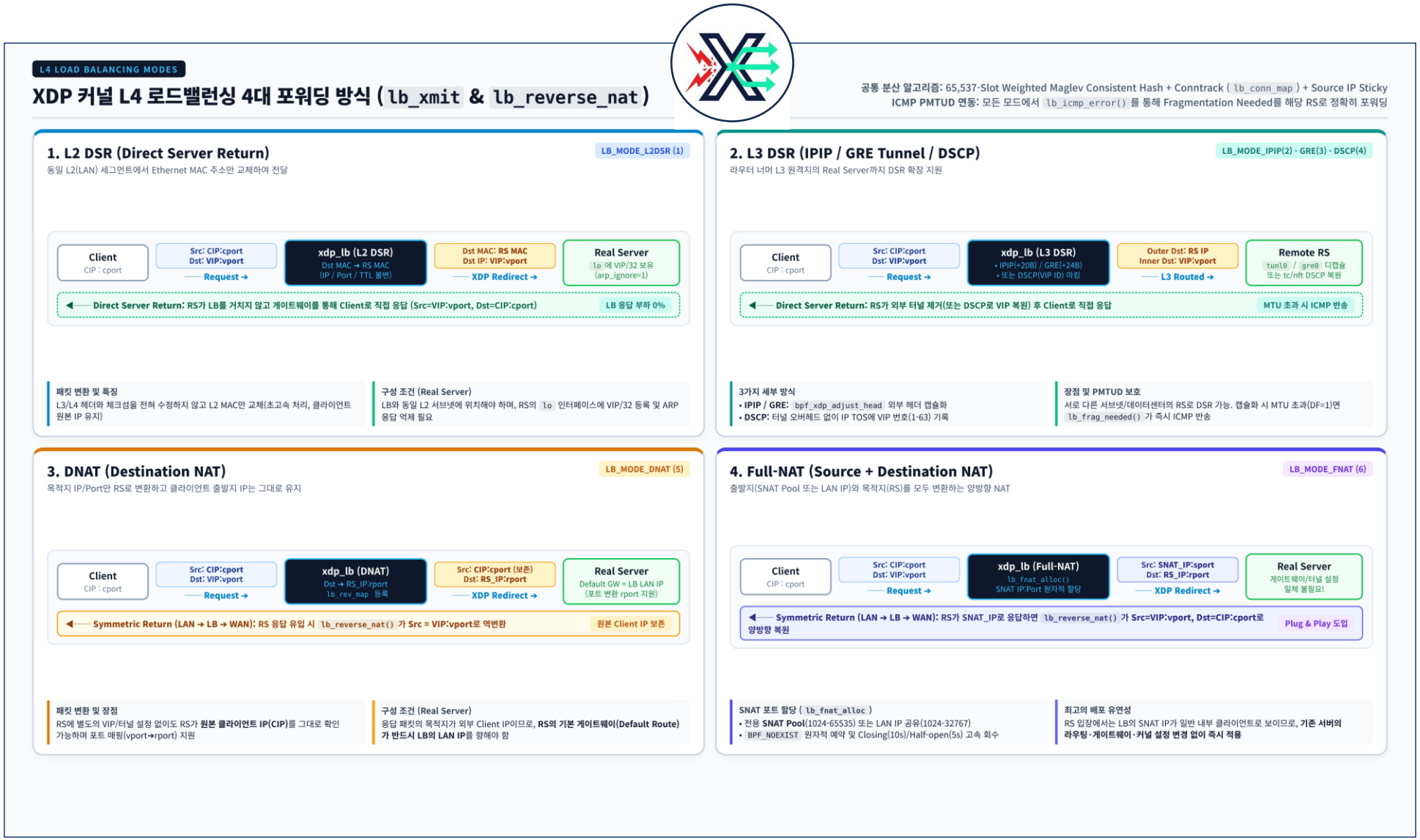


Note: The appearance of the equipment may differ from the illustration.

4. MightyADC(2) – Architecture(1)



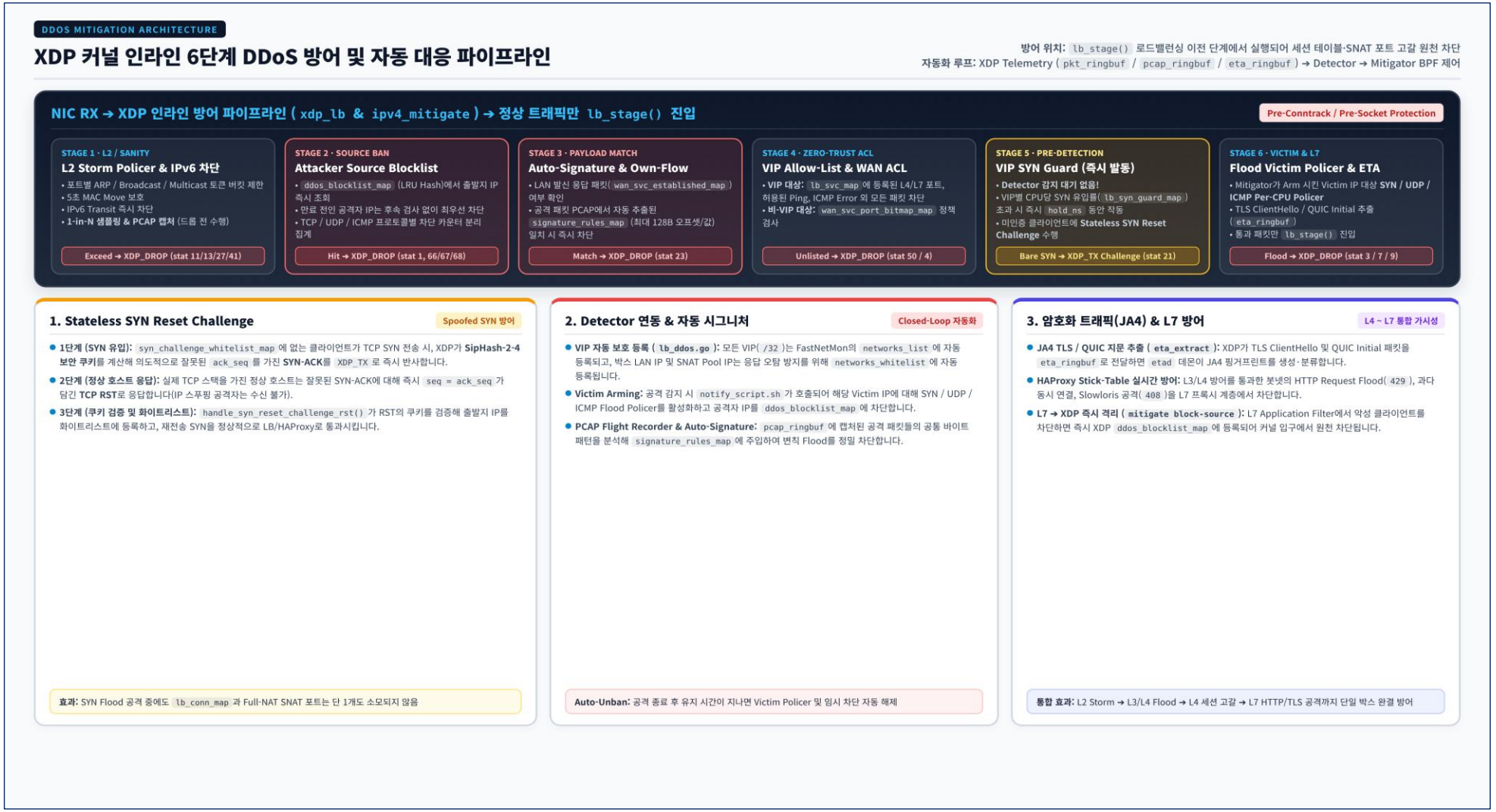
4. MightyADC(2) – Architecture(2)



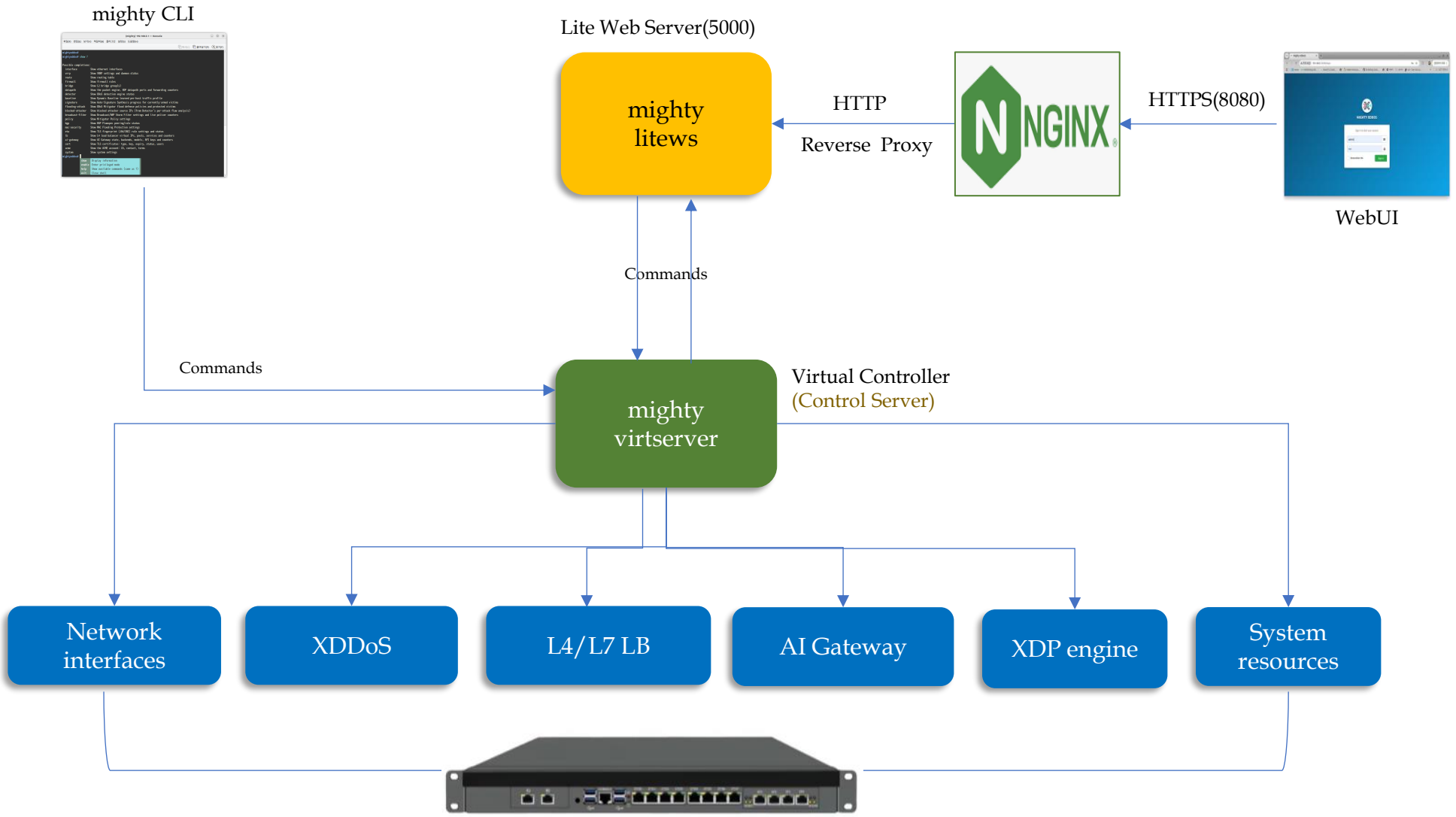
4. MightyADC(2) – Architecture(3)



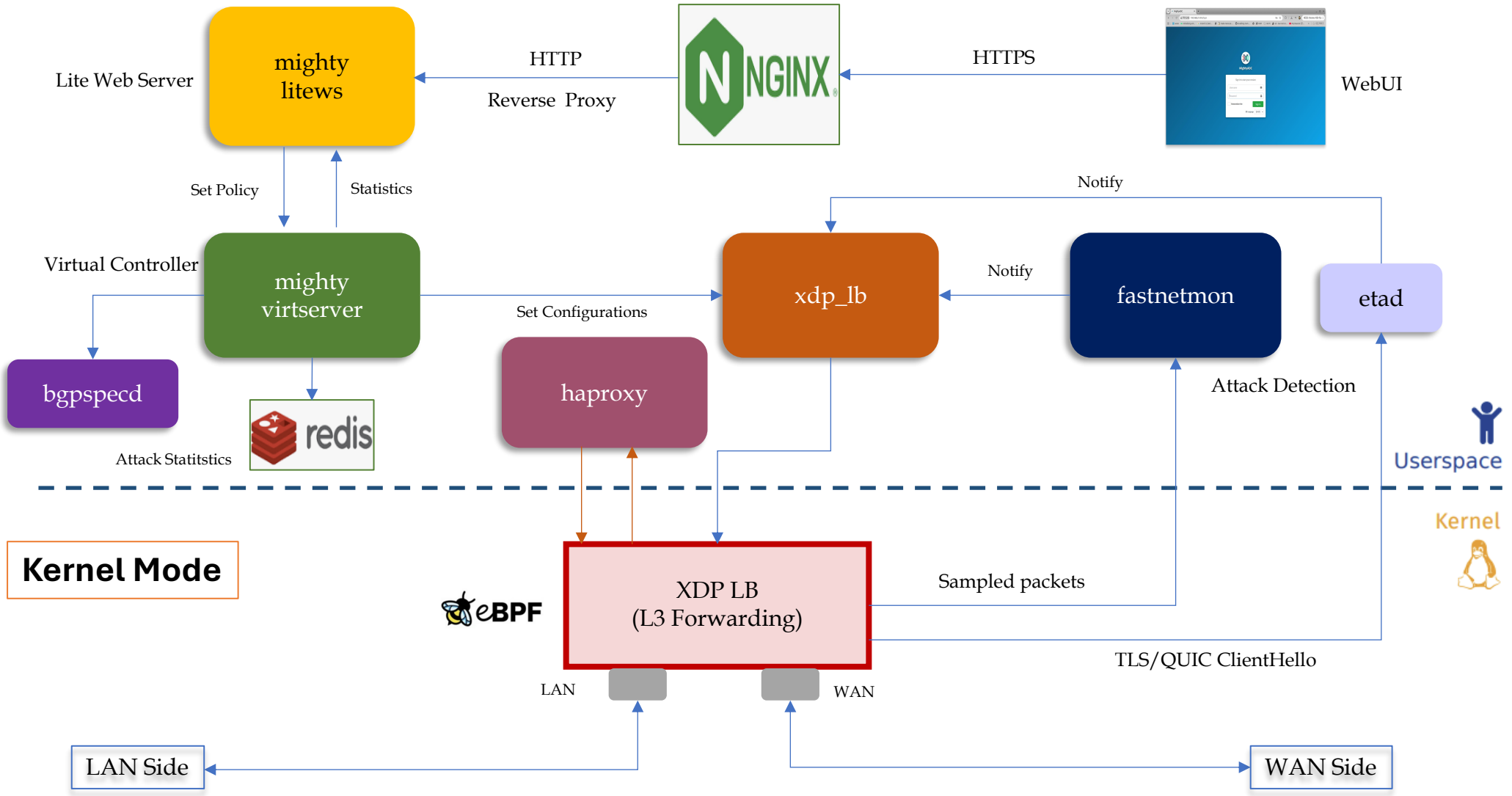
4. MightyADC(2) – Architecture(4)



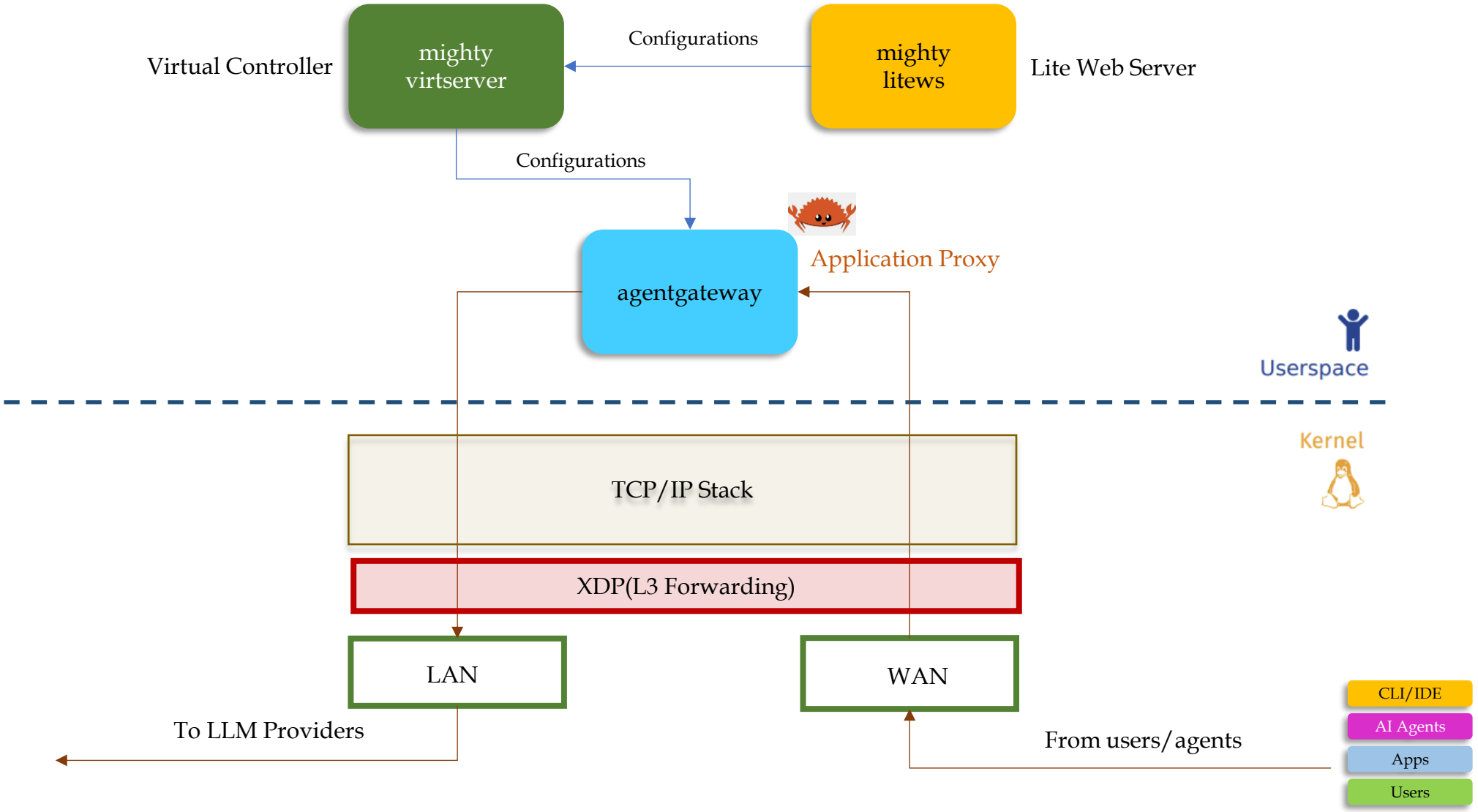
4. MightyADC(3) – Mighty Engine(1)



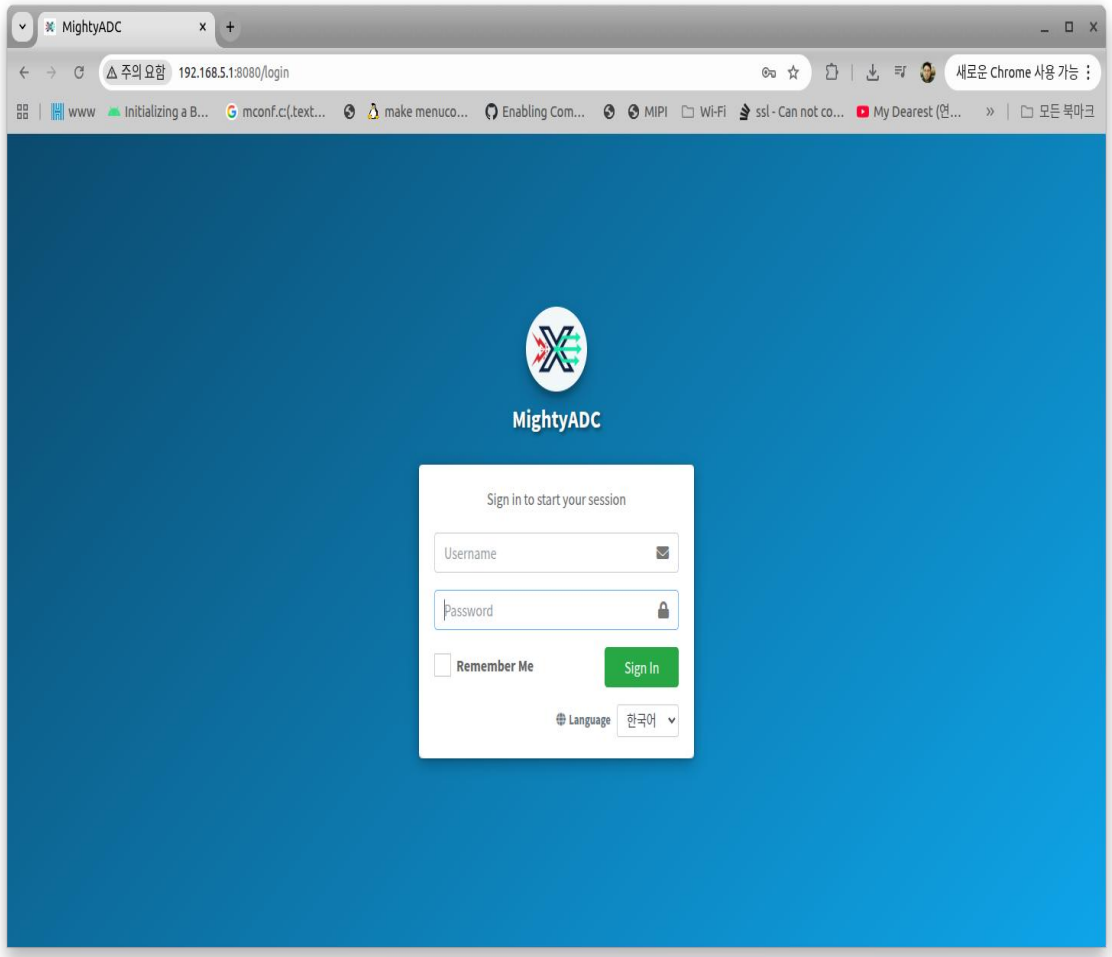
4. MightyADC(3) – Mighty Engine(2)

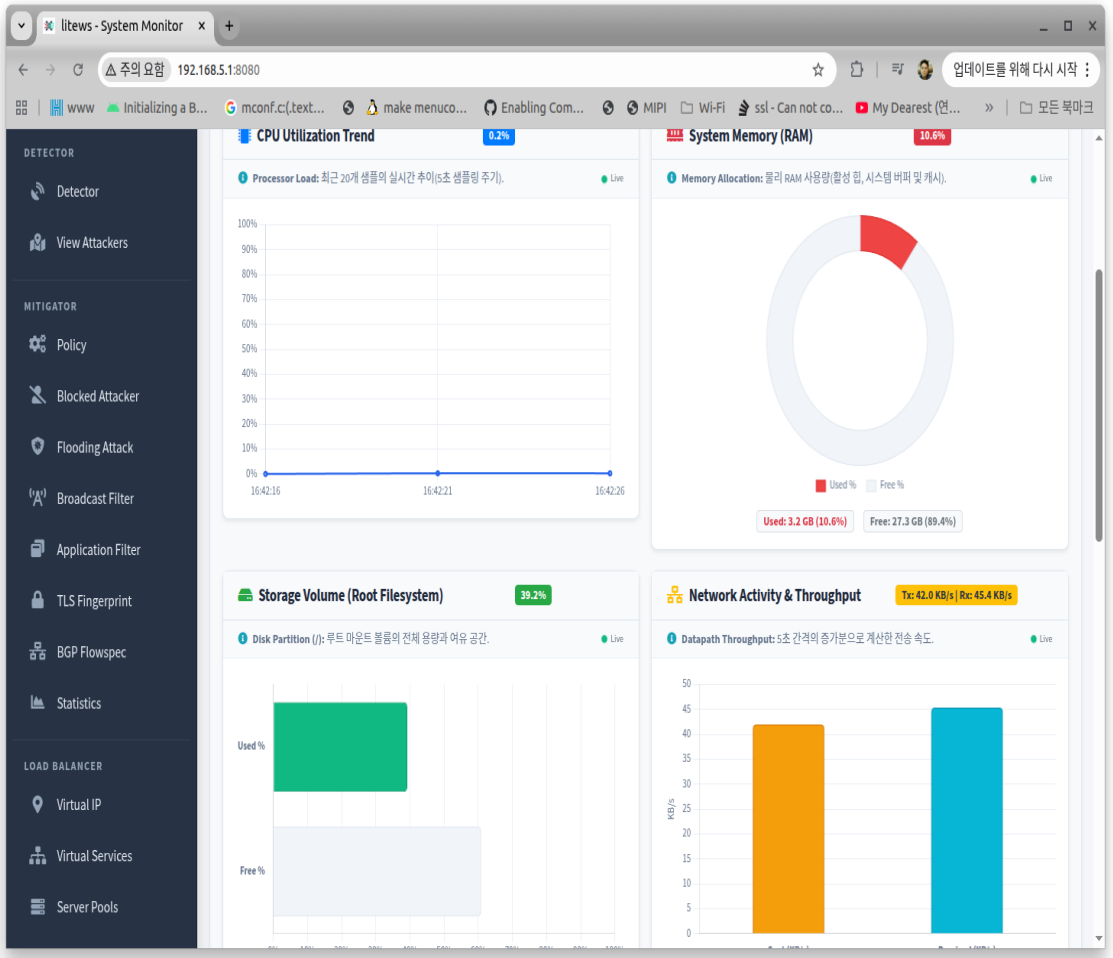


4. MightyADC(3) – Mighty Engine(3)

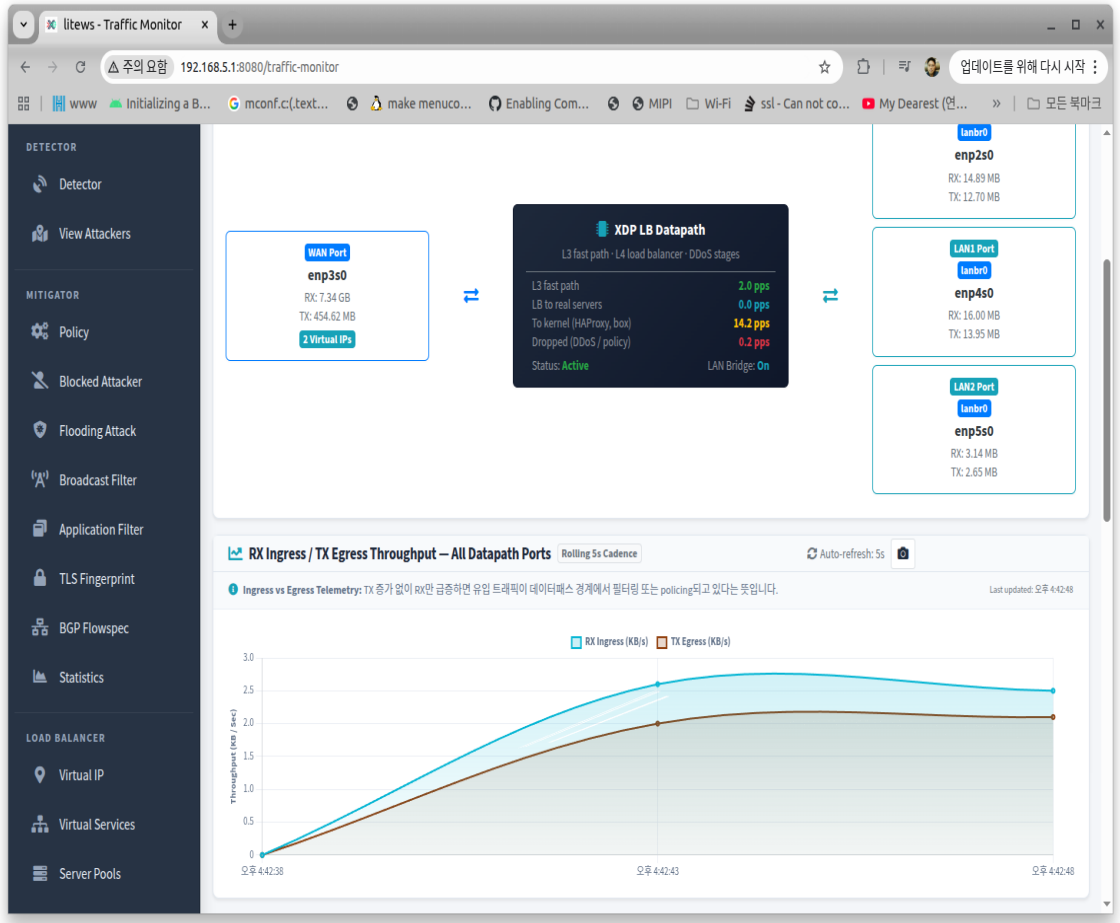


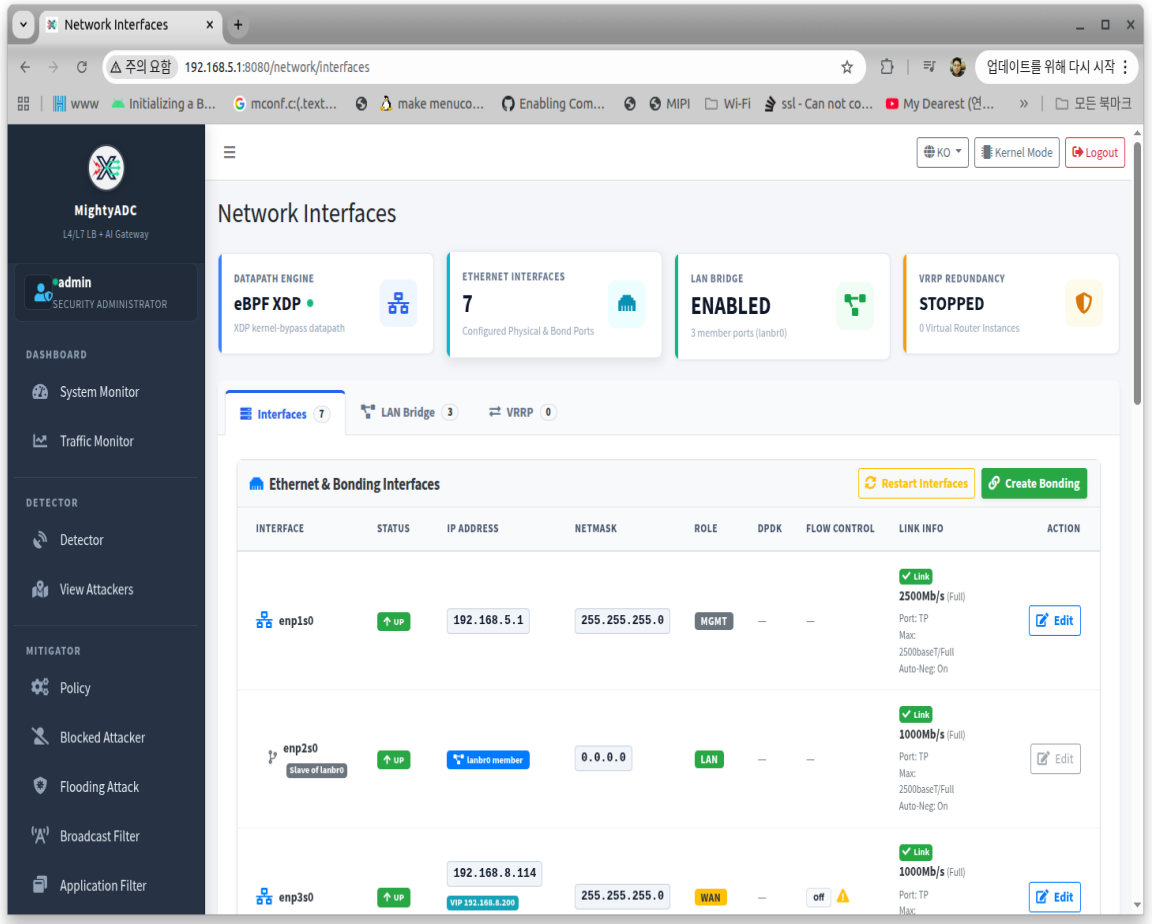
4. MightyADC(4) – WebUI(1)



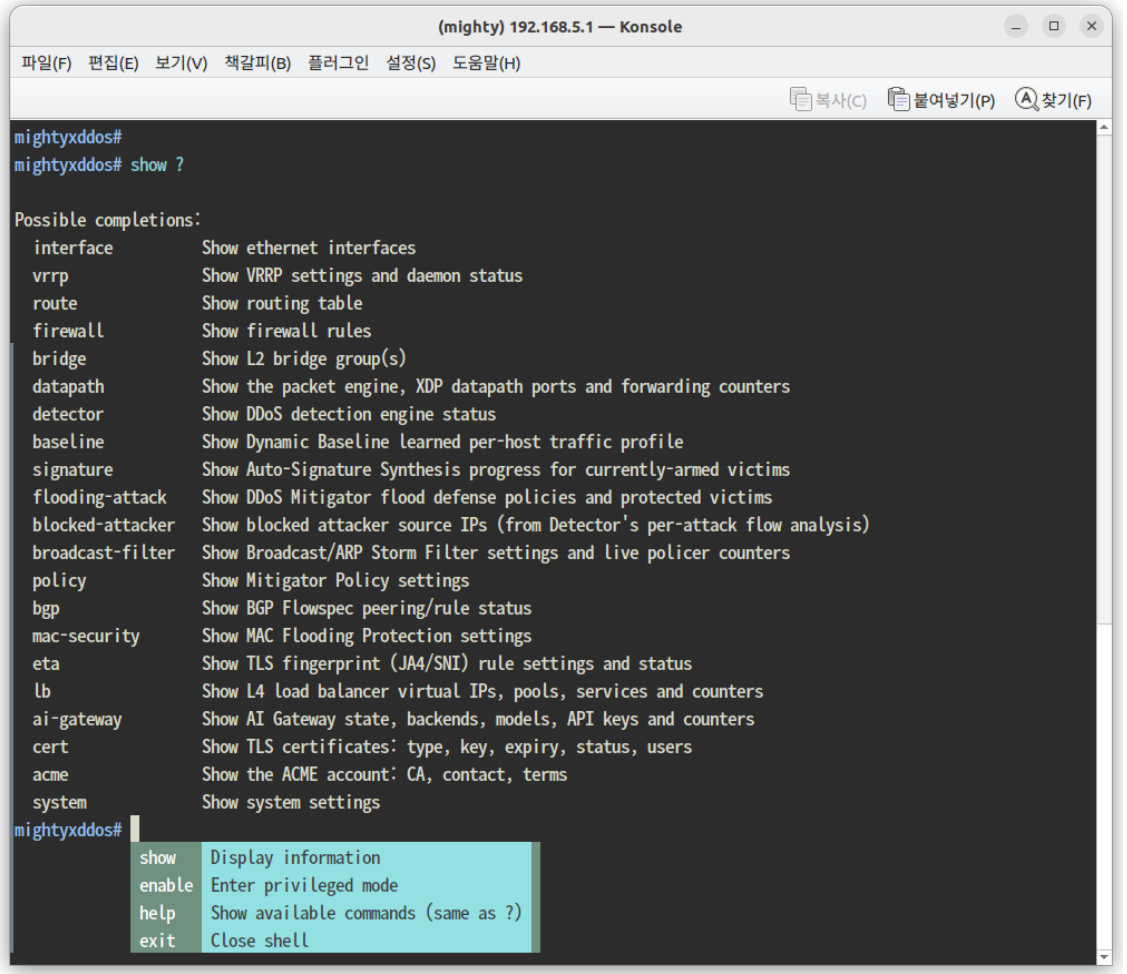


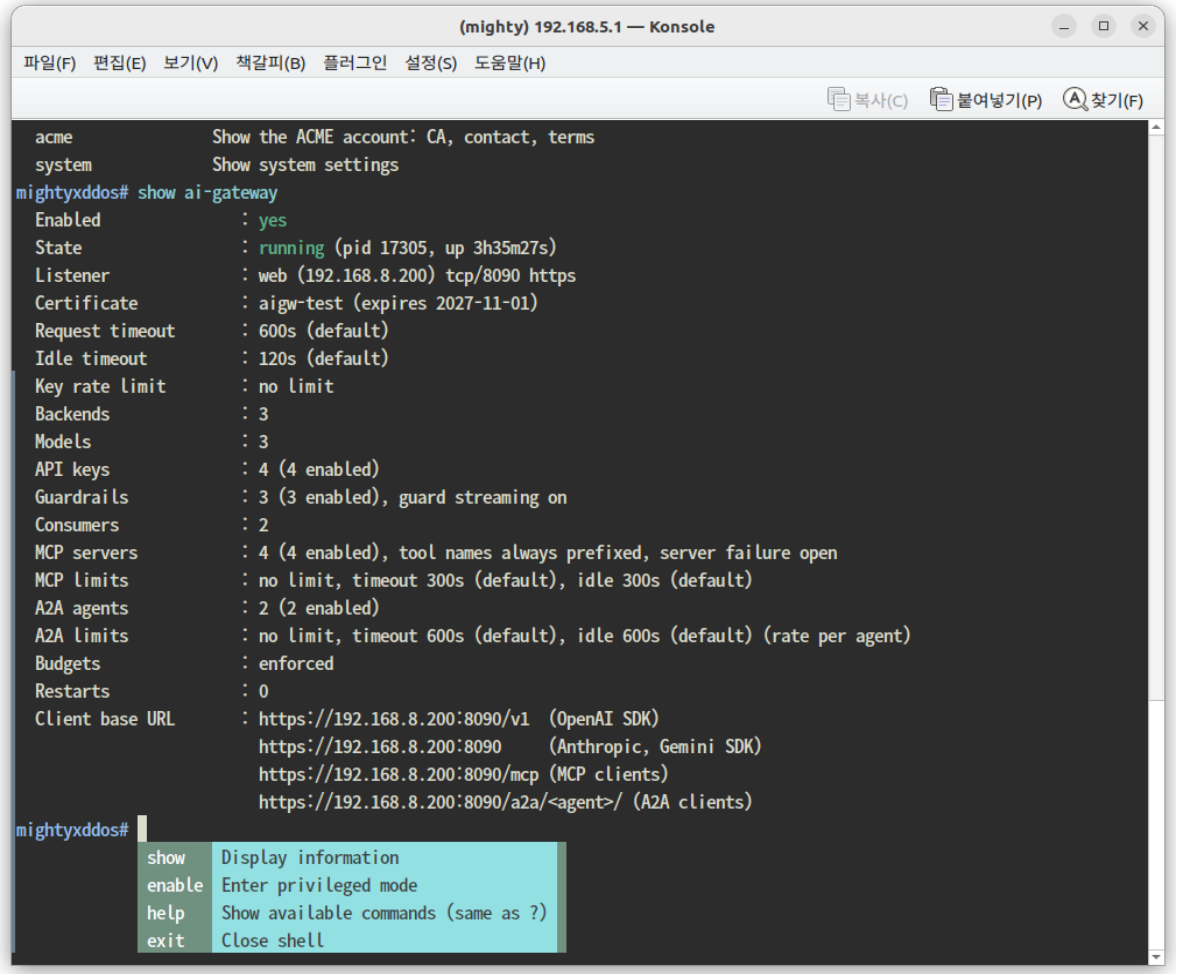
4. MightyADC(4) – WebUI(2)



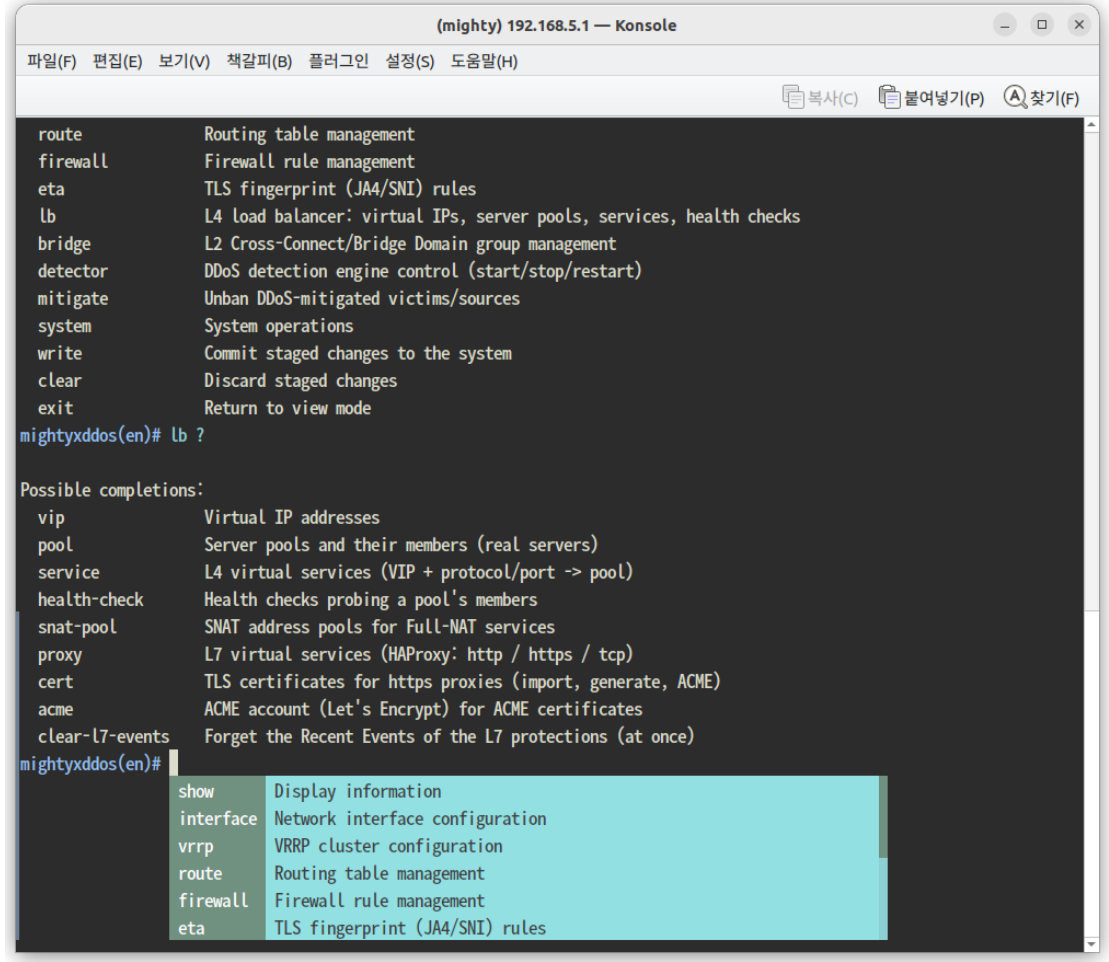


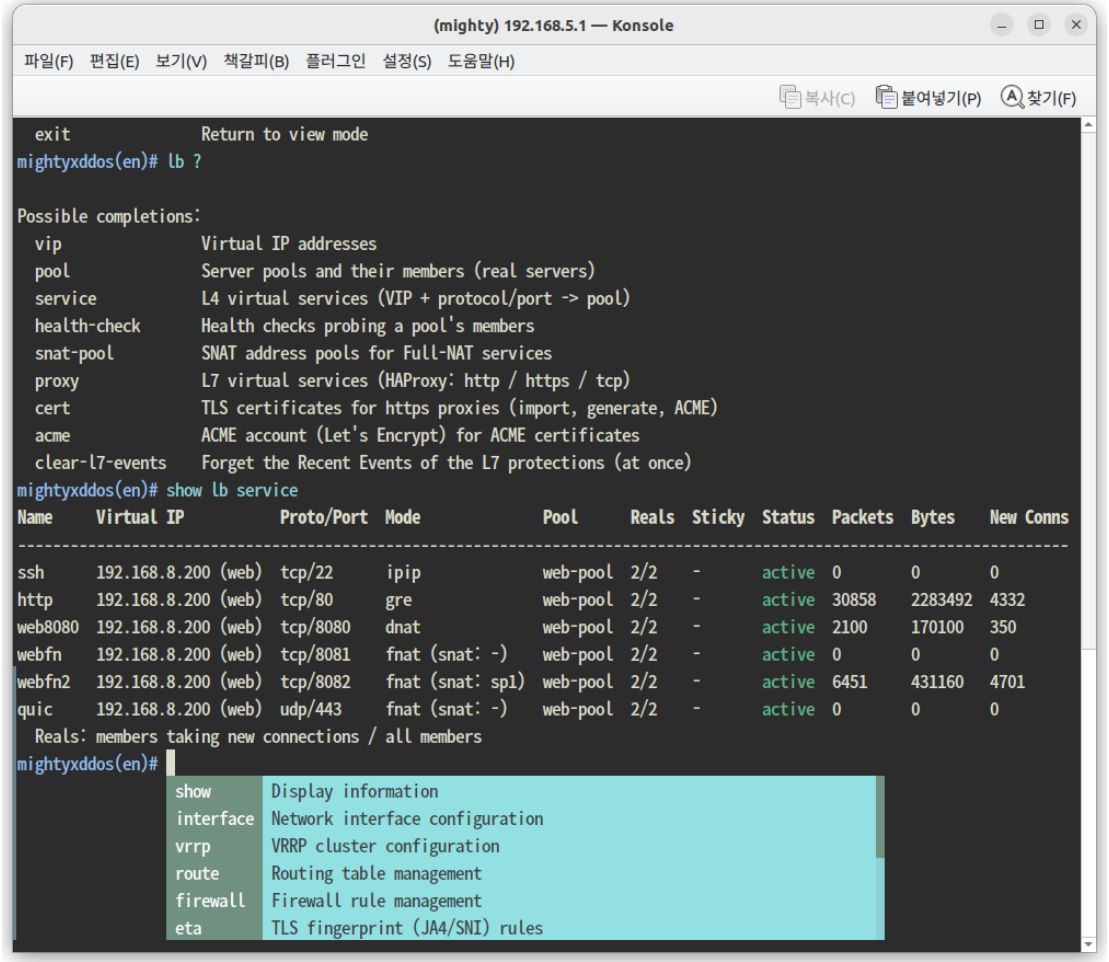
4. MightyADC(5) – CLI(1)



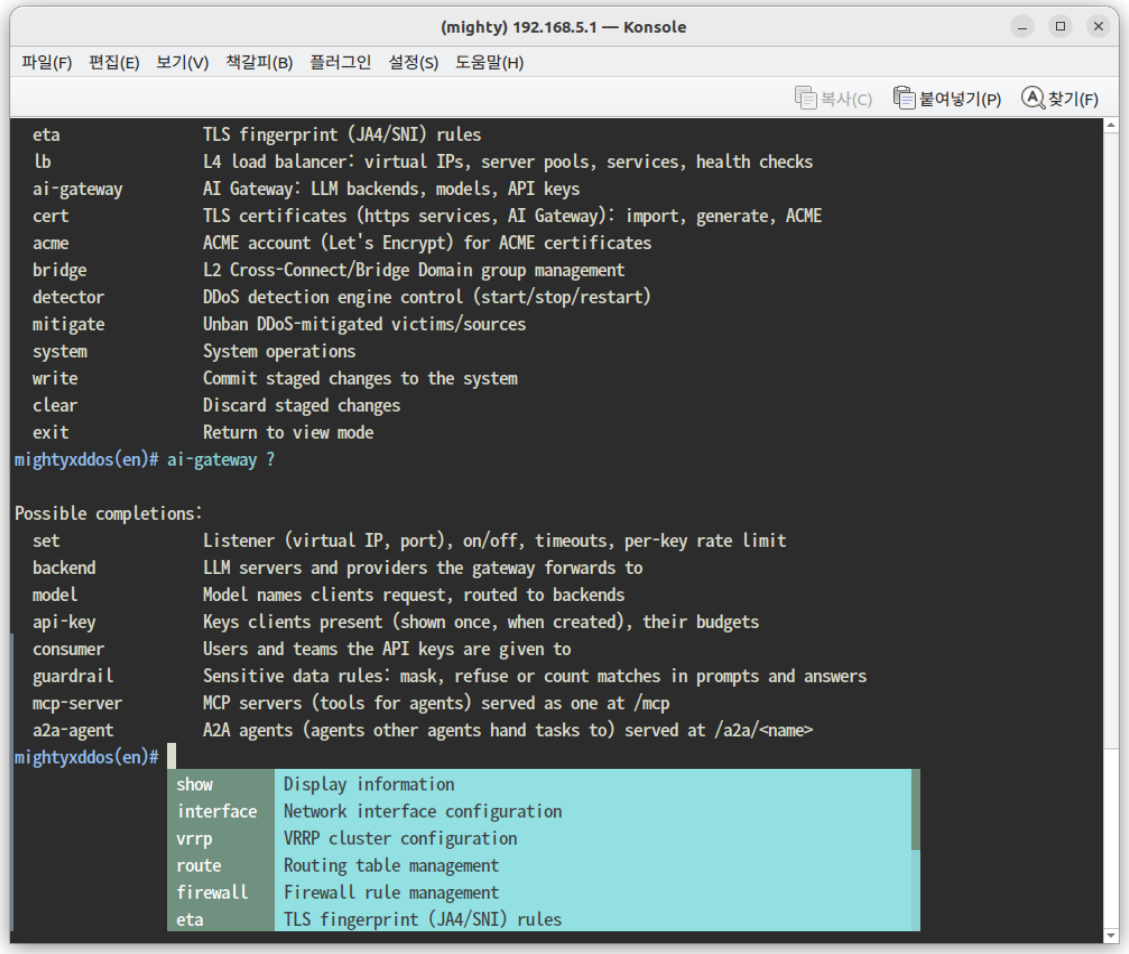


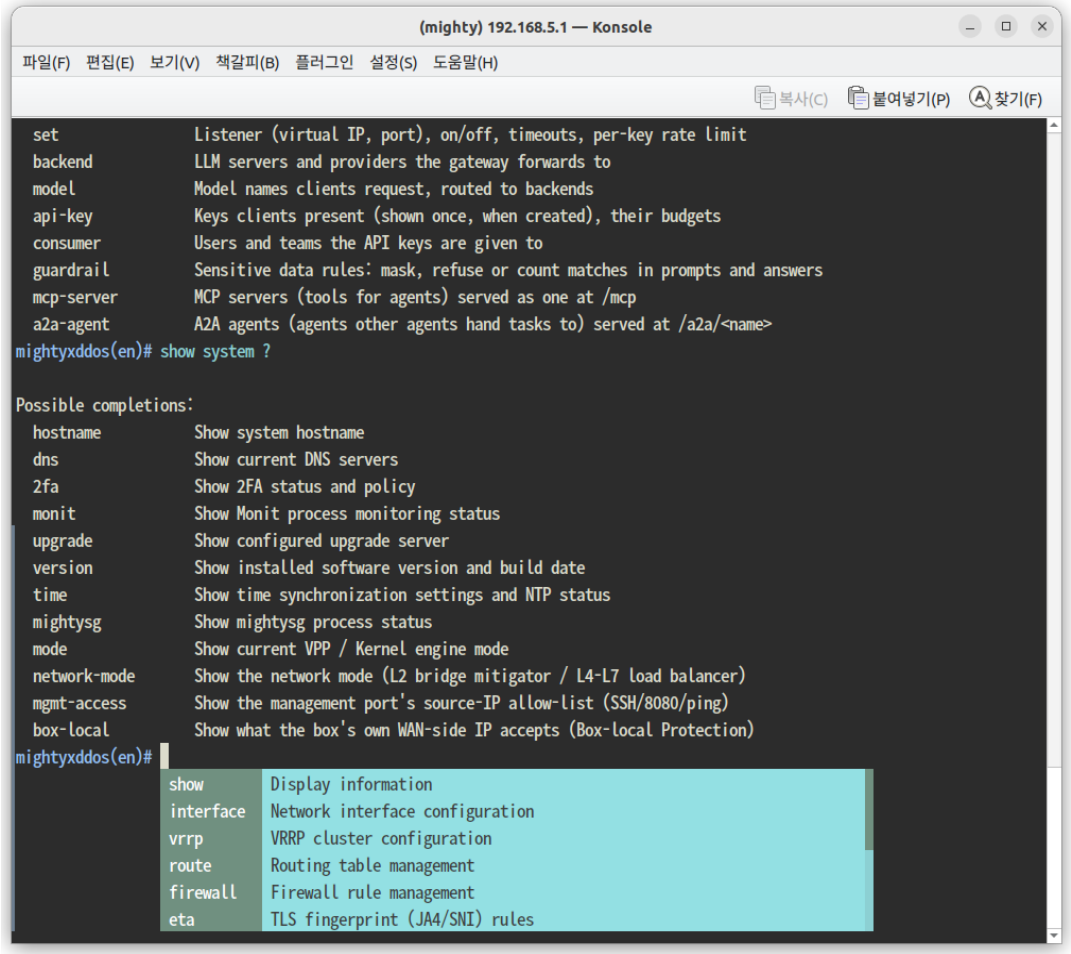
4. MightyADC(5) – CLI(2)





4. MightyADC(5) – CLI(3)





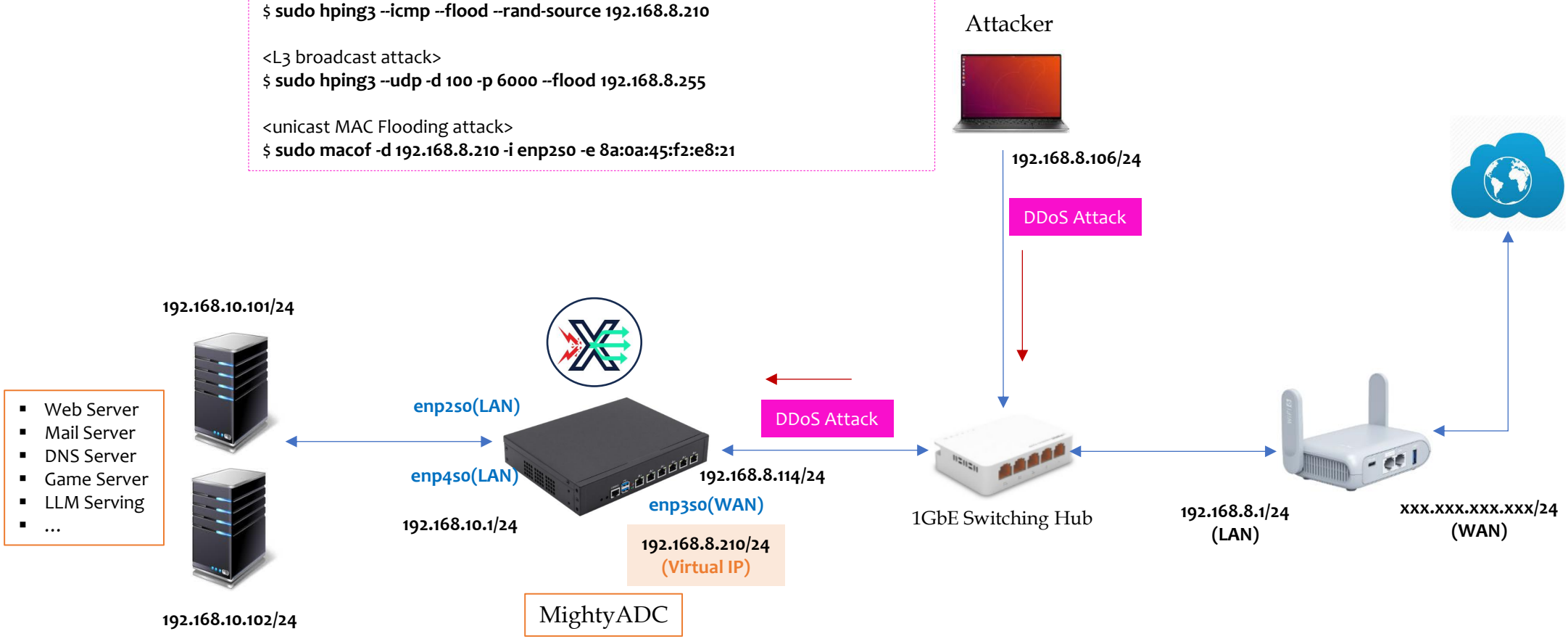
Appendix

A. Testbed(1) – TCP/UDP/ICMP Flooding Test

<TCP/UDP/ICMP Flooding attacks – random source IPs>
$ **sudo hping3 -S -p 443 --flood --rand-source 192.168.8.210**
$ **sudo hping3 -udp --flood -p 6000 --rand-source 192.168.8.210**
$ **sudo hping3 --icmp --flood --rand-source 192.168.8.210**

<L3 broadcast attack>
$ **sudo hping3 --udp -d 100 -p 6000 --flood 192.168.8.255**

<unicast MAC Flooding attack>
$ **sudo macof -d 192.168.8.210 -i enp2s0 -e 8a:0a:45:f2:e8:21**

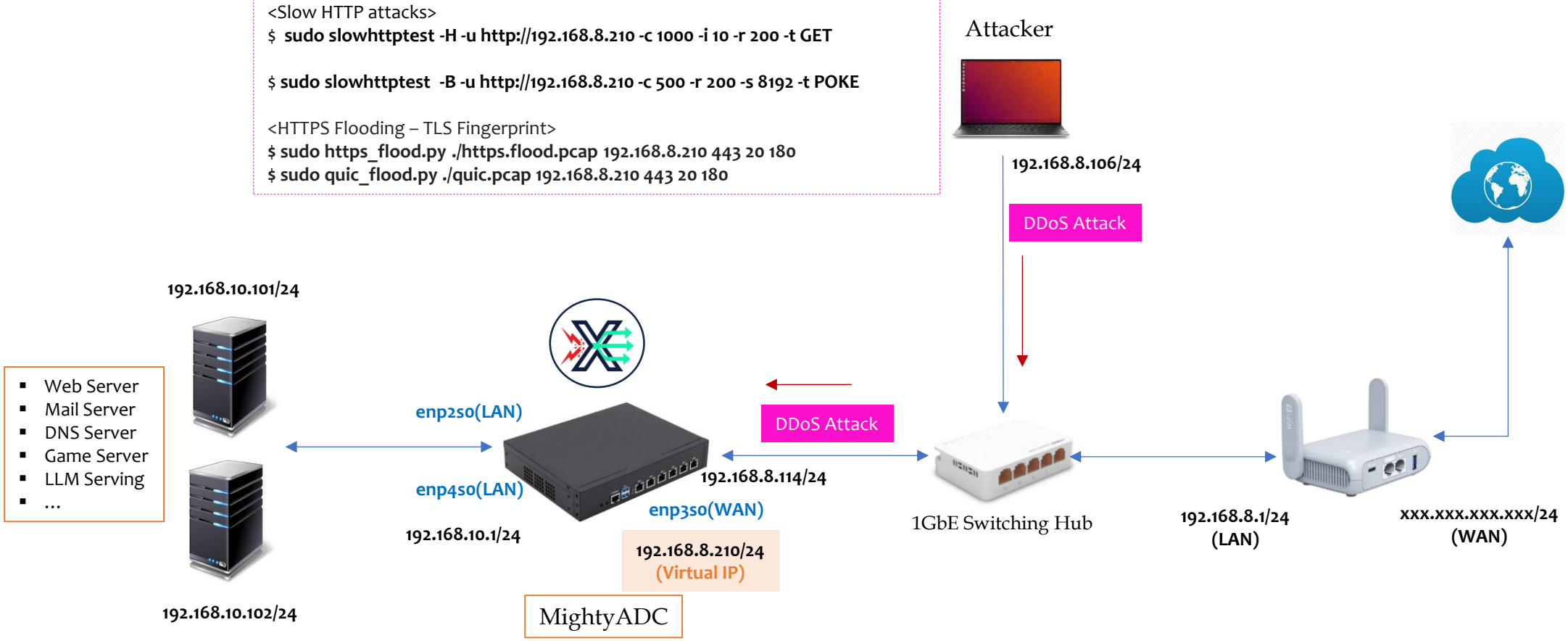


A. Testbed(2) – Slowloris and HTTPS Flooding Test

<Slow HTTP attacks>
$ **sudo slowhttptest -H -u http://192.168.8.210 -c 1000 -i 10 -r 200 -t GET**

$ **sudo slowhttptest -B -u http://192.168.8.210 -c 500 -r 200 -s 8192 -t POKE**

<HTTPS Flooding – TLS Fingerprint>
$ sudo https_flood.py ./https.flood.pcap 192.168.8.210 443 20 180
$ sudo quic_flood.py ./quic.pcap 192.168.8.210 443 20 180



B. References

- [1] https://ebpf.io/
- [2] https://fastnetmon.com/
- [3] https://www.haproxy.org/
- [4] https://agentgateway.dev/
- [5] vLLM : https://vllm.ai/
- [6] SGLang: https://www.sglang.io/
- [7] Ollama: https://ollama.com/
- [8] MAX: https://www.modular.com/
- [9] https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro
- [10] https://github.com/a2aproject/a2a
- [11] https://a2a-protocol.org/latest/
- [12] https://static.googleusercontent.com/media/research.google.com/en//pubs/archive/44824.pdf
- [13] And Google and Gemini~

Faster Connectivity, Flawless Security

©opyright 2026 MightyLink Lab., All rights reserved.

